

Learn Java for FTC

Alan G. Smith

May 11, 2026

Cover Photo Credit: [Nastassia Bas on 123rf.com](#)

All Rights Reserved.

FIRST Tech Challenge, and FTC are registered trademarks of For Inspiration and Recognition of Science and Technology (FIRST) which does not sponsor, authorize, or endorse this book.

REV, REV Control Hub, REV Expansion Hub, and Rev Robotics are trademarks of Rev Robotics which does not sponsor, authorize, or endorse this book.

Learn Java for FTC

Copyright © 2020 - 2024 Alan G. Smith. All Rights Reserved.

The author can be contacted at: alan@randomsmiths.com

The hardcopy of the book can be purchased from Amazon at <https://www.amazon.com/dp/B08DBVKXLZ>

The most recent PDF is free at <https://github.com/alan412/LearnJavaForFTC>

ISBN: 9798644009886

This book is dedicated to:

My wife who after suffering through my first book
encouraged me to write this one.

My FTC team that excites me about teaching

My father who spent many hours with me on the Vic 20,
Commodore 64, and the robotic arm science project.
Without his investment, I wouldn't be the engineer I am
today.

FTC coaches everywhere that teach their students to think
well and work hard

*Whatever you do, work at it with all your heart, as working
for the Lord, not for men.
Colossians 3:23 (NIV 1984)*

Contents

1.	Introduction	1
1.1.	Hardware	1
1.1.1.	Robot Controller	1
1.1.2.	Programming Board	1
1.1.3.	Driver Station	2
1.2.	Our first OpMode	2
1.2.1.	Some terminology	2
1.2.2.	What is an OpMode?	3
1.2.3.	Parts of an OpMode	3
1.2.4.	Hello, World	3
1.3.	Now you try	6
1.4.	Comments	8
1.5.	Sending to the Robot Controller	9
1.6.	Gotchas	10
1.7.	Exercises	10
2.	Variables and Data Types	13
2.1.	Primitive Data Types	13
2.2.	String	15
2.3.	Scope	16
2.4.	Exercises	16
3.	Gamepad and basic math	17
3.1.	Basic Math	18
3.2.	Other assignment operators	20
3.3.	Exercises	20
4.	Making decisions	21
4.1.	If	21
4.2.	Else	23
4.2.1.	Else if	23
4.3.	Combinations	24
4.4.	While	25

Contents

4.5.	For	26
4.6.	Exercises	27
5.	Class Members and Methods	29
5.1.	Class Members	29
5.2.	Class Methods	30
5.2.1.	Return Types	31
5.2.2.	Parameters	31
5.2.3.	Special Methods: Constructors	32
5.2.4.	Another special method: toString	33
5.3.	Controlling access- Keep your private things private	34
5.4.	Creating your own classes	34
5.5.	static	39
5.6.	Exercises	40
6.	Our first hardware	41
6.1.	Configuration file	41
6.2.	Mechanisms	43
6.3.	OpMode	45
6.4.	Making changes	46
6.5.	Exercises	47
7.	Motors	49
7.1.	Editing Configuration File	49
7.2.	Mechanisms	50
7.3.	OpMode	53
7.4.	Motor as Sensor	53
7.5.	Motors and Sensors together	56
7.6.	Motors and Gamepads	57
7.7.	Exercises	58
8.	Servos	59
8.1.	Configuration File	59
8.2.	Mechanisms	59
8.3.	OpMode	61
8.4.	Exercises	62
9.	Analog Sensors	63
9.1.	Configuration File	63
9.2.	Mechanisms	63
9.3.	OpMode	65

9.4.	Exercises	66
10.	Color and Distance Sensors	67
10.1.	Configuration File	67
10.2.	Mechanisms	67
10.3.	OpMode	69
10.4.	Exercises	71
11.	Gyro (IMU)	73
11.1.	Configuration File	73
11.2.	Mechanisms	73
11.3.	OpMode	76
11.4.	Exercises	76
12.	Dealing with State	77
12.1.	A simple example	77
12.2.	Autonomous state - Example	79
12.2.1.	Using the switch statement	81
12.2.2.	Switch with strings	83
12.2.3.	Enumerated types	84
12.3.	It's all relative	86
12.4.	Exercises	88
13.	Arrays	91
13.1.	ArrayList	92
13.1.1.	Making your own generic class	93
13.2.	Exercises	93
14.	Inheritance	95
14.1.	Isa vs. hasa	96
14.2.	So why in the world would you use this?	97
14.3.	Exercises	104
15.	Rumble with Gamepad	105
15.1.	Exercises	107
16.	Computer Vision	109
16.1.	April Tags	109
16.1.1.	The Opmode	110
16.2.	The empty processor	113

Contents

16.3.	Our first vision processor	115
16.3.1.	The processor	115
16.3.2.	The opmode	117
16.3.3.	Bonus - using EOCVSim (Optional)	118
16.4.	Expanding to 3 rectangles	119
16.5.	Actual computer vision...	122
16.5.1.	The opmode	125
16.6.	Exercises	126
17.	Javadoc	127
17.1.	Exercises	129
18.	Finding things in FTC SDK	131
18.1.	Exercise	131
19.	A few other topics	133
19.1.	Math class	133
19.2.	final	134
19.3.	Make telemetry prettier	135
19.4.	Interfaces (implements)	136
19.4.1.	When to use an interface instead of an abstract class?	137
19.5.	Exercises	137
20.	Making Robots Drive	139
20.1.	2 motor drive	139
20.1.1.	Two Motor Drive Mechanism	139
20.1.2.	OpMode	141
20.2.	4 motor mecanum drive	142
20.2.1.	Mecanum Mechanism	143
20.2.2.	Robot oriented driving	146
20.2.3.	Field oriented driving	147
20.3.	Exercises	149
21.	Some hardware to help with Odometry	151
21.1.	OctoQuad	151
21.1.1.	What is it?	151
21.1.2.	Using it simply	151
21.1.3.	Using it to get multiple encoders	153
21.1.4.	Using the cached attribute for clean programming	154
21.1.5.	Other features	158
21.1.5.1	Velocity	158

21.1.5.2	Absolute encoders	158
21.2.	Sparkfun Optical Tracking Odometry Sensor	158
21.2.1.	What is it?	158
21.2.2.	Using it	159
21.3.	GoBilda Pinpoint	162
21.4.	Exercises	165
22.	LEDs - Adding some bling feedback...	167
22.1.	REV Digital LED Indicator	167
22.2.	Sparkfun QWIIC LED Stick	168
22.3.	REV Blinkin	170
22.4.	Exercises	173
23.	Limelight 3A	175
23.1.	Simple color example	175
23.1.1.	On the Limelight	175
23.1.2.	Your Java Code	178
23.1.3.	Changing Limelight Pipeline	180
23.1.4.	Swapping between pipelines	180
23.2.	Localization with AprilTags	182
23.2.1.	On the Limelight	182
23.2.2.	Your Java Code	183
23.3.	Exercises	184
24.	Introduction to Control Theory	185
24.1.	Open Loop Control	186
24.2.	Closed Loop Control - Intro	189
24.3.	Closed loop control - Bang Bang	189
24.3.1.	Adding Hysteresis (Dead band)	191
24.3.2.	Using a limit switch to improve!!	192
24.4.	Closed loop control - Proportional	194
24.5.	Closed loop control - Full PID(f)	194
24.5.1.	Tuning your PID	195
24.5.2.	When to use F term	197
24.5.3.	Improving your PIDF	197
24.5.4.	Built-in vs Roll your own	198
24.5.4.1	Built-in PIDF	198
24.5.4.2	How to decide	200
24.6.	For more information	200
24.7.	Exercises	200

Contents

25. Behavior Trees (another way to handle auto)	201
25.1. Execution Nodes	201
25.1.1. Code for simplest Behavior Tree	201
25.2. Control Flow Nodes	204
25.2.1. Sequence	204
25.2.2. Failover	208
25.2.3. Parallel	209
25.2.4. Combining these	209
25.2.5. NOT	210
25.3. Generating and testing your own behavior trees (the easy way)	210
25.4. Exercises	212
A. Making your own Programming Board	213
B. LinearOpMode	215
B.1. What is it?	215
B.2. Should you use it?	216
B.2.1. Benefits of LinearOpMode	216
B.2.2. Drawbacks of LinearOpMode	217
C. Sample Solutions	219
D. Credits	221
Index	223

1. Introduction

In coaching an FTC team¹, I found that students wanted to be effective coders but had trouble figuring out where to start. When they took online courses, they ended up learning a lot of things that weren't helpful for FTC (or even usable). In addition, many of the online sources and even books teach bad habits. I started this as some slides for my team, but decided it would be better as a book that could be shared widely.

You'll notice that throughout the book some words are written in a different font like `this`. That means that it is code that needs to be exactly like that (capitalization, etc)

1.1. Hardware

1.1.1. Robot Controller

The Robot Controller, often abbreviated “RC”, is the “brains” of your robot. It is what our programs run on. The RC can be either an Android phone or a REV Control Hub. When using an Android phone, it is connected to a REV Expansion Hub over USB. The REV Expansion Hub is what all motors, servos, and sensors connect to. A Rev Control Hub is new for the 2020-2021 FTC Season and is basically an Android phone and Expansion Hub in the same package instead of having them separate.

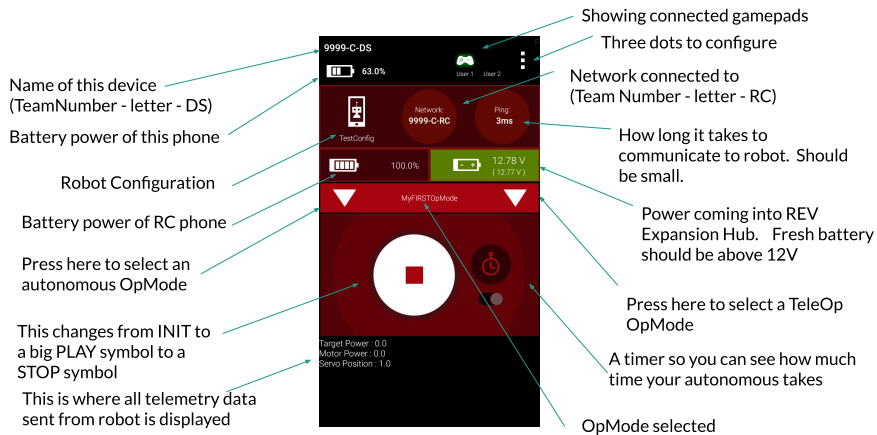
1.1.2. Programming Board

For this book, instead of a full robot we have made a simple *Programming Board* (just the electrical components that we are using) that we can use throughout the book so that we all have the same hardware. For directions on how to make your own, see [Appendix A](#).

¹Go Quantum Quacks - FTC #16072

1. Introduction

1.1.3. Driver Station



The Driver Station, often abbreviated “DS”, is an Android Phone with 1 or 2 USB gamepads connected that are used during the game to drive the robot. Above is an example driver station with descriptions for everything on it. This changes slightly from year to year.

1.2. Our first OpMode

1.2.1. Some terminology

A little terminology before we get started.

class In Java all code is grouped together in classes. We’ll discuss exactly what classes are later in [chapter 5](#). For now, just know that a class groups like code together and in Java, each class is in its own file that is named the same as the class with `.java` at the end.

method A method is a group of code within a class. Methods are the smallest group of code that can be executed. It is like a function in some languages or a MyBlock in EV3-G. We’ll talk more about this later in [section 5.2](#).

package A directory in JAVA. It is where the code is located. Files in the same package have special privileges with each other. We’ll talk about this in [section 5.3](#). And yes, a package can have packages within it.

1.2.2. What is an OpMode?

In FTC, An OpMode² is a program for our robot. We can have multiple OpModes. They are all stored in the TeamCode package.

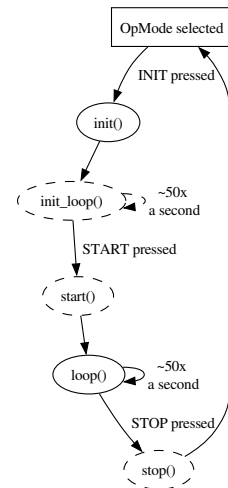
1.2.3. Parts of an OpMode

OpModes are required to have two methods:

1. `init()` - This is run **once** when the driver presses INIT.
2. `loop()` - This is run **repeatedly** after driver presses PLAY but before STOP.

In addition, there are three optional methods. These are less common but can be very useful.

1. `init_loop()` - This is run **repeatedly** after driver presses INIT but before PLAY.
2. `start()` - This is run **once** when the driver presses PLAY.
3. `stop()` - This is run **once** when the driver presses STOP.



If you look over on the right, you'll see a diagram that explains roughly how it works. The solid ovals are required and the dashed ones are optional. After `stop()` is executed it goes back to the top.

I know this seems strange, but I promise it will make more sense as we continue.

1.2.4. Hello, World

Traditionally, the first program written in every programming language simply writes "Hello, World!" to the screen. But instead of writing to the robot's screen, we'll write to the screen on the Driver Station. (Throughout this book we will show the program in its entirety first, and then explain it afterwards. So if you

²You will likely run across LinearOpMode as many teams use it. There is a discussion in [Appendix B](#) for why we don't use it but it is probably best left for the end.

1. Introduction

see something that doesn't make sense, keep reading and hopefully it will be cleared up.)

Listing 1.1: HelloWorld.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class HelloWorld extends OpMode {
8     @Override
9     public void init() {
10         telemetry.addData("Hello", "World");
11     }
12
13     @Override
14     public void loop() {
15
16     }
17 }
```

Here is a breakdown of what this program does.

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
```

If you are working in Android Studio, you won't have to enter any of these lines as it will add them for you. Line 1 basically says where this file is located. 3 and 4 bring in code from the FTC SDK (Software Development Kit) so we can use them.

```
6 @TeleOp()
```

This is **CRITICAL**. If you forget this line, it won't show up on the Driver-Station as an OpMode to select from. Any line that starts with an @ is called an annotation. You can choose from `@TeleOp()` or `@Autonomous()`. You can optionally give it a name and a group, but if you leave those off then it will use your class name as the name. This works well enough, so we'll typically leave those pieces out. Another annotation that you'll see commonly is `@Disabled`. If you have that, then your code will compile but it won't be shown in the list of OpModes.³

³Our team often does that for test code that we don't want to distract us during a tournament but is VERY helpful to have where we can make it available quickly.

```

7  public class HelloWorld extends OpMode {
17 }

```

`public` - means others can see it. Required for OpModes. We'll discuss this more in [section 5.3](#).

`class` - means we are defining a class

`HelloWorld` - this is the name of the class. It must be the same as the filename (except the filename has `.java` on it). By convention, it should be started with a capital letter and each new word is a capital letter (Pascal case). We'll talk more about classes in [chapter 5](#).

`extends OpMode` - This means the class is a child of `OpMode`. A child gets all of the behavior of its parent and then can add (or replace) functionality. We'll talk about what this means in [chapter 14](#).

a class is defined from the opening curly brace “{” to the closing curly brace “}”

```

8      @Override
9      public void init() {
10          telemetry.addData("Hello", "World");
11      }

```

`@Override` tells the compiler that we are meaning to override (replace) functionality in our parent class. We'll talk more about this in [chapter 14](#).

`public` means this method is callable from outside the class. We'll discuss this more in [section 5.3](#)

`void` means it doesn't return anything. We'll talk about return types in [subsection 5.2.1](#)

`init` is the name of a method. We'll talk more about methods in [section 5.2](#)

Inside of the parenthesis are any parameters passed in or none. (as in this case) We'll talk about parameters in [subsection 5.2.2](#)

The method is defined from the opening curly brace “{” to the closing curly brace “}”

`telemetry.addData(caption, value);` This is very cool because it sends data to the driver station which lets us debug problems. In this case we sent back a string (a group of characters - we'll talk about strings in [section 2.2](#)), but you can also send back numbers or variables. You'll notice that this ends in a semi-colon “;” All statements in JAVA either end with a semi-colon or have a set of curly braces attached.

```

13      @Override
14      public void loop() {

```

1. Introduction

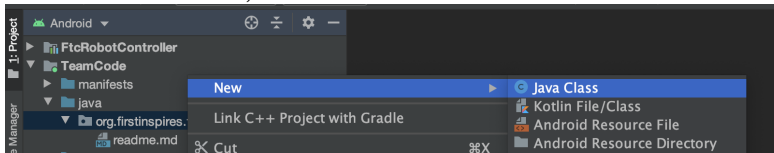
```
15  
16 }
```

This looks much the same as our `init()` method, but there is no code in the `loop()` method, so the program won't do anything here. (We included it because it is required.)

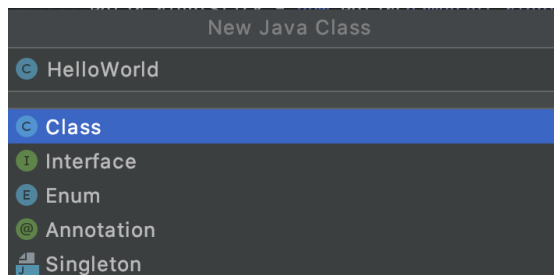
1.3. Now you try

Before you do this, you need to have your phones ready to go and Android Studio installed with a copy of the FTC SDK. For instructions, see the FTC document *Android Studio Guide*⁴.

You'll learn the best here if you type in the examples (and you'll get faster at Android Studio). While this may seem like it slows you down, it helps you learn faster. This is the only time in the book I'll mention "Now you try". For the rest, I suggest you type it in AFTER we have explained what it does and then try it. To start with, change the project area to show "Android" (by using the dropdown). If you are wondering why your Android Studio is white colored while mine is Dark, that is because I use the built-in theme "Darcula".⁵



1. Right click on `org.firstinspires.ftc.teamcode` under TeamCode
2. Select New > Java Class



3. (If you are using Android Studio 4.x, it will look like this....)

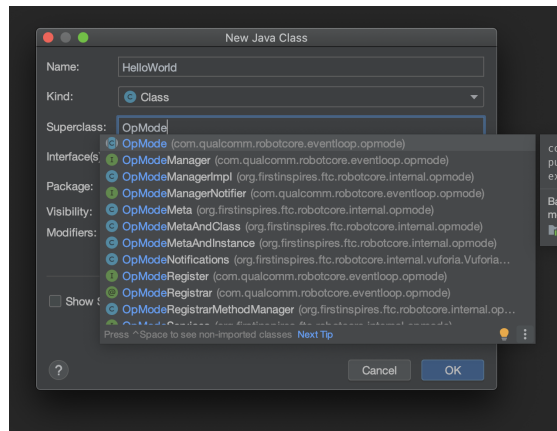
⁴https://ftc-docs.firstinspires.org/programming_resources/android_studio_java/install/install.html

⁵To change your theme click File > Settings from the menu bar (or Android Studio > Preferences on macOS). Go to Appearance under Appearance and Behavior, and you'll see Theme.

- a) Fill in the name as HelloWorld
- b) Press “OK”
- c) If you get another dialog box with a bunch of blanks, leave them blank and press “OK”
- d) You’ll get a listing that will look like this

```
package org.firstinspires.ftc.teamcode;

public class HelloWorld {
}
```



4. (If you are using Android Studio 3.x, it will look like this...)

- a) Fill in the name as HelloWorld
- b) Fill in the Superclass as OpMode. (We’ll explain what this means in [chapter 14](#)) As you type it in, it will show you the matches. When you select it, it will fill in as `com.qualcomm.robotcore.eventloop.opmode`
- c) Press “OK”. You’ll get a file that will be like this:

```
package org.firstinspires.ftc.teamcode;
import com.qualcomm.robotcore.eventloop.opmode.OpMode;

public class HelloWorld extends OpMode {
}
```

- d) It will have a red squiggle line under the class declaration. That is because you haven’t implemented the two required methods yet. You haven’t done anything wrong.

1. Introduction

Make yours look like the HelloWorld.java file in Listing 1.1 earlier. (You can start at line 6 and you'll watch it make the import statements as you type)

As you start typing, you'll notice that Android Studio is giving suggestions. You can either click on the one you want, or when it is at the top of the list then press tab.

This is the same pattern you'll follow for all OpModes in this book.

1.4. Comments

So far our programs have been only for the computer. But it turns out that you can put things in them that are only for the human readers. You can (and should) add comments to the program which the computer ignores and are for human readers only. Comments should explain things that are not obvious from the code such as why something is being done. In general, comments should explain why and not what. Please don't just put in a comment that repeats the code.

Java supports two forms of comments:

1. A single line comment. It starts with a `//` and tells the computer to ignore the rest of the line.

```
// This is a comment
```

2. The block comment style. It starts with a `/*` and continues until a `*/` is encountered. This can cross multiple lines. Below are three examples.

```
/* This is also a comment */

/* So is this */

/*
 * And
 * this
 * as
 * well */
```

In addition, there is a subset of this type of comment called a javadoc that we'll talk about in [chapter 17](#). This starts on a line with a `/**` and then goes until it sees `*/`. This is used for automatically creating documentation from your comments.

```
/**
 * This is a javadoc comment
 */
```

Here is what it looks like with comments added.

Listing 1.2: HelloWorldCommented.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class HelloWorldCommented extends OpMode {
8     /**
9      * This is called when the driver presses INIT
10     */
11     @Override
12     public void init() {
13         // this sends to the driver station
14         telemetry.addData("Hello", "World");
15     }
16
17     /**
18      * This is called repeatedly while OpMode is playing
19     */
20     @Override
21     public void loop() {
22         // intentionally left blank
23     }
24 }
```

1.5. Sending to the Robot Controller

1. Make sure your phones are setup as it describes in the FTC document *Configuring Your Android Devices*⁶ and that they can see each other.
2. Connect the Robot Controller to the computer.
3. Press the green play arrow next to the name of the device on the top toolbar.

⁶https://github.com/ftctechnh/ftc_app/wiki/Configuring-Your-Android-Devices

1. Introduction

4. Wait until you hear the sound from the Robot Controller and the Driver Station.
5. Now press the right arrow on the driver station to see the list of TeleOp OpModes. (The arrow on the left shows the list of Autonomous OpModes)
6. Select HelloWorld, and then press the big INIT button.
7. You should see “Hello: World” in the area where the Telemetry data is reported.

1.6. Gotchas

If your program won't compile (or it doesn't do what you expect), here are a few things to check that often confuse people:

- Java is case sensitive. In other words, `myVar` is different than `MyVar`
- Whitespace (spaces, tabs, blank lines) is all collapsed to the equivalent of a single space. It is for the human reader only.
- Blocks of code are encapsulated with curly braces `{` and `}`
- Every open parenthesis `(` must have a matching close parenthesis `)`
- Each program statement needs to end with a semicolon `;`. In general, this means that each line of your program will have a semicolon. Exceptions are:
 - Semicolons are not used when a code block follows - for example the class or method declarations we have seen so far
 - Semicolons (like everything) are ignored in comments
 - Semicolons are not used after the end curly brace. `}`

1.7. Exercises

After you have done the exercise, send it to the robot controller to make sure it works.

There are sample solutions in [Appendix C](#). However, you should struggle with them first and only look there when you are stuck. If you end up looking there, you should make up another exercise for yourself.

1. Change the code so that instead of saying “Hello: World” it says Hello and then your name.
2. Change the OpMode so it shows up in the Autonomous section of the Driver Station instead of the Teleop section.

2. Variables and Data Types

A variable is a named location in memory where we can store information. While we don't have to, by convention we name variables starting with a lower case letter and then every word after that starts with a capital letter.¹ For example: `motorSpeed` or `gyroHeading`. In Java, we specify what type of information we are storing. *Primitive datatypes* are types that are built-in to Java.

We must declare a variable before we can use it. Declaring a variable requires that we specify the type and name. It is always followed by a `;` (semi-colon).

```
// datatype name
int teamNumber;
double motorSpeed;
boolean touchSensorPressed;
```

The above variable types are `int`, `double`, and `boolean` (These are the three you'll use most often in FTC). We'll discuss these and the other primitive datatypes in the next section.

In Java, if you don't assign a value to a variable when you create it then it starts out being equal to 0. (or `false` for `boolean`)

To assign a value to a variable, you use the `=` operator like this:

```
teamNumber = 16072;
motorSpeed = 0.5;
touchSensorPressed = true;
```

You can assign a value to a variable multiple times and it will be equal to what you assigned it to most recently.

It's common to declare a variable and assign the value in one line!

For example, to assign 0.5 to a variable named `motorSpeed` of type `double`, we write:

```
double motorSpeed = 0.5;
```

2.1. Primitive Data Types

There are 8 primitive data types in Java:

¹This is called camelCase because the upper case letters look like humps.

2. Variables and Data Types

1. `byte` - from the range -128 to 127
2. `char` - for holding a single unicode character
3. `short` - a smaller integer (almost never used in FTC)
4. `int` - this is short for integer. It is for numbers with no decimal.²
5. `long` - this is a larger integer. You can use it when you are concerned about running out of room in an `int`.³
6. `float` - this is for floating point numbers. It is smaller than a `double` so we typically convert to a `double`.
7. `double` - this is for floating point numbers. It can hold numbers with decimals.⁴
8. `boolean` - this can be either `true` or `false`. (Yes, it contains one or the other of these values.)

In the code below, there are examples of the three most typical primitive types for FTC.

Listing 2.1: `PrimitiveTypes.java`

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class PrimitiveTypes extends OpMode {
8     @Override
9     public void init() {
10         int teamNumber = 16072;
11         double motorSpeed = 0.5;
12         boolean touchSensorPressed = true;
13
14         telemetry.addData("Team Number", teamNumber);
15         telemetry.addData("Motor Speed", motorSpeed);
16         telemetry.addData("Touch Sensor", touchSensorPressed);
17     }
18
19     @Override
```

²It is also limited in the range from +2,147,483,647 to -2,147,483,648

³It is limited in the range from +9,223,372,036,854,775,807 to -9,223,372,036,854,775,808

⁴while technically it is limited, it is so large you can think of it as unlimited

```

20     public void loop() {
21
22     }
23 }

```

In the three lines below you'll see them defined. Notice how they all follow the same pattern:

```

10     int teamNumber = 16072;
11     double motorSpeed = 0.5;
12     boolean touchSensorPressed = true;

```

They are sent to the driver station using `telemetry.addData`. Again, you'll notice that they all follow the same pattern.

```

14     telemetry.addData("Team Number", teamNumber);
15     telemetry.addData("Motor Speed", motorSpeed);
16     telemetry.addData("Touch Sensor", touchSensorPressed);

```

2.2. String

A `String` is for holding text. You might be wondering why it is capitalized when all of the other data types we have seen so far aren't. This is because `String` is really a class. By convention, class names start with a Capital letter and then every other word is also capitalized.⁵ We'll talk more about classes in [chapter 5](#).

In the code below, there is an example of using a `String` data type.

Listing 2.2: `UseString.java`

```

1  package org.firstinspires.ftc.teamcode;
2
3  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6  @TeleOp()
7  public class UseString extends OpMode {
8      @Override
9      public void init() {
10         String myName = "Alan Smith";
11
12         telemetry.addData("Hello", myName);
13     }

```

⁵This is called `PascalCase` because it was popularized by one of the lead designers of Turbo Pascal.

2. Variables and Data Types

```
14
15     @Override
16     public void loop() {
17
18     }
19 }
```

You'll notice that the pattern here is similar with `datatype variableName;` or `datatype variableName = initialValue;`

2.3. Scope

This may seem unimportant, but you'll see why it matters later. A variable is only usable within its scope. Its scope is from where it is declared until the end of the block it is defined within. A block is defined as any set of open and close curly braces. `{ }`

A simple example:

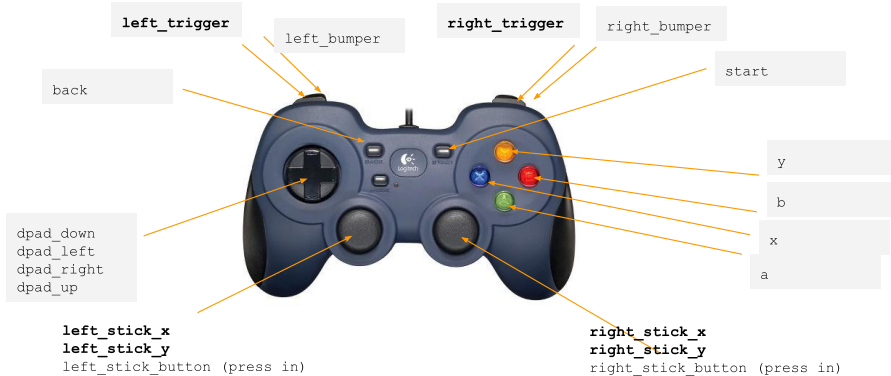
```
public void loop(){
    int x = 5;
    // x is visible here
    {
        int y = 4;
        // x and y are visible here
    }
    // only x is visible here
}
```

2.4. Exercises

1. Change the `String` to have your name instead of mine in the code in [section 2.2](#)
2. Add a variable of type `int` that is called `grade` that has your grade in it. Use `telemetry` to send that to the driver station.

3. Gamepad and basic math

We can access the gamepads connected to the driver station from our OpMode. They are of the Gamepad class. We'll talk more about classes in [chapter 5](#). Since there are two of them, they are called `gamepad1` and `gamepad2`.¹ The buttons on the gamepad are all boolean (`true` if they are pressed, `false` if they aren't). The d-pad is exposed as four buttons.² The joysticks are double with values between -1.0 and 1.0 (0.0 means in the center). There is one for each *x* (side to side) and one for each *y* (up and down). The *x* is negative to the left and positive to the right. For strange reasons, up is negative and down is positive. The left trigger and right trigger are also double with values between 0.0 and 1.0 (0.0 means not pressed, 1.0 means fully pressed). To get to these we use `variableName.memberName`. Below, we show what the memberNames are for all of the parts of the gamepad. In the image below, the ones that are bolded are double (Sometimes we call these analog and the ones that are binary - digital)



In the code below is an example of reading the Gamepad. The reason it is in `loop()` is because we want to update the telemetry as the gamepad changes. You'll remember that `loop` is called over and over again (approximately 50 times a second)

Listing 3.1: `GamepadOpMode.java`

¹You might be wondering where these are declared. We'll talk about that in [chapter 14](#)

²Technically you can pull out the analog but that is a lot of work and not typically done.

3. Gamepad and basic math

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class GamepadOpMode extends OpMode {
8     @Override
9     public void init() {
10    }
11
12    @Override
13    public void loop() {
14        telemetry.addData("Left stick x", gamepad1.left_stick_x);
15        telemetry.addData("Left stick y", gamepad1.left_stick_y);
16        telemetry.addData("A button", gamepad1.a);
17    }
18 }
```



You have to press the “Start” and “A” *simultaneously* on a gamepad to get the driver station to recognize gamepad1 (and “Start” and “B” for gamepad2). Once the gamepad has been recognized the gamepad icon in the upper right corner of the DS (Driver Station) will be illuminated.

We talk about the new functionality (as of FTC SDK 6.0) of using rumble on gamepads that support it in [chapter 15](#).

3.1. Basic Math

In the last section, we talked about how to read a gamepad. You probably noticed that reading the joystick gave us a number. Once something is a number, we own it. We can do any kind of math to it to get what we wanted. Below are some of the most common operators.

Math Operator	Meaning
=	assignment operator
+	addition operator
-	subtraction operator AND negative operator (So saying -x is the same thing as saying (0 - x))
*	multiplication operator
/	division operator - be aware that if you are using integers only the whole part is kept. It is NOT rounded. For example: 5 / 2 == 2 (== is how we describe two things are equal. We'll talk about it in section 4.1.)
%	modulo operator - This gives the remainder. For example: 5 % 2 == 1
(and)	These are parenthesis and they allow you to specify the order of operations just like in regular math. You can use these to tell the difference between 3 * (4 + 2) or (3 * 4) + 2 While there is a well defined order of operations, instead of memorizing that it makes more sense to use parenthesis to be specific.

Below is an example of how we might set the speed forward we want to go based off of the joystick. In this case we are limiting our speed from -0.5 to 0.5. The joystick y-value is negative when you press it up and positive when you press it down which is backwards of how most people want to drive the robot, so we “negate” the value here to flip it (so negative is positive and vice versa)

Listing 3.2: MathOpMode.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class MathOpMode extends OpMode {
8     @Override
9     public void init() {
10     }
11
12     @Override
13     public void loop() {
14         double speedForward = -gamepad1.left_stick_y / 2.0;

```

3. Gamepad and basic math

```
15     telemetry.addData("Left stick y", gamepad1.left_stick_y);
16     telemetry.addData("speed Forward", speedForward);
17 }
18 }
```

The first thing we do is create a new variable and assign to it from another variable using math.

```
14     double speedForward = -gamepad1.left_stick_y / 2.0;
```

You'll notice that then we can send that variable directly using telemetry

```
16     telemetry.addData("speed Forward", speedForward);
```

3.2. Other assignment operators

There are some shortcuts where you can combine a math operator and an assignment operator. Below are some of the most common.

Operator	Meaning	Example
++	increment	x++ means the same as x = x + 1
--	decrement	x-- means the same as x = x - 1
+=	Add and assignment	x += 2 means the same as x = x + 2
*=	Multiply and assignment	x *= 2 means the same as x = x * 2
/=	divide and assignment	x /= 2 means the same as x = x / 2
%=	modulo and assignment	x %= 2 means the same as x = x % 2

3.3. Exercises

1. Add telemetry to show the right stick of gamepad1.
2. Add telemetry to show whether the b button is pressed on gamepad1
3. Report to the user the difference between the left joystick y and the right joystick y on gamepad1.
4. Report to the user the sum of the left and right triggers on gamepad1.

4. Making decisions

4.1. If

So far our programs have executed all of their code. Control structures allow you to change which code is executed and even to execute code multiple times.

The `if` statement is the first control structure we'll talk about. Here is an example of a program using it:

Listing 4.1: `IfOpMode.java`

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class IfOpMode extends OpMode {
8     @Override
9     public void init() {
10    }
11
12     @Override
13     public void loop() {
14         if(gamepad1.left_stick_y < 0){
15             telemetry.addData("Left stick", " is negative");
16         }
17
18         telemetry.addData("Left stick y", gamepad1.left_stick_y);
19     }
20 }
```

Can you figure out what this is doing?

`if` clauses start with `if(conditionalExpression)`. They then have either a single statement or a block of code. A block of code starts with an open curly brace `{`, then it has 0 or more statement, and then a close curly brace `}`.

The code in the block is *only* executed if the conditional expression inside the parenthesis is `true`.

4. Making decisions



I **strongly** recommend using a block of code instead of a single statement. The reason why is that using a single statement can lead to unexpected errors. For example:

```
if(gamepad1.left_stick_y < 0)
    telemetry.addData("Left stick", " is negative");
    telemetry.addData("Looks like it is part of the if, but↔
        ↪ it isn't");
```

There are several conditional operators that we can use:

Operator	Meaning
==	is equal to
!=	is not equal to
<	is less than
>	is greater than
<=	is less than or equal to
>=	is greater than or equal to



A common mistake is trying to test for equality with the assignment operator = instead of the equality operator ==.

Not only can we use conditional operators, we can also use a `boolean` variable to make the decision. Here is an example:

Listing 4.2: IfOpMode2.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class IfOpMode2 extends OpMode {
8     @Override
9     public void init() {
10     }
11
12     @Override
```

```

13     public void loop() {
14         if(gamepad1.a){
15             telemetry.addData("A Button", "pressed");
16         }
17     }
18 }

```

4.2. Else

An if statement can have an else clause which handles what should be done if the if expression is false. That sounds confusing, but here is an example:

Listing 4.3: IfElseOpMode.java

```

1  package org.firstinspires.ftc.teamcode;
2
3  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6  @TeleOp()
7  public class IfElseOpMode extends OpMode {
8      @Override
9      public void init() {
10     }
11
12     @Override
13     public void loop() {
14         if(gamepad1.left_stick_y < 0){
15             telemetry.addData("Left stick", " is negative");
16         }
17         else{
18             telemetry.addData("Left stick", " is positive");
19         }
20
21         telemetry.addData("Left stick y", gamepad1.left_stick_y);
22     }
23 }

```

4.2.1. Else if

Since an else statement can have a single statement OR a block of code we can chain them together like this:

Listing 4.4: IfElseIfOpMode.java

```

1  package org.firstinspires.ftc.teamcode;

```

4. Making decisions

```
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class IfElseIfOpMode extends OpMode {
8     @Override
9     public void init() {
10    }
11
12    @Override
13    public void loop() {
14        if (gamepad1.left_stick_y < -0.5) {
15            telemetry.addData("Left stick", " is negative and large");
16        }
17        else if (gamepad1.left_stick_y < 0){
18            telemetry.addData("Left stick", " is negative and small");
19        }
20        else if (gamepad1.left_stick_y < 0.5){
21            telemetry.addData("Left stick", " is positive and small");
22        }
23        else {
24            telemetry.addData("Left stick", " is positive and large");
25        }
26        telemetry.addData("Left stick y", gamepad1.left_stick_y);
27    }
28 }
```

4.3. Combinations

Sometimes you want to test for more than one thing. For example, you may want to test if a variable is between two numbers. While you can use multiple `if` statements, it is often more convenient and readable to use logical combinations. There are four¹ simple ways that you can combine logical conditions. (and then you can combine these even further)

¹Technically there are three, but you can use the XOR bitwise operator in a similar manner. Just be careful to make sure you are operating on boolean expressions and not ones that are integers or you'll get unexpected results.

Operator	Example	Meaning
&&	(A < 10) && (B > 5)	logical AND (return true if condition A AND condition B are true, otherwise return false.)
	(A < 10) (B > 5)	logical OR (return true if condition A OR condition B is true, otherwise return false.)
!	!(A < 10)	logical NOT (return true if condition A is false, otherwise return false.)
^	(A < 10) ^ (B > 5)	XOR (return true if either A or B is true but if they both are then return false). This is used more rarely than the others but is included for completeness.



A common mistake is accidentally using the single & instead of && or using the single | instead of ||. The single versions are for doing binary arithmetic operations. That is pretty rare in your Java FTC code so we won't be talking about it in this book.

One thing that might not be obvious is that you can use these to set a value for a boolean variable. So for example:

```
boolean bVar;
bVar = !bVar;
```

When it is declared, `bVar` will be `false`. (Since all boolean variables are initialized to `false` by default.) After the line `bVar = !bVar` it will be equal to `true`.

4.4. While

A `while` loop is much like an `if` statement except for after it is done it goes back to the beginning and checks the conditional again. What if we had the amount the robot had turned, but we wanted its heading (between -180 and 180). We could use code like this:

```
while(angle > 180){
```

4. Making decisions

```
        angle -= 360;
    }
    while(angle < -180){
        angle += 360;
    }
```

The reason it takes two while clauses is because one takes care of the case where we had turned more than 180 degrees in the positive direction, and the other takes care of the case where we had turned more than 180 degrees in the negative direction.²

You might be tempted to write code like



```
while(gamepad1.a){
    // do something
}
```

That code won't work in an OpMode because gamepad1 is only updated between calls to loop()

There is also a do...while loop which executes once regardless and checks the condition at the end instead of the beginning. This is pretty rare in Java FTC code but is included here for completeness. A quick example:

```
do{
    // code goes here
    a++;
}while(a < 10)
```

4.5. For

There are two types of for loops. The traditional type looks like many programming languages, for(start; conditional; update) The start is executed once before we begin, the conditional is checked every time before we execute, the end is done at the end of EVERY time through.

```
for(int i = 0; i < 4; i++){
    // This code will happen 4 times
}
```

²If we were doing this for real, we would do it in radians. But we used degrees here to make the concept simpler.

This is often used, but in many cases it is to go through an array and you are better off using a `for-each` that we'll talk about when we talk about arrays in [chapter 13](#).

4.6. Exercises

1. Make a “turbo button”. When `gamepad1.a` is not pressed, multiply the joystick by 0.5 and when it is pressed multiply by 1 and report to the user as Forward Speed.
2. Make a “crazy mode”. When `gamepad1.a` is pressed, report X as Y and Y as X. When it isn't pressed, report the joystick as normal....

5. Class Members and Methods

A `class` is a model of something. It can contain data (members) and functions (methods). Whenever you create a `class`, it becomes a data type that people can make variables of that type. You can think of a `class` like a blueprint that can be used to make any number of identical things. (called “objects”) For example, the `String` data type is a `class` but we can have multiple objects of type `String` in our programs. Remember that we name classes starting with a Capital letter and then every other word in the class name is also capitalized.¹

5.1. Class Members

So far, we have had variables in our methods but we can also have them belong to our class. To have them belong to our class, they need to be within the class body but outside of every method body. By convention, they are at the beginning of the class but they don’t have to be. If they are in our class, then every method in our class can use them and when they get changed everyone sees the new value. However, every object (copy) has its own member variables²

Listing 5.1: `ClassMemberOpMode.java`

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class ClassMemberOpMode extends OpMode {
8     boolean initDone;
9
10    @Override
11    public void init() {
12        telemetry.addData("init Done", initDone);
13        initDone = true;
14    }
```

¹Remember this is called Pascal Case.

²unless they are declared `static` which means they are shared between all objects of the class. We’ll talk about this in [section 5.5](#).

5. Class Members and Methods

```
15
16     @Override
17     public void loop() {
18         telemetry.addData("init Done", initDone);
19     }
20 }
```

Even though `initDone` gets updated in `init()`, nothing sends it to the driver station until `loop()` gets called for the first time.

You can use the `this` keyword to unambiguously say you are referring to the class member, but if there isn't a variable with the same name in your method then you can leave it off. That would look like `this.initDone`.

5.2. Class Methods

We can create new methods. A method has a return type (which is any data type), a name, and can take 0 or more parameters. A parameter is a way you can pass information into a method. Each parameter has a data type and a name. Inside the method, it is just like you had a variable defined inside the method with that data type and name. (but it received its value from whomever called the class method.)

By convention we name methods starting with a lowercase letter and then having each additional word in the name start with an uppercase letter³ After its parameters, there is the method body which goes from the opening curly bracket `{` to the close curly bracket `}`.

Listing 5.2: `ClassMethodOpMode.java`

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class ClassMethodOpMode extends OpMode {
8
9     @Override
10     public void init() {
11     }
12
13     double squareInputWithSign(double input){
14         double output = input * input;
15         if(input < 0){
```

³Remember this is called camelCase

```

16         output = output * -1;
17     }
18     return output;
19 }
20
21 @Override
22 public void loop() {
23     double leftAmount = gamepad1.left_stick_x;
24     double fwdAmount = -gamepad1.left_stick_y;
25
26     telemetry.addData("Before X", leftAmount);
27     telemetry.addData("Before Y", fwdAmount);
28
29     leftAmount = squareInputWithSign(leftAmount);
30     fwdAmount = squareInputWithSign(fwdAmount);
31
32     telemetry.addData("After X", leftAmount);
33     telemetry.addData("After Y", fwdAmount);
34 }
35 }

```

5.2.1. Return Types

The return type is simply the data type in front of the name. You can also say that a method doesn't return anything. In that case, instead of the data type you put the keyword `void` before the name. To return the value you use the return statement. It is simply `return <value>`; You can return a variable or a constant (typed in number, string, etc.) You can see this done in the example above. As soon as the return keyword is executed the method returns to whomever called it.

If you have a class method that returns `void`, then you can either have a return with nothing after it like `return;` or you can omit the return statement and it will return at the end of the code.

5.2.2. Parameters

You probably noticed that the name had an open parenthesis (after it. Then each parameter is listed like a variable (except no default assignment allowed). If there is more than one parameter, they are separated by a comma , Then at the end of the parameters is a close parenthesis)

So some examples of methods:

```

// returnType name(parameters)
double squareInputWithSign(double input){

```

5. Class Members and Methods

```
double output = input * input;
if(input < 0){
    output = output * -1;
}
return output;
}
void setMotorSpeed(double speed){
    motor.set(speed);
}
boolean isSensorPressed(){
    return touchSensor.isPressed();
}
double min(double x, double y){
    if(x < y){
        return x;
    }
    return y;
}
```

The `min` example may have surprised you because there isn't an `else`. There isn't any need because if `x < y`, then it will return out of the method. So the only way it will get to the `return y;` statement is if `x >= y`. So you could write it with an `else`, but that isn't necessary.

5.2.3. Special Methods: Constructors

A constructor is a special method in a Java class that has the same name as the class and it has no return type. It gets called whenever the class is initialized. (created). In Java you can have multiple constructors where each one has different parameters.

An example:

```
public class Point{
    int x;
    int y;

    public Point(int x, int y){
        this.x = x;
        this.y = y;
    }
}
```

In this case, we had to use the `this` keyword because the class member is named the same as the parameter. Sometimes people will change the parameter name instead - like this:

```
public class Point{
```

```

int x;
int y;

public Point(int x_in, int y_in){
    x = x_in;
    y = y_in;
}

```

Or, people that are coming from other languages will sometimes start all class members with `m_` so it looks like this:

```

public class Point{
    int m_x;
    int m_y;

    public Point(int x, int y){
        m_x = x;
        m_y = y;
    }
}

```

Personally, I prefer the first option, but it is a preference. All three are legal options and will do the same thing.

5.2.4. Another special method: toString

All objects in Java have a method called `toString()`. This is used whenever we convert to a string (like when we send to `telemetry.addData`). The default has the name of the class and its hash code (typically NOT useful.) This makes it easier to debug when there are problems by showing what is inside the class.

So using our `Point` class example from above:

```

public class Point{
    int x;
    int y;

    public Point(int x, int y){
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString(){
        return "Point " + x + " " + y;
    }
}

```

5. Class Members and Methods

You might be wondering why we use `@Override` when we are not extending another class. It turns out in Java that all classes extend the base class `Object`

We are adding strings and numbers together here which may seem strange. The `String` class redefines (overloads) the `+` operator to mean concatenate (join) two strings together. It also overloads `+=` to concatenate and then assign the resultant string.⁴ If it comes across something that isn't a string, it calls its `toString()` method which works (mostly) as you would expect for primitive types.

5.3. Controlling access- Keep your private things private

You can also modify all class methods and members with an access modifier. (that is who can access it.) By default, members and methods are all package-private. That means that only that class and other classes in the same package (directory) can see them. The options are: (from most to least restrictive)

- `private` - It can only be seen with the class. It cannot be accessed from outside the class.
- (default - none specified) - only that class and other classes in the same package (directory) can see them
- `protected` - It can only be seen with the class, its children, and other classes in the same package (We'll talk about children in [chapter 14](#))
- `public` - It can be seen from everywhere. (You have seen this on `init()` and `loop()` in your `OpModes`)

In general, you want to be as restrictive as makes sense. If you are modifying the access, it goes first.

5.4. Creating your own classes

Hopefully you have been following along, so you are a pro at making your own `OpMode` classes by now. We start the same (remember [section 1.3](#))

1. Right click on `org.firstinspires.ftc.teamcode`
2. Select New > Java Class

⁴No, in Java you can't overload operators in your own classes.

But in this case we are going to name it `RobotLocation` and it will have no Superclass so in Android Studio 3.x make sure the superclass is blank. (In Android Studio 4.x there is no place to put in superclass)

Listing 5.3: `RobotLocation.java`

```

1 package org.firstinspires.ftc.teamcode;
2
3 public class RobotLocation{
4     double angle;
5
6     public RobotLocation(double angle){
7         this.angle = angle;
8     }
9
10    public double getHeading(){
11        double angle = this.angle;
12        while(angle > 180){
13            angle -= 360;
14        }
15        while(angle < -180){
16            angle += 360;
17        }
18        return angle;
19    }
20
21    @Override
22    public String toString(){
23        return "RobotLocation: angle (" + angle + ")";
24    }
25
26    public void turn(double angleChange){
27        angle += angleChange;
28    }
29    public void setAngle(double angle){
30        this.angle = angle;
31    }
32 }

```

Let's talk about what makes up this class.

```

4     double angle;

```

Here is an example of the class member we talked about in [section 5.1](#) Since it doesn't have an access modifier, it is default which means it is only available to this class and other classes in the same package.

```

6     public RobotLocation(double angle){
7         this.angle = angle;

```

5. Class Members and Methods

```
8     }
```

This is an example of a constructor like we talked about in [subsection 5.2.3](#). You can tell a constructor because it has no return type and it has the same name as the class. Constructors typically have the `public` access modifier so a class can be created using it from anywhere. You'll notice that it assigns a value to the the class member. It uses the `this` keyword so that we can have the parameter named the same thing.

```
10    public double getHeading(){
11        double angle = this.angle;
12        while(angle > 180){
13            angle -= 360;
14        }
15        while(angle < -180){
16            angle += 360;
17        }
18        return angle;
19    }
```

This is a public class method that returns the heading (so it needs to be within -180 and 180). This would be a great place for a comment describing the method. We left comments out of most source in the book since the text of the book comments them.

```
21    @Override
22    public String toString(){
23        return "RobotLocation: angle (" + angle + ")";
24    }
```

This is the special method `toString()` that we talked about in [subsection 5.2.4](#).

```
26    public void turn(double angleChange){
27        angle += angleChange;
28    }
```

This is a public class method where we can specify how much the robot is turning. You'll notice that since the parameter is not the same as the class member we are using that we don't have to use the `this` keyword for the class member. You'll also notice that we use the add and assign operator `+=` as a shortcut.

```
29    public void setAngle(double angle){
30        this.angle = angle;
31    }
```

Here is another public class method where we can set the angle.

You might have noticed that there is no way to get the angle out. (We can only get out the heading). We could absolutely add this method if we needed it.

Sometimes you'll see programmers take the lazy way out and make class members public so they don't have to write "setter" or "getter" methods (also called accessor methods). The problem with that is that it makes it hard for you to change the internals later without affecting other parts of your code.

For example, if you wanted to change it to keep things in radians internally:

Listing 5.4: RobotLocationRadians.java

```

1  package org.firstinspires.ftc.teamcode;
2
3
4  public class RobotLocationRadians {
5      double angleRadians;
6
7
8      public RobotLocationRadians(double angleDegrees) {
9          this.angleRadians = Math.toRadians(angleDegrees);
10     }
11
12     public double getHeading() {
13         double angle = this.angleRadians;
14         while (angle > Math.PI) {
15             angle -= 2 * Math.PI;
16         }
17         while (angle < -Math.PI) {
18             angle += 2 * Math.PI;
19         }
20         return Math.toDegrees(angle);
21     }
22
23     @Override
24     public String toString() {
25         return "RobotLocationRadians: angle (" + angleRadians + ")";
26     }
27
28     public void turn(double angleChangeDegrees) {
29         angleRadians += Math.toRadians(angleChangeDegrees);
30     }
31
32     public void setAngle(double angleDegrees) {
33         this.angleRadians = Math.toRadians(angleDegrees);
34     }
35 }

```

I used some methods in the Math class so I wouldn't have to write the rou-

5. Class Members and Methods

tines to convert from Degrees to Radians and back. We talk about the Math class in [section 19.1](#).⁵

You'll notice that the way your class is used doesn't have to change (I only renamed it so I could keep them in the same package. In practice, you wouldn't even rename your class.)



Laziness is no longer a good excuse in Android Studio because you can right click on a member variable, select “Generate...” and choose getter and setter and Android Studio will make these methods for you!

It is interesting that we made our own class, but to be useful we need an OpMode that uses it.

Listing 5.5: UseRobotLocationOpMode.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp
7 public class UseRobotLocationOpMode extends OpMode {
8     RobotLocation robotLocation = new RobotLocation(0);
9
10    @Override
11    public void init() {
12        robotLocation.setAngle(0);
13    }
14
15    @Override
16    public void loop() {
17        if(gamepad1.a){
18            robotLocation.turn(0.1);
19        }
20        else if(gamepad1.b){
21            robotLocation.turn(-0.1);
22        }
23        telemetry.addData("Location", robotLocation);
24        telemetry.addData("Heading", robotLocation.getHeading());
25    }
26 }
```

This is the OpMode that uses our new class. The first 7 lines are the same so we'll start after that.

⁵In general, using a routine in a library is better than writing it yourself.

```
8 RobotLocation robotLocation = new RobotLocation(0);
```

This is a data member in our OpMode. You'll notice that it uses the `new` keyword. We use this whenever we are creating an instance of a class (or object). The `new` keyword tells the compiler to reserve room for it and call the constructor that matches the parameters you gave it. (type only, the names are ignored) Also by convention variables start with a lower case letter while the class starts with an upper case letter. They don't have to be named the same but often are.

```
10 @Override
11 public void init() {
12     robotLocation.setAngle(0);
13 }
```

Inside our `init()` method, we call the `setAngle()` method of the `robotLocation` object. The reason we call `setAngle()` here is in case we select the `opMode`, init it, run it and then stop and press init again. If we don't set it in `init()` then it will keep its value from the last time it was modified.



As a best practice for FTC, your `init()` method should set things back to their expected default state.

```
15 @Override
16 public void loop() {
17     if(gamepad1.a){
18         robotLocation.turn(0.1);
19     }
20     else if(gamepad1.b){
21         robotLocation.turn(-0.1);
22     }
```

Obviously this doesn't turn the robot (because we don't have any motors hooked up), so perhaps `turn()` was an unfortunate naming choice. Run this and you'll get a feel for how fast `loop()` is called. Also, we don't allow the user to turn positively and negatively at the same time (since that makes no sense). Since it looks at `gamepad1.a` first, if they are both pressed then it will turn positively.

5.5. static

The `static` keyword means that it belongs to the type instead of the object. This can be used for methods (but then they can't access any non-static members

5. Class Members and Methods

or methods) or for class members.

For class members, it is used typically for constants when you want all instances to share it.

For methods, it is often used when you want to let someone call a method and they don't need to have an object of that type first.

5.6. Exercises

1. Add a double `getAngle()` method to `RobotLocation` and then display it in your `opMode`.
2. This exercise has two parts.
 - a) Add a member of type `double` called `x` to your `RobotLocation` and add `double getX()`, `void changeX(double change)`, and `setX(double x)` methods.
 - b) Change the `OpMode` to have `robotLocation.changeX(-0.1)` called when `gamepad1.dpad_left` is pressed and `robotLocation.changeX(0.1)` when `gamepad1.dpad_right` is pressed
3. After you have done exercise 2, also add in support for `y`. Use `gamepad1.dpad_up` for `robotLocation.changeY(0.1)` and `gamepad1.dpad_down` for `robotLocation.changeY(-0.1)`

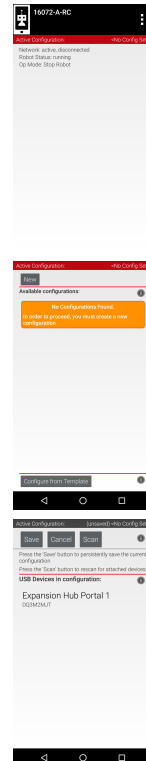
6. Our first hardware

Until this point, we have been in pure software that hasn't used any of our hardware. That is fine, but our robot will be pretty boring without any sensors, motors, or servos. This (and following chapters) assume you have a programming board setup like in [Appendix A](#)

6.1. Configuration file

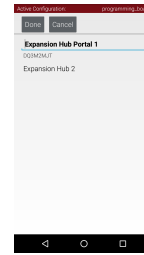
This will feel like a lot of steps the first time, but soon it'll become very natural to run through them.

1. From either the Driver Station or the Robot Controller - select the three dots in the upper right
2. Select New in the upper left
3. After you press new, it should find your expansion hub. If it doesn't, please make sure your USB cable is connected between the phone and the expansion hub. (The letters and digits of your expansion hub will be unique to your hub.)
4. Press on "Expansion Hub Portal 1"



6. Our first hardware

5. While you can rename it from “Expansion Hub Portal 1”, I don’t see any reason to. You will see each expansion hub that is plugged in. If you only have 1, it should say “Expansion Hub 2”. Press on it.

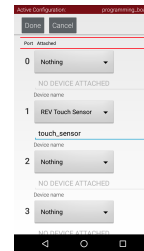


6. This will give you a listing of all of the areas where you can have communication from your REV expansion hub. Press on “Digital Devices”



7. On Port 1, Change to “Digital Channel” (Earlier versions of the FTC SDK allowed “REV Touch Sensor” to also work as a Digital Channel, but newer ones have these different)

8. Change its name to be “touch_sensor”



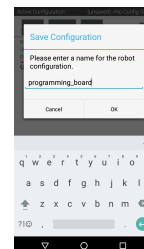
9. Press Done in the upper left (going up to Expansion Hub 2)

10. Press Done again (going up to Expansion Hub Portal 1)

11. Press Done again (going up to top level)

12. Press Save

13. Change name to “programming_board”



14. Press OK
15. Press Activate under “programming_board” The upper right should now say “programming_board”
16. Press the left pointing arrow on the bottom. This will restart the robot
17. On the Driver Station, you should see “programming_board” under the image of a robot.

6.2. Mechanisms

Until this point we have had everything in one package. At this point, we are going to split things into two packages. One will hold our mechanisms (For this book, we have one mechanism called the ProgrammingBoard.¹ On our real robot we would likely have multiple mechanisms.) The other will hold our opModes.

So there are now two classes:

This one is in the mechanisms package. To create a package, right click in the same place that we have to make a new class, but select new package and type in “mechanisms”.

That will make the package. Then right click on the package and select new class. This one should be “ProgrammingBoard1” and it should have no superclass.

Listing 6.1: ProgrammingBoard1.java

```

1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.DigitalChannel;
4 import com.qualcomm.robotcore.hardware.HardwareMap;
5
6 public class ProgrammingBoard1 {
7     private DigitalChannel touchSensor;
8
9     public void init(HardwareMap hwMap) {
10         touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
11         touchSensor.setMode(DigitalChannel.Mode.INPUT);
12     }
13

```

¹To make writing the book easier, I added a number at the end so I could keep them all in one package. Normally you would just add things to your existing class instead of making a new one.

6. Our first hardware

```
14     public boolean getTouchSensorState() {  
15         return touchSensor.getState();  
16     }  
17 }
```

Line 1 should be put in for you by Android Studio.

Lines 3 & 4 will be put in as you type items.

Line 6 should start out that way as you create the class

```
7     private DigitalChannel touchSensor;
```

This line says that we have a class member of type `DigitalChannel` with a name of `touchSensor`. `DigitalChannel` comes from the FTC SDK. We'll talk about how to navigate the SDK to find out what is there in [chapter 18](#). This needs to be a class member since it is set in `init()` and used in other methods. We set it to `private` to make sure only our class can interact directly with it. This is a good practice for all hardware. Normally you would want to name it with what the sensor does (like `armInPositionTouchSensor`, but since this is part of a programming board it doesn't have more of a purpose than being a Touch Sensor.

```
9     public void init(HardwareMap hwMap) {
```

We have an `init()` method. We could have called it anything, but since we'll call it from our `init()` in our `OpMode` it seemed reasonable. While it might be tempting to make this the constructor, that limits what we can and can't do, so it is easier to follow the same structure. You'll notice that this takes one parameter of type `HardwareMap` and it is called `hwMap`. We could have called it `hardwareMap` but I am lazy so I took a shortcut. `HardwareMap` also comes from the FTC SDK and it is how our programs get information from the configuration file on the robot.

```
10         touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
```

This assigns to the variable `touchSensor` the hardware that is in the configuration file of type `DigitalChannel.class` and with a name of `touch_sensor`. This name has to match **EXACTLY** what is in the configuration file. It may seem strange to you that you don't have to use `new` here. That is because the `get()` method of `HardwareMap` does it for you.

```
11         touchSensor.setMode(DigitalChannel.Mode.INPUT);
```

It turns out that you can set each `DigitalChannel` as either `INPUT` or `OUTPUT`. Since we are reading from the touch sensor, we need to set it as an `INPUT`.

```

14     public boolean getTouchSensorState() {
15         return touchSensor.getState();
16     }

```

We create a class method so that those outside of our class can read the state of the touchSensor. This is better than making touchSensor public because nobody can change how it is configured.

6.3. OpMode

This one is in the opmodes package

Listing 6.2: TouchSensorOpMode.java

```

1  package org.firstinspires.ftc.teamcode.opmodes;
2
3  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6  import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard1;
7
8  @TeleOp()
9  public class TouchSensorOpMode extends OpMode {
10     ProgrammingBoard1 board = new ProgrammingBoard1();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         telemetry.addData("Touch sensor", board.getTouchSensorState());
19     }
20 }

```

The first few lines of this should look amazingly familiar by now.

```

10     ProgrammingBoard1 board = new ProgrammingBoard1();

```

Here we create a class member of type ProgrammingBoard1 named board and we set it equal to a new instance of ProgrammingBoard1. It has to be a class member so all of our methods can access it.

```

12     public void init() {
13         board.init(hardwareMap);
14     }

```

6. Our first hardware

Our init is very clean. It only calls the init of our board object. The variable hardwareMap is part of the OpMode and it is how we see how the robot is configured.

```
17     public void loop() {
18         telemetry.addData("Touch sensor", board.getTouchSensorState());
19     }
```

For the loop all we do is send to the telemetry the state of the touch sensor.

6.4. Making changes

One of the huge advantages of splitting things out is that we can isolate hardware “weirdness”. For example, you were probably surprised that pushing in the touch sensor returns false and it not pushed in was true. So let’s change that.

First, we’ll change our ProgrammingBoard class. The easiest way to do this is right click on the file, select copy. Then select paste and give it the new file name. Then you can just make the changes instead of typing everything in again.

Listing 6.3: ProgrammingBoard2.java

```
1  package org.firstinspires.ftc.teamcode.mechanisms;
2
3  import com.qualcomm.robotcore.hardware.DigitalChannel;
4  import com.qualcomm.robotcore.hardware.HardwareMap;
5
6  public class ProgrammingBoard2 {
7      private DigitalChannel touchSensor;
8
9      public void init(HardwareMap hwMap) {
10         touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
11         touchSensor.setMode(DigitalChannel.Mode.INPUT);
12     }
13
14     public boolean isTouchSensorPressed() {
15         return !touchSensor.getState();
16     }
17 }
```

While we could have done code like:

```
public boolean isTouchSensorPressed(){
    if(!touchSensor.getState()){    // if state is false, touch sensor is pressed
        return true;
    }
```

```

    }
    return false;
}

```

It turns out that doing it in one line does exactly the same thing.

Also, since we changed the name of the method, we have to change it in the `OpMode` as well.



If we right click on a class method (or class member) name and Refactor->Rename in Android Studio then it will magically change it both in its declaration and everywhere it is called.

Listing 6.4: `TouchSensorOpMode2.java`

```

1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard2;
7
8 @TeleOp()
9 public class TouchSensorOpMode2 extends OpMode {
10     ProgrammingBoard2 board = new ProgrammingBoard2();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         telemetry.addData("Touch pressed", board.isTouchSensorPressed());
19     }
20 }

```

6.5. Exercises

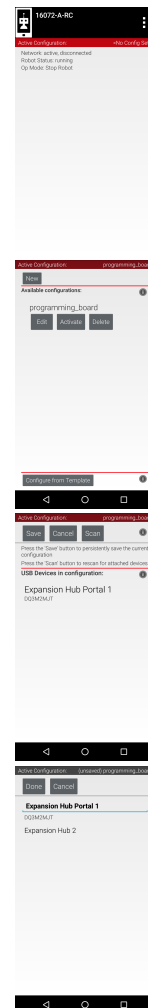
1. Add a method `isTouchSensorReleased()` to the `ProgrammingBoard2` class and use it in your `opMode`
2. Have your `opMode` send “Pressed” and “Not Pressed” for the “Touch sensor” instead of `true` or `false`. There are lots of ways to do this.

7. Motors

It is great that we have a sensor, but it is time to make things move!!

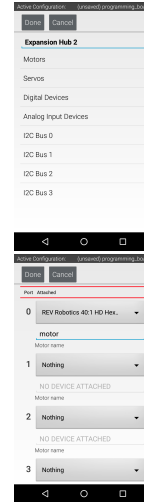
7.1. Editing Configuration File

1. From either the Driver Station or the Robot Controller - select the three dots in the upper right
2. Press edit under the “programming_board” config that we made earlier
3. Press on “Expansion Hub Portal 1”
4. While you can rename it from “Expansion Hub Portal 1”, I don’t see any reason to. You will see each expansion hub that is plugged in. If you only have 1, it should say “Expansion Hub 2”. Press on it.



7. Motors

5. This will give you a listing of all of the areas where you can have communication from your REV expansion hub. Press on “Motors”
6. On Port 0, Change to “Rev Robotics 40:1 HD Hex Motor”
7. Change its name to be “motor”
8. Press Done in the upper left (going up to Expansion Hub 2)
9. Press Done again (going up to Expansion Hub Portal 1)
10. Press Done again (going up to top level)
11. Press Save
12. Press OK
13. Press Activate under “programming_board” The upper right should now say “programming_board”
14. Press the left pointing arrow on the bottom. This will restart the robot
15. On the Driver Station, you should see “programming_board” under the image of a robot.



7.2. Mechanisms

Listing 7.1: ProgrammingBoard3.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.DcMotor;
4 import com.qualcomm.robotcore.hardware.DigitalChannel;
5 import com.qualcomm.robotcore.hardware.HardwareMap;
```

```

6
7 public class ProgrammingBoard3 {
8     private DigitalChannel touchSensor;
9     private DcMotor motor;
10
11     public void init(HardwareMap hwMap) {
12         touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
13         touchSensor.setMode(DigitalChannel.Mode.INPUT);
14         motor = hwMap.get(DcMotor.class, "motor");
15         motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
16     }
17     public boolean isTouchSensorPressed() {
18         return !touchSensor.getState();
19     }
20
21     public void setMotorSpeed(double speed){
22         motor.setPower(speed);
23     }
24 }

```

Most of this should look the same as our last file, so we'll just talk about the changes

```

9     private DcMotor motor;

```

Here we are adding a variable of type `DcMotor` with name `motor`. Normally you would want to name the motor with what it does, but since this is part of a programming board - we'll just call in `motor`. `DcMotor` comes from the FTC SDK.

```

14     motor = hwMap.get(DcMotor.class, "motor");

```

This assigns to the variable `motor` the hardware that is in the configuration file of type `DcMotor.class` and with a name of `motor`. This name has to match **EXACTLY** what is in the configuration file.

```

15     motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);

```

This sets how we want to use the motor. The choices are:

7. Motors

RunMode	Meaning
RUN_TO_POSITION	The motor is to attempt to rotate in whatever direction is necessary to cause the encoder reading to advance or retreat from its current setting to the setting which has been provided through the <code>setTargetPosition()</code> method.
RUN_USING_ENCODER	The motor is to do its best to run at targeted velocity.
RUN_WITHOUT_ENCODER	The motor is simply to run at whatever velocity is achieved by applying a particular power level to the motor.
STOP_AND_RESET_ENCODER	The motor is to set the current encoder position to zero.

We set it here to `DcMotor.RunMode.RUN_USING_ENCODER` which means that it uses the encoder on the motor so that we are setting a speed and it figures out how to modify power to get to that speed (if possible). We like this mode because if you set two motors to the same speed then they have a better chance at being at the same speed than in any other mode. (We have met teams that don't even plug in the encoders and they are having weird problems with the robot not driving straight.)



While `RUN_TO_POSITION` can be very handy for single motors, we recommend AGAINST using it in a drive train because the different speeds for the different wheels trying to get to a position can cause wacky side effects. This is because each motor is trying to get to its position irregardless of the other motors. So you can have a robot “wiggle”. In a perfect world without friction this would work the same. However, we don't live in that world. :-)

```
21 public void setMotorSpeed(double speed){
22     motor.setPower(speed);
23 }
```

This is a class method so that code outside our class can set the speed of the motor. This is better than exposing the `motor` as public because people can't accidentally change configuration. `setPower()` on a motor takes a double between -1.0 and 1.0. -1.0 is full speed “backwards”, 0.0 is stopped, and 1.0 is full speed “forwards”.

7.3. OpMode

This one is in the opmodes package

Listing 7.2: MotorOpMode.java

```

1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard4;
7
8 @TeleOp()
9 public class MotorOpMode extends OpMode {
10     ProgrammingBoard4 board = new ProgrammingBoard4();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         board.setMotorSpeed(0.5);
19     }
20 }

```

This has very little that is new, so we'll only talk about that.

```

17     public void loop() {
18         board.setMotorSpeed(0.5);
19     }

```

Here we don't do anything conditional. We just set the motor to a speed of 0.5 (half way forwards) Technically we could have had a `start()` method that did this but since we have to have a `loop()` in our `OpMode` anyway, we went for the simple. Yes, it will tell the motor to go to the same speed over and over. It doesn't matter.

7.4. Motor as Sensor

The motor also has a rotation sensor built into it. We are using it when we say `RUN_USING_ENCODER`, but we can also read it and use it in our code. It'll need a method in the `ProgrammingBoard` class so we can read it.

Listing 7.3: ProgrammingBoard4.java

7. Motors

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.DcMotor;
4 import com.qualcomm.robotcore.hardware.DcMotorEx;
5 import com.qualcomm.robotcore.hardware.DigitalChannel;
6 import com.qualcomm.robotcore.hardware.HardwareMap;
7
8 public class ProgrammingBoard4 {
9     private DigitalChannel touchSensor;
10    private DcMotor motor;
11    private double ticksPerRotation;
12
13    public void init(HardwareMap hwMap) {
14        touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
15        touchSensor.setMode(DigitalChannel.Mode.INPUT);
16        motor = hwMap.get(DcMotor.class, "motor");
17        motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
18        ticksPerRotation = motor.getMotorType().getTicksPerRev();
19        motor.getPower();
20    }
21    public boolean isTouchSensorPressed() {
22        return !touchSensor.getState();
23    }
24
25    public void setMotorSpeed(double speed){
26        motor.setPower(speed);
27    }
28    public double getMotorRotations(){
29        return motor.getCurrentPosition() / ticksPerRotation;
30    }
31 }
```

Most of this is the same so we'll just talk about the differences

```
10    private DcMotor motor;
```

This is a member variable where we will store the number of encoder ticks per rotation. We do this to make things easier for the opModes.

```
17        motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
```

If we set the exact motor we have in the configuration, then we can do this to get the number of ticks per rev (revolution). I prefer to call them rotation since our students come from FLL teams where they are more used to that terminology. If you have additional gear changes after the motor, you'll have to calculate this. For example if you have a 2:1 gear reduction then you would simply multiply the number of ticks per rev at the motor by 2 to get the number of revolutions of your mechanism.

```

26     motor.setPower(speed);
27 }
28 public double getMotorRotations(){

```

This is a class method where we return the number of motor rotations. To get the number of rotations from the number of encoder ticks, we simply divide the number of ticks by the number of ticks in a rotation. One nice thing about Java is that if there is math between an `int` and a `double`, the result will be a `double`. (However, be warned that dividing an `int` by an `int` always gives an `int` result even if it doesn't divide equally. So for example `5 / 2` will be `2`.)

Listing 7.4: MotorOpMode2.java

```

1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard4;
7
8 @TeleOp()
9 public class MotorOpMode2 extends OpMode {
10     ProgrammingBoard4 board = new ProgrammingBoard4();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         board.setMotorSpeed(0.5);
19         telemetry.addData("Motor rotations", board.getMotorRotations());
20     }
21 }

```

This only has one line added from before

```

19     telemetry.addData("Motor rotations", board.getMotorRotations());

```

Here we are simply sending to telemetry what we are seeing from the motor rotations.



If your encoder counts are not going up when you are sending a positive speed to your motor, you probably have the power wires flipped going to the motor.

7. Motors

7.5. Motors and Sensors together

We don't need to make any change to our configuration file or our ProgrammingBoard file since they already have a motor and a sensor.

Listing 7.5: MotorSensorOpMode.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard4;
7
8 @TeleOp()
9 public class MotorSensorOpMode extends OpMode {
10     ProgrammingBoard4 board = new ProgrammingBoard4();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         if(board.isTouchSensorPressed()) {
19             board.setMotorSpeed(0.5);
20         }
21         else{
22             board.setMotorSpeed(0.0);
23         }
24         telemetry.addData("Motor rotations", board.getMotorRotations());
25     }
26 }
```

Remember that setting the motor speed to 0 makes it stop. You can set for each motor what you would like it to do when set to zero by calling `setZeroBehavior()` with either `DcMotor.ZeroPowerBehavior.BRAKE` or `DcMotor.ZeroPowerBehavior.FLOAT`

So in this case, when the touch sensor is pressed we move the motor “forward” at half speed. When it isn't, we stop it.

You may end up in a circumstance where you want “forward” to be the opposite direction of clockwise. (Like on the left hand side of your drive train). To do this, you simply call the motor's method `setDirection()` with `DcMotorSimple.Direction.REVERSE` and if you want to change it back you call it with `DcMotorSimple.Direction.FORWARD`. The motor remembers these settings.

So you might make your ProgrammingBoard class `init()` method look like

this:

```
...
motor = hwMap.get(DcMotor.class, "motor");
motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
motor.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.BRAKE);
motor.setDirection(DcMotorSimple.Direction.REVERSE);
ticksPerRotation = motor.getMotorType().getTicksPerRev();
...
```

7.6. Motors and Gamepads

And of course, we can use our Gamepad just like a sensor. (we are sensing what the human is doing.)

Listing 7.6: MotorGamepadOpMode.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard4;
7
8 @TeleOp()
9 public class MotorGamepadOpMode extends OpMode {
10     ProgrammingBoard4 board = new ProgrammingBoard4();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         if(gamepad1.a) {
19             board.setMotorSpeed(0.5);
20         }
21         else{
22             board.setMotorSpeed(0.0);
23         }
24         telemetry.addData("Motor rotations", board.getMotorRotations());
25     }
26 }
```

This is exactly the same as before except for using `gamepad1.a` instead of the touch sensor.

7. Motors

But we don't have to be limited to just the buttons. We can make it finer controlled by using an analog input from the gamepad

Listing 7.7: MotorGamepadOpMode2.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard4;
7
8 @TeleOp()
9 public class MotorGamepadOpMode2 extends OpMode {
10     ProgrammingBoard4 board = new ProgrammingBoard4();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         double motorSpeed = gamepad1.left_stick_y;
19
20         board.setMotorSpeed(motorSpeed);
21
22         telemetry.addData("Motor speed", motorSpeed);
23         telemetry.addData("Motor rotations", board.getMotorRotations());
24     }
25 }
```

Yes, we could have used `gamepad1.left_stick_y` twice instead of making a `motorSpeed` variable. But I prefer to do it this way in case I want to do any math on the `motorSpeed` before using it.

7.7. Exercises

1. Add a method to the `ProgrammingBoard` that allows you to change the `ZeroPowerBehavior` of the motor, and then add to your `OpMode` where pressing `gamepad1.a` sets it to `BRAKE` and `gamepad1.b` sets it to `FLOAT`.
2. Make the joystick less sensitive in the middle without losing range by bringing in the `squareInputWithSign()` method from [section 5.2](#) into your `opMode` and using it.

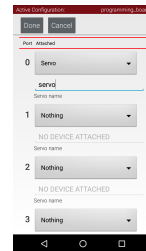
8. Servos

8.1. Configuration File

Follow steps 1-5 of [section 7.1](#), but select Servos

6. On Port 0, Change to “Servo”
7. Change its name to be “servo”

Continue with steps 8 and on of [section 7.1](#)



8.2. Mechanisms

Let’s start with what we need to do to our ProgrammingBoard class.

Listing 8.1: ProgrammingBoard5.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.DcMotor;
4 import com.qualcomm.robotcore.hardware.DigitalChannel;
5 import com.qualcomm.robotcore.hardware.HardwareMap;
6 import com.qualcomm.robotcore.hardware.Servo;
7
8 public class ProgrammingBoard5 {
9     private DigitalChannel touchSensor;
10    private DcMotor motor;
11    private double ticksPerRotation;
12    private Servo servo;
13
14    public void init(HardwareMap hwMap) {
15        touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
16        touchSensor.setMode(DigitalChannel.Mode.INPUT);
17        motor = hwMap.get(DcMotor.class, "motor");
18        motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
19        ticksPerRotation = motor.getMotorType().getTicksPerRev();
20    }
21 }
```

8. Servos

```
20     servo = hwMap.get(Servo.class, "servo");
21 }
22 public boolean isTouchSensorPressed() {
23     return !touchSensor.getState();
24 }
25
26 public void setMotorSpeed(double speed){
27     motor.setPower(speed);
28 }
29 public double getMotorRotations(){
30     return motor.getCurrentPosition() / ticksPerRotation;
31 }
32 public void setServoPosition(double position){
33     servo.setPosition(position);
34 }
35 }
```

This is very similar to the ones before. We'll just talk about the new parts.

```
12     private Servo servo;
```

Here we create a class member of type `Servo` named `servo`. The `Servo` class comes from the FTC SDK. Again, we would use a more descriptive name on our robot.

```
20     servo = hwMap.get(Servo.class, "servo");
```

This assigns to the variable `servo` the hardware that is in the configuration file of type `Servo.class` and with a name of `servo`. This name has to match **EXACTLY** what is in the configuration file.

```
32     public void setServoPosition(double position){
33         servo.setPosition(position);
34     }
```

This allows code outside of our class to set the servo position. Typically we might expose a method for each position we want it to go to - for example `setClawOpen()` and `setClawClose()`

`servo.setPosition()` takes a double which is a fraction between 0.0 and 1.0 saying where in that range to move. We can programmatically change what that means with two methods:

1. `servo.setDirection(Servo.Direction.REVERSE)` flips your range. (and yes you can also call it with `Servo.Direction.FORWARD` to flip it back)
2. `servo.scaleRange(double min, double max)` sets the logical min and max. Then `servo.setPosition()` is a fraction between that range. ¹ It is relative

¹The min has to be less than the max, so you can't use this to flip the direction.

to the entire range, so you can set it back with `servo.scaleRange(0.0, 1.0)`.

As an example, you might have this in the `init()` method

```
...
servo = hwMap.get(Servo.class, "servo");
servo.setDirection(Servo.Direction.REVERSE);
servo.scaleRange(0.5, 1.0); // only go from midpoint to far right point
...
```

8.3. OpMode

This one is in the `opmodes` package

Listing 8.2: `ServoGamepadOpMode.java`

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard5;
7
8 @TeleOp()
9 public class ServoGamepadOpMode extends OpMode {
10     ProgrammingBoard5 board = new ProgrammingBoard5();
11     @Override
12     public void init() {
13         board.init(hardwareMap);
14     }
15
16     @Override
17     public void loop() {
18         if(gamepad1.a) {
19             board.setServoPosition(1.0);
20         }
21         else if(gamepad1.b){
22             board.setServoPosition(0.0);
23         }
24         else{
25             board.setServoPosition(0.5);
26         }
27     }
28 }
```

The only new thing here is:

8. Servos

```
17  public void loop() {  
18      if(gamepad1.a) {  
19          board.setServoPosition(1.0);  
20      }  
21      else if(gamepad1.b){  
22          board.setServoPosition(0.0);  
23      }  
24      else{  
25          board.setServoPosition(0.5);  
26      }  
27  }
```

You'll see that we are using chained if and else so that we only try to set the servo position to one location. Otherwise we will confuse the servo and you'll likely see some jitter on it. (although the last one will likely win since there is more time in between calls to `loop()` than within `loop()`)

8.4. Exercises

1. Change the `ProgrammingBoard` class so that the servo is backwards and only goes from the midpoint to far left.
2. Change the `opMode` so that how far you push in `gamepad1.left_trigger` determines the position of the servo.

9. Analog Sensors

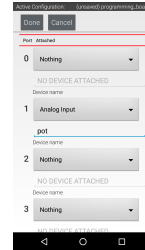
We'll be using a potentiometer here, but the same concepts work for all analog sensors. It is very common to abbreviate potentiometer as “pot” because potentiometer is hard to spell.

9.1. Configuration File

Follow steps 1-5 of [section 7.1](#), but select Analog Input Devices

6. On Port 0, Change to “Analog Input”

7. Change its name to be “pot”



Continue with steps 8 and on of [section 7.1](#)

9.2. Mechanisms

First, lets add support to our ProgrammingBoard class.

Listing 9.1: ProgrammingBoard6.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.AnalogInput;
4 import com.qualcomm.robotcore.hardware.DcMotor;
5 import com.qualcomm.robotcore.hardware.DigitalChannel;
6 import com.qualcomm.robotcore.hardware.HardwareMap;
7 import com.qualcomm.robotcore.hardware.Servo;
8 import com.qualcomm.robotcore.util.Range;
9
10 public class ProgrammingBoard6 {
11     private DigitalChannel touchSensor;
12     private DcMotor motor;
13     private double ticksPerRotation;
14     private Servo servo;
```

9. Analog Sensors

```
15     private AnalogInput pot;
16
17     public void init(HardwareMap hwMap) {
18         touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
19         touchSensor.setMode(DigitalChannel.Mode.INPUT);
20         motor = hwMap.get(DcMotor.class, "motor");
21         motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
22         ticksPerRotation = motor.getMotorType().getTicksPerRev();
23         servo = hwMap.get(Servo.class, "servo");
24         pot = hwMap.get(AnalogInput.class, "pot");
25     }
26     public boolean isTouchSensorPressed() {
27         return !touchSensor.getState();
28     }
29
30     public void setMotorSpeed(double speed){
31         motor.setPower(speed);
32     }
33     public double getMotorRotations(){
34         return motor.getCurrentPosition() / ticksPerRotation;
35     }
36     public void setServoPosition(double position){
37         servo.setPosition(position);
38     }
39     public double getPotAngle(){
40         return Range.scale(pot.getVoltage(), 0, pot.getMaxVoltage(), 0, 270);
41     }
42 }
```

Most of this is the same, so we'll just explain the new bits.

```
15     private AnalogInput pot;
```

We are declaring a class member of type `AnalogInput` with name `pot`. The `AnalogInput` class comes from the FTC SDK.

```
24         pot = hwMap.get(AnalogInput.class, "pot");
```

This assigns to the variable `pot` the hardware that is in the configuration file of type `AnalogInput.class` and with a name of `pot`. This name has to match **EXACTLY** what is in the configuration file.

```
39     public double getPotAngle(){
40         return Range.scale(pot.getVoltage(), 0, pot.getMaxVoltage(), 0, 270);
41     }
```

This is a class method that returns the angle to potentiometer is currently at. It turns out that the `AnalogInput` class gives us a voltage. We could just expose that with a `getPotVoltage()` method, but then our other code has to

know about voltage when it makes more sense to think in terms of the angle it is pointing at. We use a cool trick here to translate from voltage to angle.

There is a utility class in the FTC SDK called `Range` that has a method called `scale()`. It will translate a number from one range to another one. So for example if you call

```
double output = Range.scale(25, 0, 100, 0.0, 1.0);
```

then it would figure out that the input (25) was 1/4 of the way between 0 and 100. It would then figure out what 1/4 between 0 and 1.0 is and would set output to 0.25.

In this case we know that the lowest possible voltage that could be detected is 0, the highest we can get by calling `pot.getMaxVoltage()`. We know our potentiometer can be between 0 and 270 degrees. So we use `Range.scale` to convert for us.

You might have noticed that you made a method call on a class instead of an object (a variable of type class). That is because it is a `static` method. This is an example of what we talked about in [section 5.5](#).

9.3. OpMode

Now we need an OpMode that can use it.

Listing 9.2: PotOpMode.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard5;
7 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard6;
8
9 @TeleOp()
10 public class PotOpMode extends OpMode {
11     ProgrammingBoard6 board = new ProgrammingBoard6();
12     @Override
13     public void init() {
14         board.init(hardwareMap);
15     }
16
17     @Override
18     public void loop() {
19         telemetry.addData("Pot Angle", board.getPotAngle());
20     }
21 }
```

```
}
```

Since we are doing the conversion in our `ProgrammingBoard` class, this becomes trivial. We are simply reporting the angle. This can be used on our robot to know what angle something is turned to.

9.4. Exercises

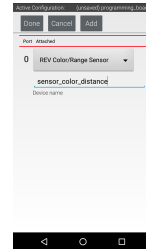
1. Make a class method for your `ProgrammingBoard` that exposes the pot in the range `[0.0..1.0]`
2. Now make an `OpMode` that sets the servo to the position that the pot is returning in that range. Then you can turn the pot and it will cause the servo to “follow” it.

10. Color and Distance Sensors

10.1. Configuration File

Follow steps 1-5 of [section 7.1](#), but select I2C Bus 1

6. On Port 0, Change to “REV Color/Range Sensor”
7. Change its name to be “sensor_color_distance”



Continue with steps 8 and on of [section 7.1](#)

10.2. Mechanisms

Let’s start by making a change to our ProgrammingBoard class.

Listing 10.1: ProgrammingBoard7.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.AnalogInput;
4 import com.qualcomm.robotcore.hardware.ColorSensor;
5 import com.qualcomm.robotcore.hardware.DcMotor;
6 import com.qualcomm.robotcore.hardware.DigitalChannel;
7 import com.qualcomm.robotcore.hardware.DistanceSensor;
8 import com.qualcomm.robotcore.hardware.HardwareMap;
9 import com.qualcomm.robotcore.hardware.Servo;
10 import com.qualcomm.robotcore.util.Range;
11
12 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
13
14 public class ProgrammingBoard7 {
15     private DigitalChannel touchSensor;
16     private DcMotor motor;
17     private double ticksPerRotation;
18     private Servo servo;
19     private AnalogInput pot;
```

10. Color and Distance Sensors

```
20 private ColorSensor colorSensor;
21 private DistanceSensor distanceSensor;
22
23 public void init(HardwareMap hwMap) {
24     touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
25     touchSensor.setMode(DigitalChannel.Mode.INPUT);
26     motor = hwMap.get(DcMotor.class, "motor");
27     motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
28     ticksPerRotation = motor.getMotorType().getTicksPerRev();
29     servo = hwMap.get(Servo.class, "servo");
30     pot = hwMap.get(AnalogInput.class, "pot");
31
32     colorSensor = hwMap.get(ColorSensor.class, "sensor_color_distance");
33     distanceSensor = hwMap.get(DistanceSensor.class, "sensor_color_distance");
34 }
35 public boolean isTouchSensorPressed() {
36     return !touchSensor.getState();
37 }
38
39 public void setMotorSpeed(double speed){
40     motor.setPower(speed);
41 }
42 public double getMotorRotations(){
43     return motor.getCurrentPosition() / ticksPerRotation;
44 }
45 public void setServoPosition(double position){
46     servo.setPosition(position);
47 }
48 public double getPotAngle(){
49     return Range.scale(pot.getVoltage(), 0, pot.getMaxVoltage(), 0, 270);
50 }
51 public int getAmountRed(){
52     return colorSensor.red();
53 }
54 public double getDistance(DistanceUnit du){
55     return distanceSensor.getDistance(du);
56 }
57 }
```

Most of this is similar so we'll only talk about the new parts.

```
20 private ColorSensor colorSensor;
21 private DistanceSensor distanceSensor;
```

This is a little different. A REV ColorSensor can act as both a color sensor and a distance sensor.¹ So we make two variables - one for the ColorSensor

¹Although the distance sensor part of a color sensor is much less accurate and over a smaller range than a REV Distance sensor.

class and one for the `DistanceSensor` class. Both of these classes are in the FTC SDK.

```
32 colorSensor = hwMap.get(ColorSensor.class, "sensor_color_distance");
33 distanceSensor = hwMap.get(DistanceSensor.class, "sensor_color_distance");
```

Both of these follow the pattern we have seen before. The unusual part is that they use the SAME string for the sensor. Again, it has to match EXACTLY what is in the configuration file.

```
51 public int getAmountRed(){
52     return colorSensor.red();
53 }
```

This is a class method that returns the amount of red that the color sensor sees (between 0 and 255) . The `colorSensor` class has several class methods that are useful.

Method	What it returns
<code>red()</code>	Amount of red seen (0-255)
<code>green()</code>	Amount of green seen (0-255)
<code>blue()</code>	Amount of blue seen (0-255)
<code>argb()</code>	An integer in the format #aarrggbb (where a is alpha, r is red, g is green, b is blue)

```
54 public double getDistance(DistanceUnit du){
55     return distanceSensor.getDistance(du);
56 }
```

This uses a neat class included in the FTC SDK called `DistanceUnit`. It allows us to decide what units we want to work in and hopefully keeps us from making a NASA class mistake with units.² This is a simple pass through so we'll talk more about `DistanceUnit` as we discuss the OpMode.

10.3. OpMode

And we need an OpMode that can use it.

Listing 10.2: `DistanceColorOpMode.java`

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
```

²https://en.wikipedia.org/wiki/Mars_Climate_Orbiter

10. Color and Distance Sensors

```
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
7 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard7;
8
9 @TeleOp()
10 public class DistanceColorOpMode extends OpMode {
11     ProgrammingBoard7 board = new ProgrammingBoard7();
12     @Override
13     public void init() {
14         board.init(hardwareMap);
15     }
16
17     @Override
18     public void loop() {
19         telemetry.addData("Amount red", board.getAmountRed());
20         telemetry.addData("Distance (CM)", board.getDistance(DistanceUnit.CM));
21         telemetry.addData("Distance (IN)", board.getDistance(DistanceUnit.INCH));
22     }
23 }
```

A lot of this is similar, so let's talk about the new parts.

```
19 telemetry.addData("Amount red", board.getAmountRed());
```

This simply prints the amount of red seen by the color sensor

```
20 telemetry.addData("Distance (CM)", board.getDistance(DistanceUnit.CM));
21 telemetry.addData("Distance (IN)", board.getDistance(DistanceUnit.INCH));
```

This is showing the coolness of the `DistanceUnit` class. By passing in different values to `getDistance()`, we get it in the units we prefer. (you should prefer metric - but since a lot of the FTC specs are in Imperial, it is helpful to be able to do both.) The choices are:

Parameter	Unit
<code>DistanceUnit.MM</code>	millimeter
<code>DistanceUnit.CM</code>	centimeter
<code>DistanceUnit.INCH</code>	inch
<code>DistanceUnit.METER</code>	meter

If you are using this with your class, you'll have to decide what unit you are going to store things in (I typically recommend CM, but that is up to you.) Then you can convert things like this:

```
public class Square{
    double length_cm = 10;
```

```
    public double getLength(DistanceUnit du){  
        return du.fromCm(length_cm);  
    }  
    public void setLength(double length, DistanceUnit du){  
        length_cm = du.toCm(length);  
    }  
}
```

10.4. Exercises

1. Add a method `getAmountBlue()` to the `ProgrammingBoard` and report it back by changing the `OpMode`
2. Make the motor stop when the distance sensor sees something closer than 10cm and go at half speed when farther than that.

11. Gyro (IMU)

11.1. Configuration File

Unlike everything else, you don't need to add it to the robot configuration because it is already there as "imu". You can rename it or delete it.

11.2. Mechanisms

Let's start by adding support to our ProgrammingBoard class.

Listing 11.1: ProgrammingBoard8.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.hardware.rev.RevHubOrientationOnRobot;
4 import com.qualcomm.robotcore.hardware.AnalogInput;
5 import com.qualcomm.robotcore.hardware.ColorSensor;
6 import com.qualcomm.robotcore.hardware.DcMotor;
7 import com.qualcomm.robotcore.hardware.DigitalChannel;
8 import com.qualcomm.robotcore.hardware.DistanceSensor;
9 import com.qualcomm.robotcore.hardware.HardwareMap;
10 import com.qualcomm.robotcore.hardware.IMU;
11 import com.qualcomm.robotcore.hardware.Servo;
12 import com.qualcomm.robotcore.util.Range;
13
14 import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
15 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
16
17 public class ProgrammingBoard8 {
18     private DigitalChannel touchSensor;
19     private DcMotor motor;
20     private double ticksPerRotation;
21     private Servo servo;
22     private AnalogInput pot;
23     private ColorSensor colorSensor;
24     private DistanceSensor distanceSensor;
25     private IMU imu;
26
27     public void init(HardwareMap hwMap) {
28         touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
```

11. Gyro (IMU)

```
29     touchSensor.setMode(DigitalChannel.Mode.INPUT);
30     motor = hwMap.get(DcMotor.class, "motor");
31     motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
32     ticksPerRotation = motor.getMotorType().getTicksPerRev();
33     servo = hwMap.get(Servo.class, "servo");
34     pot = hwMap.get(AnalogInput.class, "pot");
35
36     colorSensor = hwMap.get(ColorSensor.class, "sensor_color_distance");
37     distanceSensor = hwMap.get(DistanceSensor.class, "sensor_color_distance");
38     imu = hwMap.get(IMU.class, "imu");
39
40     RevHubOrientationOnRobot RevOrientation =
41         new RevHubOrientationOnRobot(RevHubOrientationOnRobot.↵
42             ↵ LogoFacingDirection.UP,
43             RevHubOrientationOnRobot.UsbFacingDirection.FORWARD);
44
45     imu.initialize(new IMU.Parameters(RevOrientation));
46 }
47
48 public boolean isTouchSensorPressed() {
49     return !touchSensor.getState();
50 }
51
52 public void setMotorSpeed(double speed){
53     motor.setPower(speed);
54 }
55
56 public double getMotorRotations(){
57     return motor.getCurrentPosition() / ticksPerRotation;
58 }
59
60 public void setServoPosition(double position){
61     servo.setPosition(position);
62 }
63
64 public double getPotAngle(){
65     return Range.scale(pot.getVoltage(), 0, pot.getMaxVoltage(), 0, 270);
66 }
67
68 public int getAmountRed(){
69     return colorSensor.red();
70 }
71
72 public double getDistance(DistanceUnit du){
73     return distanceSensor.getDistance(du);
74 }
75
76 public double getHeading(AngleUnit angleUnit) {
77     return imu.getRobotYawPitchRollAngles().getYaw(angleUnit);
78 }
79 }
```

Starting with the Control Hubs shipping in late 2022, the IMU chip changed. However, the FTC SDK changed in 8.1 to isolate the differences for us. While it

has a TON of capabilities, we are going to just barely tap into it here. (Original Rev Expansion Hubs had an IMU in them, newer ones do not.)

```
25     private IMU imu;
```

We create a class member of type `IMU` (you guessed it from the FTC SDK) with the name `imu`.

```
38     imu = hwMap.get(IMU.class, "imu");
```

First, we get the `imu` from the hardware map (just like we have done with other pieces of hardware). If you didn't change the name in your configuration (and you shouldn't), it will be "imu".

```
40     RevHubOrientationOnRobot RevOrientation =
41         new RevHubOrientationOnRobot(RevHubOrientationOnRobot.↵
42             ↵ LogoFacingDirection.UP,
43             RevHubOrientationOnRobot.UsbFacingDirection.FORWARD);
44     imu.initialize(new IMU.Parameters(RevOrientation));
```

Next, we need to describe how the Rev Hub is oriented on our robot. This is simply done by stating what the orientation of the Logo and the orientation of the USB is. The valid choices are `FORWARD`, `BACKWARD`, `UP`, `DOWN`, `LEFT`, and `RIGHT`. (Yes, you can place your Hub at an angle. To do that look at the example `SensorIMUNonOrthogonal.java` in the Java examples that come with the SDK.)

Then we initialize the `imu` with parameters.

```
69     public double getHeading(AngleUnit angleUnit) {
```

We are creating a class method so code outside of our class can get the heading of the robot (actually REV hub). Much like we had `DistanceUnit` before, there is also a class called `AngleUnit`. There are two angle units supported: `DEGREES` and `RADIANS`.¹ `AngleUnit` will make sure everything is normalized (that means it will be within -180 and 180 degrees for `DEGREES` and between - Π and Π for `RADIANS`.²

```
70         return imu.getRobotYawPitchRollAngles().getYaw(angleUnit);
```

Here we get the Yaw (amount it is turning around assuming the robot is staying level), but you could also get the pitch and the roll.

This is much simpler than the old SDK, but even this is good to hide behind a class method so you don't accidentally make a mistake.

¹No love for gradians... - <https://en.wikipedia.org/wiki/Gradian>

²Yes, this means our `RobotLocation` class could have been much simpler.

11. Gyro (IMU)

11.3. OpMode

And here is our OpMode to use it.

Listing 11.2: GyroOpMode.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
7 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard8;
8
9 @TeleOp()
10 public class GyroOpMode extends OpMode {
11     ProgrammingBoard8 board = new ProgrammingBoard8();
12     @Override
13     public void init() {
14         board.init(hardwareMap);
15     }
16
17     @Override
18     public void loop() {
19         telemetry.addData("Our Heading", board.getHeading(AngleUnit.DEGREES));
20     }
21 }
```

Really the only thing that is new here is our telemetry in line 19. Put it on the programming board and turn it around and watch the telemetry change.

11.4. Exercises

1. Change the OpMode to also show the heading in RADIANS as well as DEGREES
2. Make the motor stopped when our heading is 0, go negative when our heading is negative, and positive when our heading is positive.

12. Dealing with State

State is where you remember what you have done and do something different because of what you have done in the past.

12.1. A simple example

So far we have always done something depending on whether a button is currently pressed. What if you wanted it to do something when you first pressed it (such as toggle a light)? Let's do that in an OpMode.

Listing 12.1: ToggleOpMode.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard8;
7
8 @TeleOp()
9 public class ToggleOpMode extends OpMode {
10     ProgrammingBoard8 board = new ProgrammingBoard8();
11     boolean aAlreadyPressed;
12     boolean motorOn;
13     @Override
14     public void init() {
15         board.init(hardwareMap);
16     }
17
18     @Override
19     public void loop() {
20         if(gamepad1.a && !aAlreadyPressed){
21             motorOn = !motorOn;
22             telemetry.addData("Motor", motorOn);
23             if (motorOn) {
24                 board.setMotorSpeed(0.5);
25             } else {
26                 board.setMotorSpeed(0.0);
27             }
28         }
29     }
30 }
```

12. Dealing with State

```
29     aAlreadyPressed = gamepad1.a;  
30 }  
31 }
```

Let's break this down:

```
11     boolean aAlreadyPressed;  
12     boolean motorOn;
```

Here we define two more class members. Since we don't initialize them and they are `boolean` they start out as `false`.

```
20     if(gamepad1.a && !aAlreadyPressed){
```

In this line we are saying if `gamepad1.a` is `true` (pressed) AND `aAlreadyPressed` is NOT `true` (`false`) then... (Remember that `!` means NOT. So it makes `false` turn to `true` and `true` turn to `false`.)

```
21         motorOn = !motorOn;
```

This is a common shorthand. What it does is invert the boolean value. It does exactly the same thing as this code:

```
if(motorOn){  
    motorOn = false;  
}else{  
    motorOn = true;  
}
```

Normally, I like to avoid shortcuts but in this case it is so common that most programmers would prefer the way it is done in the example.

```
23         if (motorOn) {  
24             board.setMotorSpeed(0.5);  
25         } else {  
26             board.setMotorSpeed(0.0);  
27         }  
28     }
```

This actually turns on (or off) the motor. More than one programmer has forgotten this piece and been puzzled when changing the value of a variable called `motorOn` did not actually change the motor.

```
30 }
```

Here we set `aAlreadyPressed` to the value of `gamepad1.a`.

Let's think about how this code works. The first time a user presses the A button, it will come in and `gamepad1.a` will be `true` and `aAlreadyPressed` will be `false`. So it will toggle the `motorOn` class member and change the motor. If the

button is still held down the next time through, `gamepad1.a` will be `true` but so will `aAlreadyPressed` so it won't go into the `if` code block. Eventually our user gets bored and lets go of `gamepad1.a`. The first time through, `gamepad1.a` will be `false` and `aAlreadyPressed` will be `true`. But then `aAlreadyPressed` will be set to `false` and we'll be ready for our user to press `gamepad1.a` again.

Make sure you try this one and play with turning the motor on and off.

12.2. Autonomous state - Example

When writing autonomous code, you want to write it as separate steps. This allows you to test out parts of it separately.

Listing 12.2: `AutoState1.java`

```

1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard8;
7
8 @Autonomous()
9 public class AutoState1 extends OpMode {
10     ProgrammingBoard8 board = new ProgrammingBoard8();
11     int state;
12
13     @Override
14     public void init() {
15         board.init(hardwareMap);
16     }
17
18     @Override
19     public void start() {
20         state = 0;
21     }
22
23     @Override
24     public void loop() {
25         telemetry.addData("State", state);
26         if (state == 0) {
27             board.setServoPosition(0.5);
28             if (board.isTouchSensorPressed()) {
29                 state = 1;
30             }
31         } else if (state == 1) {
32             board.setServoPosition(0.0);
33             if (!board.isTouchSensorPressed()) {

```

12. Dealing with State

```
34         state = 2;
35     }
36 } else if (state == 2) {
37     board.setServoPosition(1.0);
38     board.setMotorSpeed(0.5);
39     if (board.getPotAngle() > 90) {
40         state = 3;
41     }
42 } else if (state == 3) {
43     board.setMotorSpeed(0.0);
44     state = 4;
45 } else {
46     telemetry.addData("Auto", "Finished");
47 }
48 }
49 }
```

Let's break this down:

```
11     int state;
```

Here we create our state variable to hold which state we are in. If we don't assign an initial value it is zero.

```
18     @Override
19     public void start() {
20         state = 0;
21     }
```

Since it should be zero, why do we assign it again in `start()`. Well, imagine that you test your auto. Press Stop, and then test it again. If we don't reset the variable here then it will be whatever it was at the end of your test.

```
25     telemetry.addData("State", state);
```

It is very helpful for debugging to send to the driver station what step in your auto program you are so you can figure out what is going on.

```
26     if (state == 0) {
27         board.setServoPosition(0.5);
28         if (board.isTouchSensorPressed()) {
29             state = 1;
30         }
31     } else if (state == 1) {
```

You can see here an example of using `if/else` chaining. Also, you'll notice that when the touch sensor is pressed, we change the value of state. So the next time through we'll go to the next chain.

But there is another way...

12.2.1. Using the switch statement

In Java, if you are comparing for a number of options you can use a `switch` statement. Here is the same program rewritten with a `switch` statement.

Listing 12.3: AutoState2.java

```

1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard8;
7
8 @Autonomous()
9 public class AutoState2 extends OpMode {
10     ProgrammingBoard8 board = new ProgrammingBoard8();
11     int state;
12
13     @Override
14     public void init() {
15         board.init(hardwareMap);
16     }
17
18     @Override
19     public void start() {
20         state = 0;
21     }
22
23     @Override
24     public void loop() {
25         telemetry.addData("State", state);
26         switch (state) {
27             case 0:
28                 board.setServoPosition(0.5);
29                 if (board.isTouchSensorPressed()) {
30                     state = 1;
31                 }
32                 break;
33             case 1:
34                 board.setServoPosition(0.0);
35                 if (!board.isTouchSensorPressed()) {
36                     state = 2;
37                 }
38                 break;
39             case 2:
40                 board.setServoPosition(1.0);
41                 board.setMotorSpeed(0.5);
42                 if (board.getPotAngle() > 90) {

```

12. Dealing with State

```
43         state = 3;
44     }
45     break;
46 case 3:
47     board.setMotorSpeed(0.0);
48     state = 4;
49     break;
50 default:
51     telemetry.addData("Auto", "Finished");
52 }
53 }
54 }
```

You may think that since this is more lines that it is worse, but let's look at it anyway. (It is personal preference based on which you feel is more readable and you can do things with `if/else` chaining that you can't do with a `switch` statement)

```
26     switch (state) {
```

A `switch` statement is written as `switch( variable )`

```
27         case 0:
```

Each case starts with the `case` keyword followed by the constant followed by a colon :

```
32             break;
```

All code is executed until it hits the `break` statement. At this point, it jumps to the closing brace of the `switch` statement.



If you forget to put a `break` statement in, it will execute the next case as well. There are reasons why you might want to intentionally do this, but if it is intentional make sure you put a comment explaining why you are doing it because most people will assume it was a mistake.

```
50         default:
```

You can (but don't have to) have a `default:` clause. This will be executed if none of the other cases were a match.

But a problem with these two programs is that if you have to put one in the middle, you have to make lots of changes. We can do better....

12.2.2. Switch with strings

Listing 12.4: AutoState3.java

```

1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard8;
7
8 @Autonomous()
9 public class AutoState3 extends OpMode {
10     ProgrammingBoard8 board = new ProgrammingBoard8();
11     String state = "START";
12
13     @Override
14     public void init() {
15         board.init(hardwareMap);
16     }
17
18     @Override
19     public void start() {
20         state = "START";
21     }
22
23     @Override
24     public void loop() {
25         telemetry.addData("State", state);
26         switch (state) {
27             case "START":
28                 board.setServoPosition(0.5);
29                 if (board.isTouchSensorPressed()) {
30                     state = "WAIT_FOR_SENSOR_RELEASE";
31                 }
32                 break;
33             case "WAIT_FOR_SENSOR_RELEASE":
34                 board.setServoPosition(0.0);
35                 if (!board.isTouchSensorPressed()) {
36                     state = "WAIT_FOR_POT_TURN";
37                 }
38                 break;
39             case "WAIT_FOR_POT_TURN":
40                 board.setServoPosition(1.0);
41                 board.setMotorSpeed(0.5);
42                 if (board.getPotAngle() > 90) {
43                     state = "STOP";
44                 }
45                 break;

```

12. Dealing with State

```
46         case "STOP":
47             board.setMotorSpeed(0.0);
48             state = "DONE";
49             break;
50         default:
51             telemetry.addData("Auto", "Finished");
52     }
53 }
54 }
```

Really all we have done is change state from an integer to a String. Now our code is easier to read (called self-documenting) and it is easier to add in another state. (Win-win!!)

But now if we have a typo in a string the compiler won't catch it, and we'll have a problem in our code. What if we could have the readability of strings, but have the compiler catch typos. We can....

12.2.3. Enumerated types

Listing 12.5: AutoState4.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard8;
7
8 @Autonomous()
9 public class AutoState4 extends OpMode {
10     enum State {
11         START,
12         WAIT_FOR_SENSOR_RELEASE,
13         WAIT_FOR_POT_TURN,
14         STOP,
15         DONE
16     }
17
18     ProgrammingBoard8 board = new ProgrammingBoard8();
19     State state = State.START;
20
21     @Override
22     public void init() {
23         board.init(hardwareMap);
24     }
25
26     @Override
```

```

27     public void start() {
28         state = State.START;
29     }
30
31     @Override
32     public void loop() {
33         telemetry.addData("State", state);
34         switch (state) {
35             case START:
36                 board.setServoPosition(0.5);
37                 if (board.isTouchSensorPressed()) {
38                     state = State.WAIT_FOR_SENSOR_RELEASE;
39                 }
40                 break;
41             case WAIT_FOR_SENSOR_RELEASE:
42                 board.setServoPosition(0.0);
43                 if (!board.isTouchSensorPressed()) {
44                     state = State.WAIT_FOR_POT_TURN;
45                 }
46                 break;
47             case WAIT_FOR_POT_TURN:
48                 board.setServoPosition(1.0);
49                 board.setMotorSpeed(0.5);
50                 if (board.getPotAngle() > 90) {
51                     state = State.STOP;
52                 }
53                 break;
54             case STOP:
55                 board.setMotorSpeed(0.0);
56                 state = State.DONE;
57                 break;
58             default:
59                 telemetry.addData("Auto", "Finished");
60         }
61     }
62 }

```

Let's talk through some of this. This actually works exactly the same as our first switch statement except now it is more readable (and we can't assign values to it that we aren't expecting)

```

10     enum State {
11         START,
12         WAIT_FOR_SENSOR_RELEASE,
13         WAIT_FOR_POT_TURN,
14         STOP,
15         DONE
16     }

```

12. Dealing with State

`enum` is short for Enumerated. It is a way we can give names to values. We can add an accessor modifier to this so that the `enum` can be accessed outside the class, but we didn't in this case. By convention, we make all values of an `enum` ALL_CAPS. They have a comma in between each one. Most of the time, it is best to put each one on its own line but you don't have to.

This is declaring a new type called `State`. It is just like making a class. An `enum` is actually a special class that extends `java.lang.Enum`. So yes, you can put methods and class members in it. But you don't need to and typically don't. (So yes, you could put an `enum` in its own file. And yes, you can create a class inside of a class.)

```
28     state = State.START;
```

Now instead of type `String` it is of type `State`. We initialize it to `State.START`. Note that we use the type followed by a dot `.` followed by the `enum` value. You probably noticed that Android Studio helped you type it in. Yet another huge benefit over a `string`.

```
33     telemetry.addData("State", state);
```

One of the really cool things about `enum` is that they implement `toString` automagically so when you print them you get human readable descriptions.

12.3. It's all relative

For a lot of autonomous programs, you may want things to occur for an amount of time or an amount of encoder ticks. To do this we need to save off the time or ticks when we started.

To get the time, all `opModes` have access to `getRuntime()` which returns a `double` that is the number of seconds since the `opMode` was created. This isn't very useful by itself because we don't know how long ago that was before "START" was pushed. There is also a `resetRunTime()` which makes the current time zero. (We often put this in our `start()` method)

Listing 12.6: `AutoTime.java`

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard8;
7
8 @Autonomous()
```

```

9  public class AutoTime extends OpMode {
10      enum State {
11          START,
12          SECOND_STEP,
13          DONE
14      }
15
16      ProgrammingBoard8 board = new ProgrammingBoard8();
17      State state = State.START;
18      double lastTime;
19
20      @Override
21      public void init() {
22          board.init(hardwareMap);
23      }
24
25      @Override
26      public void start() {
27          state = State.START;
28          resetRuntime();
29          lastTime = getRuntime();
30      }
31
32      @Override
33      public void loop() {
34          telemetry.addData("State", state);
35          telemetry.addData("Runtime", getRuntime());
36          telemetry.addData("Time in State", getRuntime() - lastTime);
37          switch (state) {
38              case START:
39                  if (getRuntime() >= 3.0) {
40                      state = State.SECOND_STEP;
41                      lastTime = getRuntime();
42                  }
43                  break;
44              case SECOND_STEP:
45                  if (getRuntime() >= lastTime + 3.0) {
46                      state = State.DONE;
47                      lastTime = getRuntime();
48                  }
49                  break;
50              default:
51                  telemetry.addData("Auto", "Finished");
52          }
53      }
54  }

```

Let's talk through some of the pieces here.

12. Dealing with State

```
25  @Override
26  public void start() {
27      state = State.START;
28      resetRuntime();
29      lastTime = getRuntime();
30  }
```

We are taking advantage of the optional `start()` method here. Remember that this is called ONCE when the OpMode is started. We moved setting of our state variable here because it seemed to make more sense, but leaving it in `init()` will work fine as well. We call `resetRunTime()` which will make our runtime zero. We could have just set `lastTime` to zero here, but we like getting the runtime as it keeps things more similar.

```
35      telemetry.addData("Runtime", getRuntime());
36      telemetry.addData("Time in State", getRuntime() - lastTime);
```

In the first one, we are showing our total runtime. In the second, we show our relative. This is done by keeping track of when we went into a state and then showing the difference.

```
39      if (getRuntime() >= 3.0) {
```

It is **really** important that we compare with a `>=` instead of an `==` because the runtime increments in sub-milliseconds so the odds of it being exact are very low.

```
41          lastTime = getRuntime();
```

When we get ready to change states, we set the `lastTime` variable. We could have called `resetRunTime()` but then we wouldn't be able to know also our runtime as well as time in state.

```
45      if (getRuntime() >= lastTime + 3.0) {
```

Here you'll notice that we are comparing to `lastTime + 3.0` (Obviously if we wanted it to be 5 seconds instead, we would make it `+ 5.0`)

Hopefully, it doesn't take much imagination to do the same thing with encoder ticks (using a `lastEncoder` value)

12.4. Exercises

1. Make a program that ramps your motor to full speed (.25 for 250ms, .50 for 250ms, .75 for 250ms, 1.0) and goes at full speed until the touch sensor is pressed.

2. Make a program that turns the motor until the distance sensor is less than 10cm OR 5 seconds has passed and then turns the servo.

13. Arrays

An array can hold a fixed number of values of one type. Imagine that we had four motors on our drive train. Instead of code like:

```
DcMotor motor1;  
DcMotor motor2;  
DcMotor motor3;  
DcMotor motor4;
```

we could have:

```
DcMotor[] motors = new DcMotor[4]
```

The pattern is:

```
variableType[] variableName = new variableType[arraySize];
```

We can access each motor with an index. The index of an Array start with an index of 0. So it might look like this:

```
motors[0] = hwMap.get(DcMotor.class, "front_left");  
motors[1] = hwMap.get(DcMotor.class, "front_right");  
motors[2] = hwMap.get(DcMotor.class, "back_left");  
motors[3] = hwMap.get(DcMotor.class, "back_right");
```

This may seem interesting, but not all that useful until you start using other things you have learned

```
void stopAllMotors(){  
    for(int i = 0; i < 4; i++){  
        motors[i].setPower(0.0);  
    }  
}
```

This is done so often that Java has a cool shortcut for it. This is called the `for..each`

```
void stopAllMotors(){  
    for(DcMotor motor : motors){  
        motor.setPower(0.0);  
    }  
}
```

13. Arrays

The format here is `for( variableType variableName : arrayName )`

Below is an example op mode using `for`

Listing 13.1: ArrayOpMode.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class ArrayOpMode extends OpMode {
8     String[] words = {"Zeroth", "First", "Second", "Third", "Fourth", "Fifth", "↩"};
9     ↪ Infinity";
10     int wordIndex;
11     double DELAY_SECS = 0.5;
12
13     double nextTime;
14
15     @Override
16     public void init() {
17         wordIndex = 0;
18     }
19
20     @Override
21     public void loop() {
22         if (nextTime < getRuntime()) {
23             wordIndex++;
24             if (wordIndex >= words.length) {
25                 wordIndex = words.length - 1;
26             }
27             nextTime = getRuntime() + DELAY_SECS;
28         }
29         telemetry.addLine(words[wordIndex]);
30     }
31 }
```

13.1. ArrayList

This is all great, but an array can't grow or shrink in size. For that there is `ArrayList`.

```
ArrayList<int> items = new ArrayList<>();
```

The angle brackets are new. That means the type is a “Generic”. What that means is that you specify what type the class uses when you define your

object. So this is creating an `ArrayList` that holds integers. (It could be any type including classes)

A few common methods:

```
items.add(4);    // this adds this element to the end of the list
items.get(index); // returns the element at the index of the list (starts at 0)
items.clear();   // removes all items from list
items.size();    // returns the number of elements in the list

ArrayList<int> secondList = new ArrayList<>();
secondList.add(5);
secondList.add(6);
items.addAll(secondList); // adds all elements in second list to first list
```

13.1.1. Making your own generic class

Making generic classes is not done much in FTC, but I'll include it here for completeness

```
public class MyClass<T>{
    private T member;
    public void set(T var) { member = var; }
    public T get() { return member; }
}
```

Everywhere that `T` is gets replaced when you use the class.

13.2. Exercises

1. Modify the `opMode` to send the chorus of a song you know at a fixed rate on telemetry. Once it gets to the end, it should send it again.
2. Modify your solution for exercise 1 to use `ArrayList<String>` instead of arrays.

14. Inheritance

In Java, when you create a class it always “inherits” from a class. If you don’t use the `extends` keyword then it is inheriting from the `Object` class in Java. So what does this really do?

Let’s start with a simple example and then we’ll show how it can be useful in FTC. (We are going to put all of these in the `org.firstinspires.ftc.teamcode` package (directory))

Listing 14.1: SuperClass.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 public class SuperClass {
4     public String a() {
5         return "a";
6     }
7
8     public String b() {
9         return "b";
10    }
11 }
```

Listing 14.2: ChildClass.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 public class ChildClass extends SuperClass {
4     @Override
5     public String a() {
6         return "A";
7     }
8
9     public String c() {
10        return "c";
11    }
12 }
```

Listing 14.3: SimpleInheritance.java

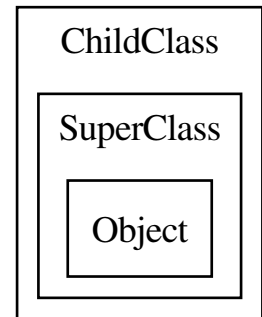
```
1 package org.firstinspires.ftc.teamcode;
2
```

14. Inheritance

```
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class SimpleInheritance extends OpMode {
8     SuperClass super_obj = new SuperClass();
9     ChildClass child_obj = new ChildClass();
10
11     @Override
12     public void init() {
13         telemetry.addData("Parent a", super_obj.a());
14         telemetry.addData("Parent b", super_obj.b());
15         telemetry.addData("Child a", child_obj.a());
16         telemetry.addData("Child b", child_obj.b());
17         telemetry.addData("Child c", child_obj.c());
18     }
19
20     @Override
21     public void loop() {
22
23     }
24 }
```

Can you guess what will show up on the telemetry screen? Try it. Were you right?

You can think about inheritance as your new class containing all of the super class (often called “parent”) plus its new stuff. This is shown in the diagram on the right. If you have a class method with the exact same name and parameters, then it will replace it. You should put an `@Override` annotation on it so that everyone knows that was intentional. (You actually don’t have to but it is good practice to do it.)



14.1. Isa vs. hasa

So now there is a question. If you can get the contents of another class by either deriving from it or having it as a class member, which should you do?

This is typically called “isa” vs “hasa” (short for *is a* and *has a*) So you should derive from it if your class is of that type, but include it if you simply have it as a class member if it just just one of the things you have. Generally I like to

start having it as a class member and only derive from another class if that is really clearly what I need to do.

14.2. So why in the world would you use this?

It is time for the largest word in this book - *polymorphism* - that is. When you are derived from another class you can be treated either as your class or your superclass. This will be the longest example in the book (5 files!!), but I hope it will help you take your programming to the next level.

We are going to make an OpMode that we can use to test out our wiring. (I **HIGHLY** recommend this for your robot. Once you have it, you'll find out how useful it is over and over again to determine whether something is a software or electrical/mechanical problem.

Listing 14.4: TestItem.java

```

1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import org.firstinspires.ftc.robotcore.external.Telemetry;
4
5 abstract public class TestItem {
6     private String description;
7
8     protected TestItem(String description) {
9         this.description = description;
10    }
11
12    public String getDescription() {
13        return description;
14    }
15
16    abstract public void run(boolean on, Telemetry telemetry);
17 }
```

There is really only one new thing in this file but it shows up twice. It is the keyword `abstract`.

```

5 abstract public class TestItem {
```

When `abstract` is before a class it means that no objects can be made of the type of this class. (In other words it is only meant to have other classes derive from it.)

```

16 abstract public void run(boolean on, Telemetry telemetry);
```

14. Inheritance

When `abstract` is before a class method it means that there is no body of this class method, but classes that derive from it that aren't abstract **MUST** implement it. (OpMode defines `init()` and `loop()` as abstract methods). Why in the world would you create a method that does nothing? Well if you require derived classes to have it, then each class can have their own implementation but you are guaranteed they have one.¹

Listing 14.5: TestMotor.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.DcMotor;
4
5 import org.firstinspires.ftc.robotcore.external.Telemetry;
6
7 public class TestMotor extends TestItem {
8     private double speed;
9     private DcMotor motor;
10
11     public TestMotor(String description, double speed, DcMotor motor) {
12         super(description);
13         this.speed = speed;
14         this.motor = motor;
15     }
16
17     @Override
18     public void run(boolean on, Telemetry telemetry) {
19         if (on) {
20             motor.setPower(speed);
21         } else {
22             motor.setPower(0.0);
23         }
24         telemetry.addData("Encoder:", motor.getCurrentPosition());
25     }
26 }
```

A few notes here.

```
7 public class TestMotor extends TestItem {
```

You'll see here that this extends the `TestItem` class we made earlier.

```
12     super(description);
```

The `super` keyword refers to the class we derived from. Since this calls `super()` that is calling our superclass constructor. This is considered the correct way to implement a constructor in a child class.

¹There is another way to accomplish this in Java called Interfaces that we'll discuss in [section 19.4](#).

14.2. So why in the world would you use this?

Everything else in this file we have seen before

Listing 14.6: TestAnalogInput.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.AnalogInput;
4 import com.qualcomm.robotcore.util.Range;
5
6 import org.firstinspires.ftc.robotcore.external.Telemetry;
7
8 public class TestAnalogInput extends TestItem {
9     private AnalogInput analogInput;
10    private double min;
11    private double max;
12
13    public TestAnalogInput(String description, AnalogInput analogInput, double min, ↵
        ↵ double max) {
14        super(description);
15        this.analogInput = analogInput;
16        this.min = min;
17        this.max = max;
18    }
19
20    @Override
21    public void run(boolean on, Telemetry telemetry) {
22        telemetry.addData("Voltage: ", analogInput.getVoltage());
23        telemetry.addData("In Range:",
24            Range.scale(analogInput.getVoltage(),
25                0, analogInput.getMaxVoltage(),
26                min, max));
27    }
28 }
```

This class should look very much like TestMotor to you. The one difference is we always read from the analogInput instead of using an if statement. (A rule I follow is you should only have to tell it to run a test if it causes something to change)

Listing 14.7: ProgrammingBoard9.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.hardware.rev.RevHubOrientationOnRobot;
4 import com.qualcomm.robotcore.hardware.AnalogInput;
5 import com.qualcomm.robotcore.hardware.ColorSensor;
6 import com.qualcomm.robotcore.hardware.DcMotor;
7 import com.qualcomm.robotcore.hardware.DigitalChannel;
8 import com.qualcomm.robotcore.hardware.DistanceSensor;
9 import com.qualcomm.robotcore.hardware.HardwareMap;
```

14. Inheritance

```
10 import com.qualcomm.robotcore.hardware.IMU;
11 import com.qualcomm.robotcore.hardware.Servo;
12 import com.qualcomm.robotcore.util.Range;
13
14 import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
15 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
16
17 import java.util.ArrayList;
18
19 public class ProgrammingBoard9 {
20     private DigitalChannel touchSensor;
21     private DcMotor motor;
22     private double ticksPerRotation;
23     private Servo servo;
24     private AnalogInput pot;
25     private ColorSensor colorSensor;
26     private DistanceSensor distanceSensor;
27     private IMU imu;
28
29     public void init(HardwareMap hwMap) {
30         touchSensor = hwMap.get(DigitalChannel.class, "touch_sensor");
31         touchSensor.setMode(DigitalChannel.Mode.INPUT);
32         motor = hwMap.get(DcMotor.class, "motor");
33         motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
34         ticksPerRotation = motor.getMotorType().getTicksPerRev();
35         servo = hwMap.get(Servo.class, "servo");
36         pot = hwMap.get(AnalogInput.class, "pot");
37
38         colorSensor = hwMap.get(ColorSensor.class, "sensor_color_distance");
39         distanceSensor = hwMap.get(DistanceSensor.class, "sensor_color_distance");
40         imu = hwMap.get(IMU.class, "imu");
41
42         RevHubOrientationOnRobot RevOrientation =
43             new RevHubOrientationOnRobot(RevHubOrientationOnRobot.↵
44                 ↵ LogoFacingDirection.UP,
45                 RevHubOrientationOnRobot.UsbFacingDirection.FORWARD);
46
47         imu.initialize(new IMU.Parameters(RevOrientation));
48     }
49
50     public boolean isTouchSensorPressed() {
51         return !touchSensor.getState();
52     }
53
54     public void setMotorSpeed(double speed) {
55         motor.setPower(speed);
56     }
57
58     public double getMotorRotations() {
```

14.2. So why in the world would you use this?

```
58     return motor.getCurrentPosition() / ticksPerRotation;
59 }
60
61 public void setServoPosition(double position) {
62     servo.setPosition(position);
63 }
64
65 public double getPotAngle() {
66     return Range.scale(pot.getVoltage(), 0, pot.getMaxVoltage(), 0, 270);
67 }
68
69 public int getAmountRed() {
70     return colorSensor.red();
71 }
72
73 public double getDistance(DistanceUnit du) {
74     return distanceSensor.getDistance(du);
75 }
76
77 public double getHeading(AngleUnit angleUnit) {
78     return imu.getRobotYawPitchRollAngles().getYaw(angleUnit);
79 }
80
81 public ArrayList<TestItem> getTests() {
82     ArrayList<TestItem> tests = new ArrayList<>();
83     tests.add(new TestMotor("PB Motor", 0.5, motor));
84     tests.add(new TestAnalogInput("PB Pot", pot, 0, 270));
85     return tests;
86 }
87 }
```

You'll notice that this has a new method at the end of it.

```
82     ArrayList<TestItem> tests = new ArrayList<>();
```

This says we will return an ArrayList containing elements of type TestItem

```
83     tests.add(new TestMotor("PB Motor", 0.5, motor));
```

Here we create the variable tests of type ArrayList<TestItem> and assign a new ArrayList to it. The <> is a shortcut since it is defined on the other side of our assignment.

```
84     tests.add(new TestAnalogInput("PB Pot", pot, 0, 270));
85     return tests;
```

Here we add our two new tests to it. Note that we had to have the new keyword and this calls their constructor. Also note that if we had three motors, we wouldn't need 3 classes - we would just have 3 copies of the line tests.add(new TestMotor.... with a different description, speed, and motor variable.

14. Inheritance

86

```
}
```

and we return our list of tests.

Now for our OpMode

Listing 14.8: TestWiring.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.ProgrammingBoard9;
7 import org.firstinspires.ftc.teamcode.mechanisms.TestItem;
8
9 import java.util.ArrayList;
10
11 @TeleOp
12 public class TestWiring extends OpMode {
13     ProgrammingBoard9 board = new ProgrammingBoard9();
14     ArrayList<TestItem> tests;
15     boolean wasDown, wasUp;
16     int testNum;
17
18     @Override
19     public void init() {
20         board.init(hardwareMap);
21         tests = board.getTests();
22     }
23
24     @Override
25     public void loop() {
26         // move up in the list of test
27         if (gamepad1.dpad_up && !wasUp) {
28             testNum--;
29             if (testNum < 0) {
30                 testNum = tests.size() - 1;
31             }
32         }
33         wasUp = gamepad1.dpad_up;
34
35         // move down in the list of tests
36         if (gamepad1.dpad_down && !wasDown) {
37             testNum++;
38             if (testNum >= tests.size()) {
39                 testNum = 0;
40             }
41         }
42         wasDown = gamepad1.dpad_down;
```

14.2. So why in the world would you use this?

```
43
44     //Put instructions on the telemetry
45     telemetry.addLine("Use Up and Down on D-pad to cycle through choices");
46     telemetry.addLine("Press A to run test");
47     //put the test on the telemetry
48     TestItem currTest = tests.get(testNum);
49     telemetry.addData("Test:", currTest.getDescription());
50     //run or don't run based on a
51     currTest.run(gamepad1.a, telemetry);
52 }
53 }
```

A few things to point out here that I hope will inspire you.

```
14     ArrayList<TestItem> tests;
15     boolean wasDown, wasUp;
16     int testNum;
```

Our list of tests as a member variable, wasDown and wasUp (like in [section 12.1](#)) and testNum to keep track of which test number we are on. For wasDown and wasUp, you see a shortcut where if you have multiple variables of the same type you can define them together with a comma.

```
26     // move up in the list of test
27     if (gamepad1.dpad_up && !wasUp) {
28         testNum--;
29         if (testNum < 0) {
30             testNum = tests.size() - 1;
31         }
32     }
33     wasUp = gamepad1.dpad_up;
34
35     // move down in the list of tests
36     if (gamepad1.dpad_down && !wasDown) {
37         testNum++;
38         if (testNum >= tests.size()) {
39             testNum = 0;
40         }
41     }
42     wasDown = gamepad1.dpad_down;
```

This uses the gamepad1.dpad_up and gamepad1.dpad_down to let us scroll through the list of tests. (Right now there are only 2 but it should give the idea). We made the decision to “wrap” around, but you could make the decision to not wrap. It is up to you.

```
45     telemetry.addLine("Use Up and Down on D-pad to cycle through choices");
46     telemetry.addLine("Press A to run test");
```

14. Inheritance

We haven't used `telemetry.addLine` before but it is just like `telemetry.addData` except it only has one parameter.

```
47      //put the test on the telemetry
48      TestItem currTest = tests.get(testNum);
49      telemetry.addData("Test:", currTest.getDescription());
```

This gets the test and then sends its description after “Test” with telemetry so the driver station can see what test they will be running.

```
51      currTest.run(gamepad1.a, telemetry);
```

`run()` takes a boolean for whether to run the test or not. We just pass in `gamepad1.a` directly here.

14.3. Exercises

1. Add a test for the touchSensor. you'll need a `TestDigitalChannel` class and add it to the `getTests()` method in `ProgrammingBoard`. (No change needed to `OpMode`)
2. Add a test for the servo, you'll need a `TestServo` class - hint your constructor probably needs an “on” value and an “off” value for the servo. You'll also need to add it to the `getTests()`
3. Change `ProgrammingBoard2` through `ProgrammingBoard9` to derive from the one before it (ie, `ProgrammingBoard2` extends `ProgrammingBoard1`) adding only what is necessary each time. Make sure all your `OpModes` still work!! (Hint: you'll have to change private members to protected so the child can access it)²

²The reason we didn't do this in the book is that you would likely only have the most recent version of a mechanism in your code instead of multiple versions.

15. Rumble with Gamepad

Starting with FTC SDK 7.0, the Gamepad is no longer only an input device. If you are using a gamepad that has rumble support, you can send information back to the humans holding the controller. You'll remember that in our `OpMode`, the gamepads are `gamepad1` and `gamepad2`.

There are several simple ways to use the rumble. First, you can simply set the rumble with the amount of time to rumble. If the gamepad is currently rumbling, it is replaced with this amount of time. For example:

Listing 15.1: `GamepadSimpleRumbleOpMode.java`

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class GamepadSimpleRumbleOpMode extends OpMode {
8     @Override
9     public void init() {
10    }
11
12     @Override
13     public void loop() {
14         if (gamepad1.a) {
15             gamepad1.rumble(100);
16         }
17     }
18 }
```

Second, you can send a number of blips. For this to work well, you want to make sure you aren't interrupting your own pattern. Here is an example:

Listing 15.2: `GamepadRumbleBlipsOpMode.java`

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
```

15. Rumble with Gamepad

```
7 public class GamepadRumbleBlipsOpMode extends OpMode {
8     boolean wasA;
9
10    @Override
11    public void init() {
12    }
13
14    @Override
15    public void loop() {
16        if (gamepad1.a && !wasA) {
17            gamepad1.rumbleBlips(3);
18        }
19        wasA = gamepad1.a;
20    }
21 }
```

Third, if you have a gamepad that supports it you can even send different amounts of rumble to the left and right rumble. Be aware that the programmer can make things more subtle than the driver can probably detect during a match. This simple example changes the intensity of the rumble based off of how far the triggers are pushed in.

Listing 15.3: GamepadRumbleOpMode.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.Gamepad;
6
7 @TeleOp()
8 public class GamepadRumbleOpMode extends OpMode {
9     @Override
10    public void init() {
11    }
12
13    @Override
14    public void loop() {
15        telemetry.addLine("Press left trigger for left rumble, and right trigger for ↵
        ↵ right rumble");
16        gamepad1.rumble(gamepad1.left_trigger, gamepad1.right_trigger, Gamepad.↵
        ↵ RUMBLE_DURATION_CONTINUOUS);
17    }
18 }
```

There could be a number of reasons why using gamepad rumble might be useful. For example: the robot might want to let the driver know when it has successfully picked up a piece. Another common reason is to give EVEN more

feedback about entering endgame. For example, here is an opmode that does just that.

Listing 15.4: GamepadRumbleEndGameOpMode.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class GamepadRumbleEndGameOpMode extends OpMode {
8     boolean inEndGame;
9     double endGameTime;
10
11     @Override
12     public void init() {
13         inEndGame = false;
14     }
15
16     @Override
17     public void start() {
18         endGameTime = getRuntime() + 90;
19     }
20
21
22     @Override
23     public void loop() {
24         if ((getRuntime() > endGameTime) && !inEndGame) {
25             gamepad1.rumbleBlips(3);
26             inEndGame = true;
27         }
28     }
29 }

```

15.1. Exercises

1. Write a program that rumbles when the touch sensor is first pressed. (Use one of the programming board files from earlier that has a touch sensor)

16. Computer Vision

Typically in FTC, there is a task where you need to determine the placement of an object in order to do something in autonomous. While there are lots of ways to solve this, often OpenCV (CV stands for Computer Vision) is a simple and elegant way to solve the problem.

Prior to FTC SDK 8.2 (in preparation for the CENTERSTAGE season - 2023-2024), you had to install EasyOpenCV. However, now it is included in the FTC SDK. Not only that, but there is new code for a VisionPortal which makes things like AprilTags and OpenCV pipelines much easier. It also makes TensorFlow easier, but our experience is that AprilTags and OpenCV are typically much easier to get to work than TensorFlow.

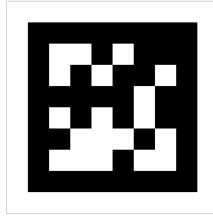
OpenCV basically looks at each frame from the camera and makes a matrix out of the color values from the camera for each pixel. Once we have sensor data as numbers, we own it and can do a lot with it.

OpenCV is a HUGE subject, and we will not fully address it in this short chapter. There are entire books written about OpenCV. Luckily for most years, you can accomplish the autonomous determination of the placement of an object with a very simple program.

But before we even look at OpenCV, let's take a look at AprilTags where the FTC SDK provides you with the ability to detect AprilTags without having to even write a pipeline.

16.1. April Tags

Developed at the University of Michigan, AprilTag is like a 2D barcode or a simplified QR Code. It contains a numeric ID code and can be used for location and orientation. Here is an example of the AprilTag from the family tag36h11 (what FTC has said will be used for CENTERSTAGE) for id 42.



For more information, see the excellent section on FTCDocs: https://ftc-docs.firstinspires.org/en/latest/apriltag/vision_portal/apriltag_intro/apriltag-intro.html

16.1.1. The Opmode

Don't panic when you see all these new things, we'll explain what they do.

Listing 16.1: SimpleAprilTags.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6 import org.firstinspires.ftc.robotcore.external.hardware.camera.WebcamName;
7 import org.firstinspires.ftc.vision.VisionPortal;
8 import org.firstinspires.ftc.vision.apriltag.AprilTagDetection;
9 import org.firstinspires.ftc.vision.apriltag.AprilTagProcessor;
10
11 import java.util.List;
12
13 @Autonomous()
14 public class SimpleAprilTags extends OpMode {
15     private AprilTagProcessor aprilTagProcessor;
16     private VisionPortal visionPortal;
17
18     @Override
19     public void init() {
20         WebcamName webcamName = hardwareMap.get(WebcamName.class, "Webcam 1");
21         aprilTagProcessor = AprilTagProcessor.easyCreateWithDefaults();
22         visionPortal = VisionPortal.easyCreateWithDefaults(webcamName, ↵
                ↵ aprilTagProcessor);
23     }
24
25     @Override
26     public void init_loop() {
27         List<AprilTagDetection> currentDetections = aprilTagProcessor.getDetections()↵
                ↵ ;
28         StringBuilder idsFound = new StringBuilder();
```

```

29     for (AprilTagDetection detection : currentDetections) {
30         idsFound.append(detection.id);
31         idsFound.append(' ');
32     }
33     telemetry.addData("April Tags", idsFound);
34 }
35
36 @Override
37 public void start() {
38     visionPortal.stopStreaming();
39 }
40
41 @Override
42 public void loop() {
43 }
44 }

```

You probably remember that all of the `import` statements are automatically added by Android Studio.

```
13 @Autonomous()
```

Unlike the rest of our `OpModes`, we are setting this one up as an `Autonomous OpMode`.

```
15     private AprilTagProcessor aprilTagProcessor;
```

We are creating a variable of type `AprilTagProcessor` that is named `aprilTagProcessor`. This is a processor provided by the FTC SDK that will look for April Tags and tell us about all the ones it found.

```
16     private VisionPortal visionPortal;
```

We are creating a variable of type `VisionPortal` that is named `visionPortal`. This is provided by the FTC SDK and gives us an easy way to handle the camera and get information about the stream.

Inside of `init()`, we handle the code we only want executed once.

```
20     WebcamName webcamName = hardwareMap.get(WebcamName.class, "Webcam 1");
```

This line of code creates our webcam and uses our old friend `getting` from the `hardwareMap`. Here we used the default “Webcam 1”, but it has to match what your webcam is in your config file on the robot.

```
21     aprilTagProcessor = AprilTagProcessor.easyCreateWithDefaults();
```

This creates our processor. In this case, we are creating the `AprilTagProcessor` with all the defaults. To see examples of how to create with other than the

16. Computer Vision

defaults, you can look at the `ConceptAprilTag` sample that comes with the FTC SDK.

```
22 visionPortal = VisionPortal.easyCreateWithDefaults(webcamName, ↵  
    ↵ aprilTagProcessor);
```

This creates our vision Portal. In this case, we are creating it with all defaults and then passing in our webCam and our aprilTagProcessor. That is all that is needed in our `init` method.

The next thing you probably noticed is we have code in the `init_loop()` method. This is called over and over again after the `opMode` has gone through `init` but before play has been pressed. For many years, you want to do your vision processing here so as soon as autonomous starts you have already acquired the vision of what to do.

```
27 List<AprilTagDetection> currentDetections = aprilTagProcessor.getDetections()↵  
    ↵ ;
```

This line is getting us the list of april tags that were detected by the aprilTagProcessor. (This list could be empty) You may not have seen the angle brackets `<` and `>` before. This lets us say that the List has to contain elements of a certain type which is inside the angle brackets.¹

```
28 StringBuilder idsFound = new StringBuilder();
```

This line is creating a variable `idsFound` of type `StringBuilder`. The reason this is not of type `String` is that we are going to do a lot of appending to it and this is a more efficient way of doing that.

```
29 for (AprilTagDetection detection : currentDetections) {
```

This line is saying for each detection found, go through the loop with the variable `detection` equal to that place in the list.

```
30 idsFound.append(detection.id);  
31 idsFound.append(' ');
```

These lines put the id of the april tag found followed by a space. The reason we follow it by the space is because otherwise we wouldn't be able to tell the difference of one found with an id of 13 or two found with ids of 1 and 3. The reason we use a space instead of a comma is that nobody will notice a space at the end, but a comma at the end looks strange.

```
33 telemetry.addData("April Tags", idsFound);
```

¹Technically this is a template class, where the class gets built when you use it for the first time. But templating is outside of the scope of this book.

This puts the string out on telemetry. If no ids were found, then the variable `idsFound` will be empty.

We also have code in our `start()` method. You'll remember that it is called once when the play button is pressed.

38

```
visionPortal.stopStreaming();
```

Here we stop the streaming, so we won't be using up the processor power once our autonomous starts. That is all you have to do for AprilTags.

16.2. The empty processor

That is very cool, but what about making our own vision processor. It turns out we can do that!

Every processor has to implement the `VisionProcessor` interface. That means you have to have a method for each method described in the interface. This is similar to deriving from an abstract class like we do with `OpMode`. The main differences are that a class can only derive from one class, but can implement multiple interfaces. Instead of using the keyword `extends` like you do for deriving from a class, you use the keyword `implements`.

In order to implement the `VisionProcessor` interface, your class has to have 3 methods. Below is the full code for an empty processor.

Listing 16.2: `EmptyProcessor.java`

```
1 package org.firstinspires.ftc.teamcode.processors;
2
3 import android.graphics.Canvas;
4
5 import org.firstinspires.ftc.robotcore.internal.camera.calibration.CameraCalibration;
6 import org.firstinspires.ftc.vision.VisionProcessor;
7 import org.opencv.core.Mat;
8
9 public class EmptyProcessor implements VisionProcessor {
10     @Override
11     public void init(int width, int height, CameraCalibration calibration) {
12
13     }
14
15     @Override
16     public Object processFrame(Mat frame, long captureTimeNanos) {
17         return null;
18     }
19
20     @Override
```

16. Computer Vision

```
21 public void onDrawFrame(Canvas canvas, int onscreenWidth, int onscreenHeight, ↵  
    ↵ float scaleBmpPxToCanvasPx, float scaleCanvasDensity, Object userContext)↵  
    ↵ {  
22 }  
23 }
```

The first method is `init`. It takes the width, the height, and the calibration (camera calibration data). This is guaranteed to be called before the other methods and is a good place for things you need to do once.

The second method is `processFrame`. It gets called once per frame of video. It receives two parameters. The first is `frame` of type `Mat` which is a `Matrix` that describes the frame. This is an OpenCV type. The second is the time at which the frame was captured in nanoseconds. A nano second is one billionth of a second. You will probably get this call about 30 times a second, so expect this number to be significantly larger each time. Most of the time, you probably can ignore this. It returns a variable which is an `Object` which means you can return anything that is derived from an `Object`. All classes in Java derive from `Object` even without your saying that explicitly. One very important thing that is different about `processFrame` compared to a typical OpenCV pipeline is that you don't want to modify the frame being passed in because all other processors will also be getting the same frame.

The third method is `onDrawFrame`. This allows us to draw on top of the frame to be seen both on the camera view and on the image you can see on the driver station.

For the camera view, you can either plug an HDMI cable into the Control Hub (easiest way) or you can use `scrcpy` - <https://github.com/Genymobile/scrcpy> which will let you see the screen wirelessly.

On the driver station, you can click on the three dots for the main menu to view the camera stream (this only works while in `init`, once you click play it won't let you see this.) This only updates each time you touch the screen.

`OnDrawFrame` receives several parameters.

1. `canvas` of type `Canvas` which is what we will draw on.
2. `onScreenWidth` - this is the width of the canvas in pixels
3. `onScreenHeight` - this is the height of the canvas in pixels
4. `scaleBmpPxToCanvasPx` - this helps us convert from the coordinates in the `processFrame` to those on the canvas
5. `scaleCanvasDensity` - this lets us draw text annotations that are the same regardless of the screen size

6. `userContext`- this has the object that was returned by the previous `processFrame`.

That is all there is to a vision processor. With the basics out of the way, let's make our first vision processor.

16.3. Our first vision processor

For our first vision processor, we'll simply draw a rectangle on top of the camera feed. This will require us to create a processor and make an `opMode` in order to use the processor.

16.3.1. The processor

Here is the full processor.

Listing 16.3: `DrawRectangleProcessor.java`

```

1 package org.firstinspires.ftc.teamcode.processors;
2
3 import android.graphics.Canvas;
4 import android.graphics.Color;
5 import android.graphics.Paint;
6
7 import org.firstinspires.ftc.robotcore.internal.camera.calibration.CameraCalibration;
8 import org.firstinspires.ftc.vision.VisionProcessor;
9 import org.opencv.core.Mat;
10 import org.opencv.core.Rect;
11
12 public class DrawRectangleProcessor implements VisionProcessor {
13     public Rect rect = new Rect(20, 20, 50, 50);
14
15     @Override
16     public void init(int width, int height, CameraCalibration calibration) {
17     }
18
19     @Override
20     public Object processFrame(Mat frame, long captureTimeNanos) {
21         return null;
22     }
23
24     private android.graphics.Rect makeGraphicsRect(Rect rect, float ↵
        ↵ scaleBmpPxToCanvasPx) {
25         int left = Math.round(rect.x * scaleBmpPxToCanvasPx);
26         int top = Math.round(rect.y * scaleBmpPxToCanvasPx);
27         int right = left + Math.round(rect.width * scaleBmpPxToCanvasPx);

```

16. Computer Vision

```
28         int bottom = top + Math.round(rect.height * scaleBmpPxToCanvasPx);
29
30         return new android.graphics.Rect(left, top, right, bottom);
31     }
32
33     @Override
34     public void onDrawFrame(Canvas canvas, int onscreenWidth, int onscreenHeight, ↵
        ↵ float scaleBmpPxToCanvasPx, float scaleCanvasDensity, Object userContext)↵
        ↵ {
35         Paint rectPaint = new Paint();
36         rectPaint.setColor(Color.RED);
37         rectPaint.setStyle(Paint.Style.STROKE);
38         rectPaint.setStrokeWidth(scaleCanvasDensity * 4);
39
40         canvas.drawRect(makeGraphicsRect(rect, scaleBmpPxToCanvasPx), rectPaint);
41     }
42 }
```

We'll start with the variables we have as members of the class

```
13     public Rect rect = new Rect(20, 20, 50, 50);
```

This creates a rectangle. This is in camera coordinates. One thing we have to be careful of is that both `android.graphics` and `org.opencv.core` have `Rect` classes and they are NOT the same. This one should be the OpenCV one. This is in the order: x, y, width, height of the rectangle. (where x and y are of the upper left corner)

```
24     private android.graphics.Rect makeGraphicsRect(Rect rect, float ↵
        ↵ scaleBmpPxToCanvasPx) {
25         int left = Math.round(rect.x * scaleBmpPxToCanvasPx);
26         int top = Math.round(rect.y * scaleBmpPxToCanvasPx);
27         int right = left + Math.round(rect.width * scaleBmpPxToCanvasPx);
28         int bottom = top + Math.round(rect.height * scaleBmpPxToCanvasPx);
29
30         return new android.graphics.Rect(left, top, right, bottom);
31     }
```

This converts from our OpenCV camera rectangle to an `android.graphics.Rect`. These are different because the OpenCV rect has top, left, width and height while the android graphics one has left, top, right, and bottom. Also the OpenCV one is in camera coordinates while the android one is in screen coordinates. You'll notice that we use `Math.round` which converts from a float to an integer (rounding down or up like you would expect based off the input. ie. 1.6 becomes 2 and 1.4 becomes 1.) We are multiplying the openCV rectangle by `scaleBmpPxToCanvasPx` for the conversion.

```

34     public void onDrawFrame(Canvas canvas, int onscreenWidth, int onscreenHeight, ↵
        ↵ float scaleBmpPxToCanvasPx, float scaleCanvasDensity, Object userContext)↵
        ↵ {
35         Paint rectPaint = new Paint();
36         rectPaint.setColor(Color.RED);
37         rectPaint.setStyle(Paint.Style.STROKE);
38         rectPaint.setStrokeWidth(scaleCanvasDensity * 4);
39
40         canvas.drawRect(makeGraphicsRect(rect, scaleBmpPxToCanvasPx), rectPaint);
41     }

```

In this method, we go ahead and setup the `rectPaint` variable. You can see we set the color, the style (`STROKE` means just outline it, the other options are `FILL` (the default) or `STROKE_AND_FILL`), and the `strokeWidth`. The reason we set it to 4 times the `scaleCanvasDensity` is that way it will be easy to see regardless of how large the image we are drawing on is.²

We then draw the rectangle.

16.3.2. The opmode

Here is the entire opmode, but you'll see that the only differences between it and the `opMode` for using the `AprilTagProcessor` are that this uses our `DrawRectangleProcessor` instead.

Listing 16.4: `SimpleOpenCV.java`

```

1  package org.firstinspires.ftc.teamcode.opmodes;
2
3  import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6  import org.firstinspires.ftc.robotcore.external.hardware.camera.WebcamName;
7  import org.firstinspires.ftc.teamcode.processors.DrawRectangleProcessor;
8  import org.firstinspires.ftc.vision.VisionPortal;
9
10 @Autonomous()
11 public class SimpleOpenCV extends OpMode {
12
13     private DrawRectangleProcessor drawRectangleProcessor;
14     private VisionPortal visionPortal;
15
16     @Override
17     public void init() {

```

²Some of you are wondering why we do this every time we call `onDrawFrame`. We could indeed check to see if `rectPaint` is set and if not, go set it. That would be better, but I think less clear for teaching.

16. Computer Vision

```
18     drawRectangleProcessor = new DrawRectangleProcessor();
19     visionPortal = VisionPortal.easyCreateWithDefaults(
20         hardwareMap.get(WebcamName.class, "Webcam 1"), drawRectangleProcessor↵↵
21         ↵ );
22
23     @Override
24     public void init_loop() {
25     }
26
27     @Override
28     public void start() {
29         visionPortal.stopStreaming();
30     }
31
32     @Override
33     public void loop() {
34
35     }
36 }
```

16.3.3. Bonus - using EOCVSim (Optional)

One of the fantastic things about the FTC community is that people share what they have made to help others. There is a simulator that people have made that allow you to test your processors (or OpenCV pipelines) with images (either photos or videos) that you have taken. This allows you to work even though people may be busy with the robot.

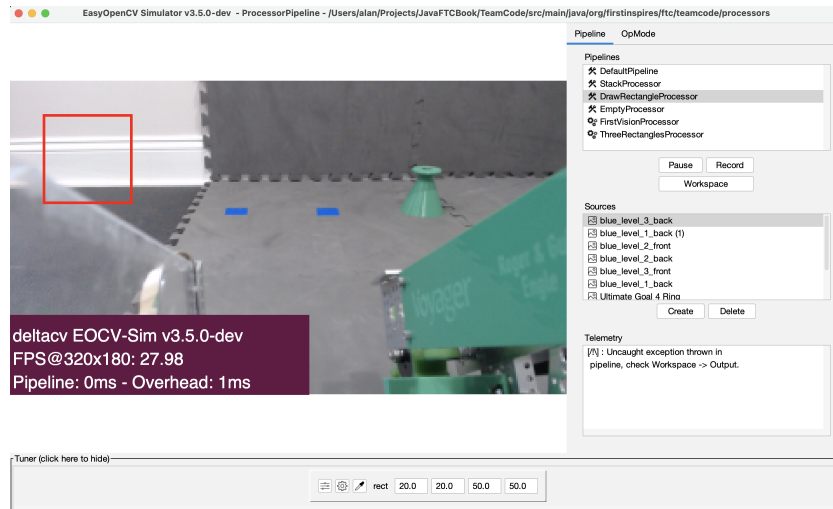
You can get the EasyOpenCV Simulator from: <https://github.com/deltacv/EOCV-Sim> It is a JAR file which you can run directly.

A few tips:

- Put your processors in a directory called `processors` under your teamcode. This will allow you to quickly point the simulator to that directory and it will rebuild them as the source code changes.
- Make sure to make variables public that you want to be able to change in the simulator. (After you are done tuning, you may move them back to no longer being public.)
- Changing values in the simulator does NOT change them in your code. You need to make sure to do that as well.
- You can use the simulator with the webcam on your computer, but even more helpful is the ability to point it to still images or video that you have

recorded. You can take a picture from where the camera on the robot is and use that.

- Thanks to FTC Team 14169 for providing some images of their TSE from the FreightFrenzy season. These are located in the /OpenCVImages directory at <https://github.com/alan412/LearnJavaForFTC> You can use these to try out your processors.



If you notice the Tuner section, you can change values and see them take effect. Be aware that this is only for variables that are `public` and that they only take effect from that point forward. For example, if you are trying to be clever and create your android graphics rectangle only once and not every time through then it won't update.³

16.4. Expanding to 3 rectangles

We are going to expand to 3 rectangles where we can change which one is selected and draw that one in a different color. Don't forget that you'll also have to change your `opMode` to use this processor. First here is all the code for the new processor:

Listing 16.5: `ThreeRectanglesProcessor.java`

```
1 package org.firstinspires.ftc.teamcode.processors;
```

³Don't ask how I found this out....

16. Computer Vision

```
2
3 import android.graphics.Canvas;
4 import android.graphics.Color;
5 import android.graphics.Paint;
6
7 import org.firstinspires.ftc.robotcore.internal.camera.calibration.CameraCalibration;
8 import org.firstinspires.ftc.vision.VisionProcessor;
9 import org.opencv.core.Mat;
10 import org.opencv.core.Rect;
11
12 public class ThreeRectanglesProcessor implements VisionProcessor {
13     public Rect rectLeft = new Rect(110, 42, 40, 40);
14     public Rect rectMiddle = new Rect(160, 42, 40, 40);
15     public Rect rectRight = new Rect(210, 42, 40, 40);
16     Selected selection = Selected.NONE;
17
18     @Override
19     public void init(int width, int height, CameraCalibration calibration) {
20     }
21
22     @Override
23     public Object processFrame(Mat frame, long captureTimeNanos) {
24         return null;
25     }
26
27     private android.graphics.Rect makeGraphicsRect(Rect rect, float ↵
        ↵ scaleBmpPxToCanvasPx) {
28         int left = Math.round(rect.x * scaleBmpPxToCanvasPx);
29         int top = Math.round(rect.y * scaleBmpPxToCanvasPx);
30         int right = left + Math.round(rect.width * scaleBmpPxToCanvasPx);
31         int bottom = top + Math.round(rect.height * scaleBmpPxToCanvasPx);
32
33         return new android.graphics.Rect(left, top, right, bottom);
34     }
35
36     @Override
37     public void onDrawFrame(Canvas canvas, int onscreenWidth, int onscreenHeight, ↵
        ↵ float scaleBmpPxToCanvasPx, float scaleCanvasDensity, Object userContext)↵
        ↵ {
38         Paint selectedPaint = new Paint();
39         selectedPaint.setColor(Color.RED);
40         selectedPaint.setStyle(Paint.Style.STROKE);
41         selectedPaint.setStrokeWidth(scaleCanvasDensity * 4);
42
43         Paint nonSelectedPaint = new Paint(selectedPaint);
44         nonSelectedPaint.setColor(Color.GREEN);
45
46         android.graphics.Rect drawRectangleLeft = makeGraphicsRect(rectLeft, ↵
            ↵ scaleBmpPxToCanvasPx);
```

```

47     android.graphics.Rect drawRectangleMiddle = makeGraphicsRect(rectMiddle, ↵
        ↵ scaleBmpPxToCanvasPx);
48     android.graphics.Rect drawRectangleRight = makeGraphicsRect(rectRight, ↵
        ↵ scaleBmpPxToCanvasPx);
49
50     switch (selection) {
51         case LEFT:
52             canvas.drawRect(drawRectangleLeft, selectedPaint);
53             canvas.drawRect(drawRectangleMiddle, nonSelectedPaint);
54             canvas.drawRect(drawRectangleRight, nonSelectedPaint);
55             break;
56         case MIDDLE:
57             canvas.drawRect(drawRectangleLeft, nonSelectedPaint);
58             canvas.drawRect(drawRectangleMiddle, selectedPaint);
59             canvas.drawRect(drawRectangleRight, nonSelectedPaint);
60             break;
61         case RIGHT:
62             canvas.drawRect(drawRectangleLeft, nonSelectedPaint);
63             canvas.drawRect(drawRectangleMiddle, nonSelectedPaint);
64             canvas.drawRect(drawRectangleRight, selectedPaint);
65             break;
66         case NONE:
67             canvas.drawRect(drawRectangleLeft, nonSelectedPaint);
68             canvas.drawRect(drawRectangleMiddle, nonSelectedPaint);
69             canvas.drawRect(drawRectangleRight, nonSelectedPaint);
70             break;
71     }
72 }
73
74 public enum Selected {
75     NONE,
76     LEFT,
77     MIDDLE,
78     RIGHT
79 }
80 }

```

Most of this should look incredibly familiar. We created an enumerated type like we talked about in [subsection 12.2.3](#). We now have 3 rectangles instead of 1, and in our `onDrawFrame` we draw the one that is selected in a different color.

Until we add the code in 16.5 to do the actual computer vision, all three of these rectangles will be drawn in green because the member variable `selection` is set to `Selected.NONE`. If you set it to `Selected.LEFT`, `Selected.MIDDLE`, or `Selected.RIGHT` then you would see the corresponding rectangle change to be red.

16.5. Actual computer vision...

Here is a processor that does actual computer vision. Again, don't forget to change the opMode to use this one.

Listing 16.6: FirstVisionProcessor.java

```
1 package org.firstinspires.ftc.teamcode.processors;
2
3 import android.graphics.Canvas;
4 import android.graphics.Color;
5 import android.graphics.Paint;
6
7 import org.firstinspires.ftc.robotcore.internal.camera.calibration.CameraCalibration;
8 import org.firstinspires.ftc.vision.VisionProcessor;
9 import org.opencv.core.Core;
10 import org.opencv.core.Mat;
11 import org.opencv.core.Rect;
12 import org.opencv.core.Scalar;
13 import org.opencv.imgproc.Imgproc;
14
15 public class FirstVisionProcessor implements VisionProcessor {
16     public Rect rectLeft = new Rect(110, 42, 40, 40);
17     public Rect rectMiddle = new Rect(160, 42, 40, 40);
18     public Rect rectRight = new Rect(210, 42, 40, 40);
19     Selected selection = Selected.NONE;
20
21     Mat submat = new Mat();
22     Mat hsvMat = new Mat();
23
24     @Override
25     public void init(int width, int height, CameraCalibration calibration) {
26     }
27
28     @Override
29     public Object processFrame(Mat frame, long captureTimeNanos) {
30         Imgproc.cvtColor(frame, hsvMat, Imgproc.COLOR_RGB2HSV);
31
32         double satRectLeft = getAvgSaturation(hsvMat, rectLeft);
33         double satRectMiddle = getAvgSaturation(hsvMat, rectMiddle);
34         double satRectRight = getAvgSaturation(hsvMat, rectRight);
35
36         if ((satRectLeft > satRectMiddle) && (satRectLeft > satRectRight)) {
37             return Selected.LEFT;
38         } else if ((satRectMiddle > satRectLeft) && (satRectMiddle > satRectRight)) {
39             return Selected.MIDDLE;
40         }
41         return Selected.RIGHT;
42     }
```

```

43
44 protected double getAvgSaturation(Mat input, Rect rect) {
45     submat = input.submat(rect);
46     Scalar color = Core.mean(submat);
47     return color.val[1];
48 }
49
50 private android.graphics.Rect makeGraphicsRect(Rect rect, float ↵
    ↵ scaleBmpPxToCanvasPx) {
51     int left = Math.round(rect.x * scaleBmpPxToCanvasPx);
52     int top = Math.round(rect.y * scaleBmpPxToCanvasPx);
53     int right = left + Math.round(rect.width * scaleBmpPxToCanvasPx);
54     int bottom = top + Math.round(rect.height * scaleBmpPxToCanvasPx);
55
56     return new android.graphics.Rect(left, top, right, bottom);
57 }
58
59 @Override
60 public void onDrawFrame(Canvas canvas, int onscreenWidth, int onscreenHeight, ↵
    ↵ float scaleBmpPxToCanvasPx, float scaleCanvasDensity, Object userContext)↵
    ↵ {
61     Paint selectedPaint = new Paint();
62     selectedPaint.setColor(Color.RED);
63     selectedPaint.setStyle(Paint.Style.STROKE);
64     selectedPaint.setStrokeWidth(scaleCanvasDensity * 4);
65
66     Paint nonSelectedPaint = new Paint(selectedPaint);
67     nonSelectedPaint.setColor(Color.GREEN);
68
69     android.graphics.Rect drawRectangleLeft = makeGraphicsRect(rectLeft, ↵
        ↵ scaleBmpPxToCanvasPx);
70     android.graphics.Rect drawRectangleMiddle = makeGraphicsRect(rectMiddle, ↵
        ↵ scaleBmpPxToCanvasPx);
71     android.graphics.Rect drawRectangleRight = makeGraphicsRect(rectRight, ↵
        ↵ scaleBmpPxToCanvasPx);
72
73     selection = (Selected) userContext;
74     switch (selection) {
75         case LEFT:
76             canvas.drawRect(drawRectangleLeft, selectedPaint);
77             canvas.drawRect(drawRectangleMiddle, nonSelectedPaint);
78             canvas.drawRect(drawRectangleRight, nonSelectedPaint);
79             break;
80         case MIDDLE:
81             canvas.drawRect(drawRectangleLeft, nonSelectedPaint);
82             canvas.drawRect(drawRectangleMiddle, selectedPaint);
83             canvas.drawRect(drawRectangleRight, nonSelectedPaint);
84             break;
85         case RIGHT:

```

16. Computer Vision

```
86         canvas.drawRect(drawRectangleLeft, nonSelectedPaint);
87         canvas.drawRect(drawRectangleMiddle, nonSelectedPaint);
88         canvas.drawRect(drawRectangleRight, selectedPaint);
89         break;
90     case NONE:
91         canvas.drawRect(drawRectangleLeft, nonSelectedPaint);
92         canvas.drawRect(drawRectangleMiddle, nonSelectedPaint);
93         canvas.drawRect(drawRectangleRight, nonSelectedPaint);
94         break;
95     }
96 }
97
98 public Selected getSelection() {
99     return selection;
100 }
101
102 public enum Selected {
103     NONE,
104     LEFT,
105     MIDDLE,
106     RIGHT
107 }
108 }
```

We now have code inside our processFrame

```
30     Imgproc.cvtColor(frame, hsvMat, Imgproc.COLOR_RGB2HSV);
```

This converts the colorspace from RGB (Red, green, blue) to HSV (Hue, Saturation, Value). This is often a really useful colorspace for doing image detection in FTC since the background is the gray mats.⁴ The Hue is the shade of color (useful when we are looking for something specific), the Saturation is how much of the color there is (so gray has a low saturation), and value is how bright it is. (This is sometimes called Brightness which means HSB)

```
32     double satRectLeft = getAvgSaturation(hsvMat, rectLeft);
33     double satRectMiddle = getAvgSaturation(hsvMat, rectMiddle);
34     double satRectRight = getAvgSaturation(hsvMat, rectRight);
```

Here we get the average saturation of each rectangle by calling a method we made to get the average saturation of a rectangle. Let's go there for a second

```
44     protected double getAvgSaturation(Mat input, Rect rect) {
45         submat = input.submat(rect);
46         Scalar color = Core.mean(submat);
47         return color.val[1];
48     }
```

⁴In the real world YUV or YCbCr is often more useful.

Here we create a submatrix based off of the rectangle passed in. We then get the average (or mean) of each pixel in this rectangle. Since we are working in HSV, the 0th is Hue, the 1st is Saturation, and the 2nd is Value.

```

36     if ((satRectLeft > satRectMiddle) && (satRectLeft > satRectRight)) {
37         return Selected.LEFT;
38     } else if ((satRectMiddle > satRectLeft) && (satRectMiddle > satRectRight)) {
39         return Selected.MIDDLE;
40     }
41     return Selected.RIGHT;

```

This returns which rectangle has the highest saturation (ie, the least gray). By doing it this way, we can detect any type of TSE ⁵. Often the trick in using OpenCV is figuring out what is the easiest way to look for what you need to make a decision. You'll notice that if we don't know, we return that it is the one on the right. This is because if we can't tell, we should just guess one because we have a 1 out of 3 chance of being correct. For diagnostics, you might want to not do this.

```

73     selection = (Selected) userContext;

```

Here we take the userContext and cast it to what we know is returned (our enumerated type) and we save it in our class.

```

98     public Selected getSelection() {
99         return selection;
100     }

```

We add this `getSelection` method so that the `opMode` can get the selection to make a decision.

16.5.1. The opmode

Listing 16.7: FirstVisionOpmode.java

```

1  package org.firstinspires.ftc.teamcode.opmodes;
2
3  import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
4  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5
6  import org.firstinspires.ftc.robotcore.external.hardware.camera.WebcamName;
7  import org.firstinspires.ftc.teamcode.processors.FirstVisionProcessor;
8  import org.firstinspires.ftc.vision.VisionPortal;
9
10 @Autonomous()

```

⁵unless someone makes a TSE gray

16. Computer Vision

```
11 public class FirstVisionOpmode extends OpMode {
12
13     private FirstVisionProcessor visionProcessor;
14     private VisionPortal visionPortal;
15
16     @Override
17     public void init() {
18         visionProcessor = new FirstVisionProcessor();
19         visionPortal = VisionPortal.easyCreateWithDefaults(
20             hardwareMap.get(WebcamName.class, "Webcam 1"), visionProcessor);
21     }
22
23     @Override
24     public void init_loop() {
25     }
26
27     @Override
28     public void start() {
29         visionPortal.stopStreaming();
30     }
31
32     @Override
33     public void loop() {
34         telemetry.addData("Identified", visionProcessor.getSelection());
35     }
36 }
```

In our opmode, you'll see that in the `loop` method we send to telemetry what the vision processor saw before the opmode was started.

16.6. Exercises

1. Using our `FirstVisionProcessor`, and the programming board from [chapter 8](#) write an opmode that moves a servo to 0 if the TSE is in the left rectangle, 90 if it is in the middle rect, and 180 if it is in the right rectangle.

17. Javadoc

We talked earlier about a special kind of comment called a Javadoc. There are several huge benefits from commenting this way. The FTC SDK is commented in this way and that is what generates the documentation.

1. Android Studio will pick it up and give help to people using your classes
2. Autogenerating documentation that will amaze the judges

There are 3 places you can put a Javadoc comment.

1. Before your class
2. Before each class member
3. Before each class method

A Javadoc comment looks like this:

```
/**
 * This is a javadoc comment
 */
```

If you write your class method declaration first, and then type in a `/**` above it then it will automatically put `@param` for each parameter you have and a `@return` if your method returns anything.

```
/**
 * gets our imu heading
 *
 * @param angleUnit this determines the angle unit (degrees/radians) that it will ↵
 *   ↵ return in
 * @return returns the current angle with the offset in the angleUnit specified
 */
private double getHeading(AngleUnit angleUnit) {
    Orientation angles;
    angles = imu.getAngularOrientation(AxesReference.INTRINSIC,
                                     AxesOrder.ZYX,
                                     angleUnit);

    return angles.firstAngle;
}
```

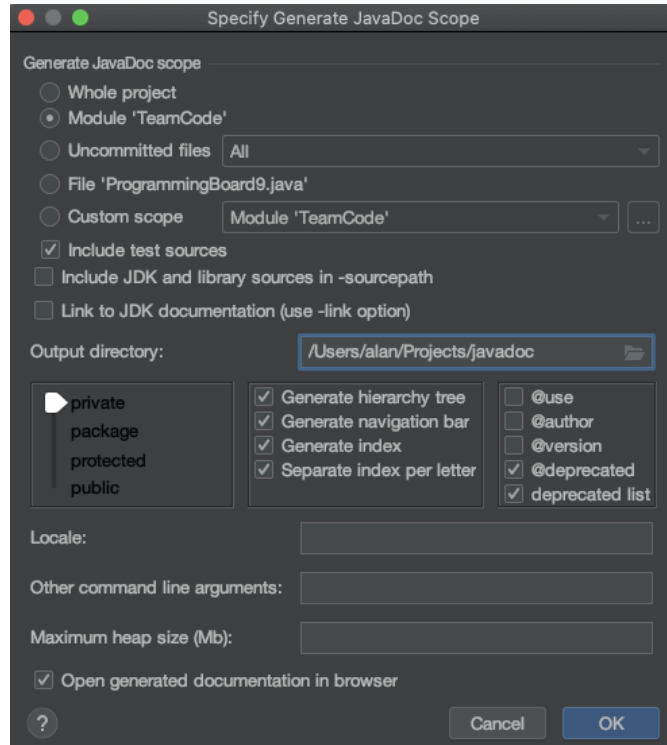
17. Javadoc

If you don't have anything more to say than the name, don't put in a comment. (For example - here is a BAD comment)



```
/* DO NOT DO THIS!!! - BAD EXAMPLE!! */  
  
/**  
 * This is the ProgrammingBoard class  
 */  
public class ProgrammingBoard{  
    ...  
}
```

After you have done this, in Android Studio go to Tools... Generate JavaDoc... and you'll see a dialog like this:



A few changes that I recommend:

1. Do it just on Module 'TeamCode'
2. Go ahead and tell it to generate the documentation on everything

3. Make sure you put it in its own directory because it creates a lot of files

17.1. Exercises

1. Add Javadoc comments to your ProgrammingBoard class
2. Add Javadoc comments to your TestMotor class (Because once you have TestWiring all working for your robot you'll want to show it to judges)

18. Finding things in FTC SDK

So far, I have told you about things that are in the FTC SDK. But there is lots more that we haven't looked at. So now let's teach you how to go looking for yourself.

As FIRST has been working on shrinking the size of the SDK, they no longer ship the Javadoc with each SDK. You can find it online at <https://javadoc.io/doc/org.firstinspires.ftc> where you can either use it online or download it to your computer. (You can also run Javadoc like you did on your code, but point it to `FtcRobotController` as well.)

You'll probably notice that this looks just like the Javadoc you created in [chapter 17](#). Sure enough, that is what they use to create the documentation for the FTC SDK as well

For example - Look through the All Classes until you get to `Telemetry` in the lower left portion of the screen. Click on it. Then the main part of the browser will have more information out about our old friend. Wait did you see that there is a `speak()` method??

18.1. Exercise

1. Write an `opMode` that uses the `telemetry.speak()` method
2. Look through the documentation and find something we haven't done before and try it

19. A few other topics

This is a place for a few other topics that I thought were important to mention but didn't really fit anywhere else

19.1. Math class

The java Math class has a lot of useful methods in it. They are all static so you don't need an object of type Math. Here is an example class to handle polar coordinates

Listing 19.1: Polar.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
4
5 public class Polar {
6     double angle;
7     double magnitude;
8
9     public Polar(double x, double y) {
10         angle = Math.atan2(y, x);
11         magnitude = Math.hypot(x, y);
12     }
13
14     public double getAngle(AngleUnit angleUnit) {
15         return angleUnit.fromRadians(angle);
16     }
17
18     public double getMagnitude() {
19         return magnitude;
20     }
21 }
```

You'll notice that we have a constructor that takes in x and y and converts it to polar coordinates.

The method called `getAngle` uses the `AngleUnit` to convert. As a bonus `AngleUnit` guarantees results to be normalized.

Some useful methods in this class: (all trig functions are in radians)

19. A few other topics

```
Math.abs(a) // take the absolute value
Math.acos(a) // take the arc cosine
Math.asin(a) // take the arc sin
Math.atan(a) // take the arc tan
Math.atan2(x, y) // This returns the angle theta from conversion of rectangular (x,y)↔
    ↪ to polar (r, theta)
Math.copySign(magnitude, sign) // return s the first argument with the sign (↔
    ↪ positive or negative) of the second
Math.cos(a) // take the cos
Math.hypot(x, y) // return the sqrt(x^2 + y^2)
Math.max(a, b) // returns the greater of a and b
Math.min(a, b) // returns the smaller of a and b
Math.random() // returns a double value with a positive sign greater than or equal ↪
    ↪ to 0.0 and less than 1.0
Math.signum(d) // returns -1.0 if d < 0, 0.0 if d == 0, 1.0 if d > 0
Math.sin(a) // take the sin
Math.sqrt(a) // take the square root
Math.tan(a) // take the tangent
Math.toDegrees(radians) // convert radians to degrees - I prefer using AngleUnit
Math.toRadians(degrees) // convert degrees to radians - I prefer using AngleUnit
```

19.2. final

`final` is a keyword that can be applied to a variable, a method or a class.

```
final int THRESHOLD = 5;
```

- `final` applied to a variable makes the variable a constant. Modifying it later will cause a compiler error. It either needs to be initialized immediately (or if it is part of a class, then in the constructor) By convention, we name these “variables” in ALL_CAPS to signify that they are constants. (unless they are initialized in the constructor because then they are different per instance.)

```
public class SuperClass{
    public String a(){
        return "a";
    }
    final public String b(){
        return "b";
    }
}
```

- `final` applied to a method means that even if a new class extends this class, this method cannot be overridden

```
final class A{
    // methods and members
}
```

- `final` applied to a class means that no class can extend this one.

19.3. Make telemetry prettier

There are some additional ways we can make our telemetry easier to see. We'll mention a few of them here.

Listing 19.2: MoreTelemetry.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class MoreTelemetry extends OpMode {
8     @Override
9     public void init() {
10         telemetry.addData("Run time", "%0.2f", getRuntime());
11     }
12
13     @Override
14     public void loop() {
15         telemetry.addData("Right Joystick",
16             "x:%+.2f y:%.2f", gamepad1.right_stick_x,
17             gamepad1.right_stick_y);
18
19         telemetry.addLine("Left joystick | ")
20             .addData("x", gamepad1.left_stick_x)
21             .addData("y", gamepad1.left_stick_y);
22     }
23 }
```

Let's talk through the three different telemetry examples here.

```
10 telemetry.addData("Run time", "%0.2f", getRuntime());
```

Here is our old friend `addData` but this time the string looks weird and there is another parameter. The string is called a format string. It can have text and

19. A few other topics

values. Every value is started with a % sign and has how to show the number. Below are the most common ones for FTC.

Conversion	Description
'b', 'B'	boolean - if the argument isn't a boolean, then it will show true unless null
'd'	decimal - for integers
'f'	decimal number - for floating point (float and double)
%	show a literal '%' character

For f, you can give it a precision after the . which is the maximum number of digits to show. If you want it to always show that number of digits (zero pad) then put 0.2 for example.

```
15 telemetry.addData("Right Joystick",
16     "x:%.2f y:%.2f", gamepad1.right_stick_x,
17     gamepad1.right_stick_y);
```

Here is an example where we are showing more than one value on the same line. The + is a flag saying to always show the sign (without it only shows the sign if it is negative.) The space on the second one says to have a space if positive instead of the positive sign. (The reason you might want this is so that the numbers don't jump as the negative sign comes in place.)

```
19 telemetry.addLine("Left joystick | ")
20     .addData("x", gamepad1.left_stick_x)
21     .addData("y", gamepad1.left_stick_y);
```

This has another way of showing multiple things per line by putting multiple addData after an addLine

19.4. Interfaces (implements)

Interfaces are similar to inheritance but are subtly different. Whereas a child class is a type of its super class, an interface is instead a “contract” that a class that implements it has to have certain method(s). A class can both be derived from a super class and implement multiple interfaces.

Let's give an example¹, first showing how we create an interface:

¹Yes, this is a contrived example, because you can always get the full name of a class in Java with `this.getClass().getName()` and just the last part of the class with `this.getClass().getSimpleName()`

Listing 19.3: SampleInterface.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 public interface SampleInterface {
4     public String getName();
5 }

```

and then here is a class that implements the interface

Listing 19.4: SampleClass.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 public class SampleClass implements SampleInterface {
4     @Override
5     public String getName() {
6         return "SampleClass";
7     }
8 }

```

If your class says it implements an interface, but it doesn't have all of the methods in it then the compiler will give an error.

19.4.1. When to use an interface instead of an abstract class?

The short version is that if you need code shared then it should be an abstract class because in an interface, each class that implements it will have to have the code in it again.

Another way to think about it is that a class can only inherit from one class, but it can implement multiple interfaces.

My personal opinion is that you are probably better off using simple inheritance unless you have something you need to do that requires multiple interfaces.

19.5. Exercises

1. Use the Polar class to make a new OpMode that reports the joysticks on the gamepad in polar coordinates - show the angle in degrees
2. Add the final keyword to various places to cause compiler errors so you can see what they look like
3. Use formatting so the telemetry for exercise 1 always shows the positive or negative sign and no decimals

20. Making Robots Drive

I have had people tell me that while this book has been helpful to them, that it would be nice to have a section on making robots drive. The beginning thing we need to think about is that it is difficult to be the driver in an FTC match. For 2 minutes everyone is staring at you and all of your mistakes. So as the programmers we need to think about how to make life as easy as possible for our drivers by coming up with control schemes that make it harder for them to make mistakes.

20.1. 2 motor drive

20.1.1. Two Motor Drive Mechanism

The easiest way to make a robot drive is to have one powered wheel on each side. (The most common way to do this is with unpowered wheels on the front of the robot and the powered ones in the back.)

First, we'll create the mechanism.

Listing 20.1: TwoMotorDrive.java

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.DcMotor;
4 import com.qualcomm.robotcore.hardware.DcMotorSimple;
5 import com.qualcomm.robotcore.hardware.HardwareMap;
6
7 public class TwoMotorDrive {
8     private DcMotor leftMotor;
9     private DcMotor rightMotor;
10
11     public void init(HardwareMap hardwareMap) {
12         leftMotor = hardwareMap.get(DcMotor.class, "left_motor");
13         rightMotor = hardwareMap.get(DcMotor.class, "right_motor");
14
15         leftMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
16         rightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
17         leftMotor.setDirection(DcMotorSimple.Direction.REVERSE);
18     }
```

20. Making Robots Drive

```
19
20 public void setPowers(double leftPower, double rightPower) {
21     double largest = 1.0;
22     largest = Math.max(largest, Math.abs(leftPower));
23     largest = Math.max(largest, Math.abs(rightPower));
24
25     leftMotor.setPower(leftPower / largest);
26     rightMotor.setPower(rightPower / largest);
27 }
28 }
```

Now, we'll go through this one bit at a time

```
8 private DcMotor leftMotor;
9 private DcMotor rightMotor;
```

Here we define our two motors. Notice that we gave them names that make sense. Also, they are `private` which means that only methods inside the class can use them.

```
11 public void init(HardwareMap hardwareMap) {
12     leftMotor = hardwareMap.get(DcMotor.class, "left_motor");
13     rightMotor = hardwareMap.get(DcMotor.class, "right_motor");
14
15     leftMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
16     rightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
17     leftMotor.setDirection(DcMotorSimple.Direction.REVERSE);
18 }
```

This gets our motors from the `hardwareMap`, sets them as using encoders, and then also sets the left one as being `REVERSE` which allows us to treat it with respect to the wheel instead of in respect to the motor.



If your robot drives backwards with this code, change which motor you have set to be `REVERSE`

```
20 public void setPowers(double leftPower, double rightPower) {
21     double largest = 1.0;
22     largest = Math.max(largest, Math.abs(leftPower));
23     largest = Math.max(largest, Math.abs(rightPower));
24
25     leftMotor.setPower(leftPower / largest);
26     rightMotor.setPower(rightPower / largest);
27 }
```

You might have expected this to just set the power of each motor. The problem with doing this is that turns are determined by the relative speeds of the motor. If you send 1.2 as the speed to the motor, the motor will treat it as 1.0.

So this code makes sure that the values being sent to the motors are within the range -1..1 (inclusive)

20.1.2. OpMode

Some teams use what is called “TankDrive” and map each joystick to each motor. While this is the easiest way to program, we can do much better than that. We’ll make what is called “ArcadeDrive”. In this way, the amount the stick is forward or back determines how fast it goes forward or reverse and the amount to the side determines how much it turns at the same time.

Listing 20.2: ArcadeDrive.java

```

1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.TwoMotorDrive;
7
8 @TeleOp()
9 public class ArcadeDrive extends OpMode {
10     TwoMotorDrive drive = new TwoMotorDrive();
11
12     @Override
13     public void init() {
14         drive.init(hardwareMap);
15     }
16
17     @Override
18     public void loop() {
19         double forward = -gamepad1.left_stick_y;
20         double right = gamepad1.left_stick_x;
21
22         drive.setPowers(forward + right, forward - right);
23     }
24 }
```

Now we’ll go through some of the interesting pieces.

```

10     TwoMotorDrive drive = new TwoMotorDrive();
```

We have our mechanism as a member. This allows us to have the details of the drive in the TwoMotorDrive class and makes each class simpler.

```

17     @Override
18     public void loop() {
19         double forward = -gamepad1.left_stick_y;
```

20. Making Robots Drive

```
20 double right = gamepad1.left_stick_x;
```

Here we call `drive.setPowers` with the amount of forward plus the amount of right to the left wheel and the amount of forward minus the amount of right to the right wheel. (You can convince yourself this is correct because to turn right the left wheel needs to travel further.)

You can try this in the simulator or on an actual robot. Many people will feel that this turns too fast. But the joystick is just providing us a number. We can make it different easily.

Listing 20.3: BetterArcadeDrive.java

```
1 package org.firstinspires.ftc.teamcode.opmodes;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.teamcode.mechanisms.TwoMotorDrive;
7
8 @TeleOp()
9 public class BetterArcadeDrive extends OpMode {
10     TwoMotorDrive drive = new TwoMotorDrive();
11
12     @Override
13     public void init() {
14         drive.init(hardwareMap);
15     }
16
17     @Override
18     public void loop() {
19         double forward = -gamepad1.left_stick_y;
20         double right = gamepad1.left_stick_x / 2;
21
22         drive.setPowers(forward + right, forward - right);
23     }
24 }
```

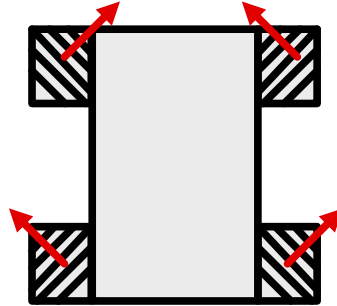
You'll notice that we only changed line 19 and now it turns slower. We could do all sorts of things such as allow it to turn at full speed when a button was pressed. That will be one of the exercises.

20.2. 4 motor mecanum drive

Mecanum drive is a very cool (albeit expensive wheels) that allows you to drive in every direction. This is what the cool kids call a “holonomic drive” Part of what makes this drive so cool is that the complication is in the wheels (bought)

and the wheels are easy to mount. I recommend the GoBilda chassis (make sure you have gotten your FTC discount)

But nothing in life is free. For this incredible flexibility, you give up some acceleration and traction.



The wheels have bearings at 45 degrees. You can do the vector math to convince yourself, but courtesy of FTC16072 (Quantum Quacks) it is:

$$\begin{aligned} \text{leftFrontPower} &= \text{forward} + \text{right} + \text{rotate} \\ \text{rightFrontPower} &= \text{forward} - \text{right} - \text{rotate} \\ \text{leftBackPower} &= \text{forward} - \text{right} + \text{rotate} \\ \text{rightBackPower} &= \text{forward} + \text{right} - \text{rotate} \end{aligned}$$

20.2.1. Mecanum Mechanism

First, we'll create the mechanism.

Listing 20.4: MecanumDrive.java

```

1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.robotcore.hardware.DcMotor;
4 import com.qualcomm.robotcore.hardware.HardwareMap;
5
6 public class MecanumDrive {
7     private DcMotor frontLeftMotor;
8     private DcMotor frontRightMotor;
9     private DcMotor backLeftMotor;
10    private DcMotor backRightMotor;
11
12    public void init(HardwareMap hardwareMap) {
13        frontLeftMotor = hardwareMap.dcMotor.get("front_left_motor");
14        frontRightMotor = hardwareMap.dcMotor.get("front_right_motor");
15        backLeftMotor = hardwareMap.dcMotor.get("back_left_motor");

```

20. Making Robots Drive

```
16     backRightMotor = hardwareMap.dcMotor.get("back_right_motor");
17
18     backLeftMotor.setDirection(DcMotor.Direction.REVERSE);
19     frontLeftMotor.setDirection(DcMotor.Direction.REVERSE);
20
21     frontLeftMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
22     frontRightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
23     backLeftMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
24     backRightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
25 }
26
27 private void setPowers(double frontLeftPower, double frontRightPower, double ↵
    ↵ backLeftPower, double backRightPower) {
28     double maxSpeed = 1.0;
29     maxSpeed = Math.max(maxSpeed, Math.abs(frontLeftPower));
30     maxSpeed = Math.max(maxSpeed, Math.abs(frontRightPower));
31     maxSpeed = Math.max(maxSpeed, Math.abs(backLeftPower));
32     maxSpeed = Math.max(maxSpeed, Math.abs(backRightPower));
33
34     frontLeftPower /= maxSpeed;
35     frontRightPower /= maxSpeed;
36     backLeftPower /= maxSpeed;
37     backRightPower /= maxSpeed;
38
39     frontLeftMotor.setPower(frontLeftPower);
40     frontRightMotor.setPower(frontRightPower);
41     backLeftMotor.setPower(backLeftPower);
42     backRightMotor.setPower(backRightPower);
43 }
44
45 // Thanks to FTC16072 for sharing this code!!
46 public void drive(double forward, double right, double rotate) {
47     double frontLeftPower = forward + right + rotate;
48     double frontRightPower = forward - right - rotate;
49     double backLeftPower = forward - right + rotate;
50     double backRightPower = forward + right - rotate;
51
52     setPowers(frontLeftPower, frontRightPower, backLeftPower, backRightPower);
53 }
54 }
```

Now, we'll go through this one bit at a time.

```
8     private DcMotor frontRightMotor;
9     private DcMotor backLeftMotor;
10    private DcMotor backRightMotor;
```

Here we define our four motors. Notice that we gave them names that make sense. Also, they are private which means that only methods inside the class

can use them.

```

13     frontLeftMotor = hardwareMap.dcMotor.get("front_left_motor");
14     frontRightMotor = hardwareMap.dcMotor.get("front_right_motor");
15     backLeftMotor = hardwareMap.dcMotor.get("back_left_motor");
16     backRightMotor = hardwareMap.dcMotor.get("back_right_motor");
17
18     backLeftMotor.setDirection(DcMotor.Direction.REVERSE);
19     frontLeftMotor.setDirection(DcMotor.Direction.REVERSE);
20
21     frontLeftMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
22     frontRightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
23     backLeftMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
24     backRightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
25 }
```

This gets our motors from the hardwareMap, sets them as using encoders, and then also sets the left ones as being REVERSE which allows us to treat it with respect to the wheel instead of in respect to the motor.



If your robot drives backwards with this code, change which motors you have set to be REVERSE

```

28     double maxSpeed = 1.0;
29     maxSpeed = Math.max(maxSpeed, Math.abs(frontLeftPower));
30     maxSpeed = Math.max(maxSpeed, Math.abs(frontRightPower));
31     maxSpeed = Math.max(maxSpeed, Math.abs(backLeftPower));
32     maxSpeed = Math.max(maxSpeed, Math.abs(backRightPower));
33
34     frontLeftPower /= maxSpeed;
35     frontRightPower /= maxSpeed;
36     backLeftPower /= maxSpeed;
37     backRightPower /= maxSpeed;
38
39     frontLeftMotor.setPower(frontLeftPower);
40     frontRightMotor.setPower(frontRightPower);
41     backLeftMotor.setPower(backLeftPower);
42     backRightMotor.setPower(backRightPower);
43 }
```

You might have expected this to just set the power of each motor. The problem with doing this is that turns are determined by the relative speeds of the motor. If you send 1.2 as the speed to the motor, the motor will treat it as 1.0. So this code makes sure that the values being sent to the motors are within the range -1..1 (inclusive)

```

46     public void drive(double forward, double right, double rotate) {
47         double frontLeftPower = forward + right + rotate;
```

20. Making Robots Drive

```
48     double frontRightPower = forward - right - rotate;
49     double backLeftPower = forward - right + rotate;
50     double backRightPower = forward + right - rotate;
51
52     setPowers(frontLeftPower, frontRightPower, backLeftPower, backRightPower);
53 }
54 }
```

You'll notice that this looks very similar to the formulas given up above, but this time it is in code.

20.2.2. Robot oriented driving

While you can use lots of schemes for driving and you should come up with what makes the most sense for your team, a common scheme is to use the left joystick for moving the robot and the right joystick for rotating the robot.

Listing 20.5: SimpleMecanumDriveOpMode.java

```
1  package org.firstinspires.ftc.teamcode.opmodes;
2
3  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6  import org.firstinspires.ftc.teamcode.mechanisms.MecanumDrive;
7
8  @TeleOp()
9  public class SimpleMecanumDriveOpMode extends OpMode {
10     MecanumDrive drive = new MecanumDrive();
11
12     @Override
13     public void init() {
14         drive.init(hardwareMap);
15     }
16
17     @Override
18     public void loop() {
19         double forward = -gamepad1.left_stick_y;
20         double right = gamepad1.left_stick_x;
21         double rotate = gamepad1.right_stick_x;
22
23         drive.drive(forward, right, rotate);
24     }
25 }
```

Now we'll go through some of the interesting pieces.

```
10     MecanumDrive drive = new MecanumDrive();
```

We have our mechanism as a member. This allows us to have the details of the drive in the MecanumDrive class and makes each class simpler.

```

18     public void loop() {
19         double forward = -gamepad1.left_stick_y;
20         double right = gamepad1.left_stick_x;
21         double rotate = gamepad1.right_stick_x;
22
23         drive.drive(forward, right, rotate);
24     }

```

This has us read the joysticks and use it to drive the robot. Simple enough!!

20.2.3. Field oriented driving

This is all fine and good, but can we take advantage of the fact that the robot can drive in any direction to make it drive field relative? Of course...

Listing 20.6: FieldRelativeMecanumDriveOpMode.java

```

1  package org.firstinspires.ftc.teamcode.opmodes;
2
3  import com.qualcomm.hardware.rev.RevHubOrientationOnRobot;
4  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
6  import com.qualcomm.robotcore.hardware.IMU;
7
8  import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
9  import org.firstinspires.ftc.teamcode.mechanisms.MecanumDrive;
10
11  @TeleOp()
12  public class FieldRelativeMecanumDriveOpMode extends OpMode {
13      MecanumDrive drive = new MecanumDrive();
14      IMU imu;
15
16      @Override
17      public void init() {
18          drive.init(hardwareMap);
19
20          imu = hardwareMap.get(IMU.class, "imu");
21          RevHubOrientationOnRobot revHubOrientationOnRobot =
22              new RevHubOrientationOnRobot(RevHubOrientationOnRobot.↵
23                  ↵ LogoFacingDirection.UP,
24                  RevHubOrientationOnRobot.UsbFacingDirection.FORWARD);
25
26          imu.initialize(new IMU.Parameters(revHubOrientationOnRobot));
27      }
28
29      private void driveFieldRelative(double forward, double right, double rotate) {

```

20. Making Robots Drive

```

29         double robotAngle = imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.RADIANS)↵
30         ↵ );
31         // convert to polar
32         double theta = Math.atan2(forward, right);
33         double r = Math.hypot(forward, right);
34         // rotate angle
35         theta = AngleUnit.normalizeRadians(theta - robotAngle);
36
37         // convert back to cartesian
38         double newForward = r * Math.sin(theta);
39         double newRight = r * Math.cos(theta);
40
41         drive.drive(newForward, newRight, rotate);
42     }
43
44     @Override
45     public void loop() {
46         double forward = -gamepad1.left_stick_y;
47         double right = gamepad1.left_stick_x;
48         double rotate = gamepad1.right_stick_x;
49
50         driveFieldRelative(forward, right, rotate);
51     }
52 }

```

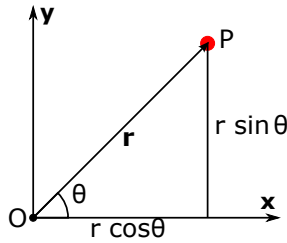
Instead of going through everything, we'll only talk about the new bits. For details about using the IMU, see [chapter 11](#).

A little math refresher (or new math if you haven't seen it before.) Typically we work with coordinates in the x and y (called Cartesian), but you can also work with r and Θ . Where Θ is the angle and r is the distance.



Hopefully this helps you see how you can cover the same space with a different coordinate system (called Polar)

In this coordinate scheme rotating is easy, you just change the angle. So how do we convert from Cartesian to Polar and back again? With our friends from trigonometry...



```

30 // convert to polar
31 double theta = Math.atan2(forward, right);
32 double r = Math.hypot(forward, right);

```

So this code changes from Cartesian to Polar by discovering what the angle is (using arctan, `Math.atan2` is the method in our math library that does that for us) and the hypotenuse. (We could have done $\sqrt{a^2 + b^2}$ but it is much simpler and cleaner to just use the `Math.hypot` method)

```

33 // rotate angle
34 theta = AngleUnit.normalizeRadians(theta - robotAngle);

```

Then we rotate it by subtracting our angle of the gyro from the angle in our polar. (`normalizeRadians` is code that makes sure our angle is between $-\pi$ and π . Yes, that is not really 0 to 360 like I showed before. If you really want to use degrees, you can.)

```

36 // convert back to cartesian
37 double newForward = r * Math.sin(theta);
38 double newRight = r * Math.cos(theta);

```

This converts it back to cartesian, exactly like the diagram above.

```

40 drive.drive(newForward, newRight, rotate);

```

and then we simply call our drive method with the new values.

20.3. Exercises

1. Make the arcade drive (2 wheel drive) only be able to go half as fast unless the A button is pressed in and then it can go full speed.
2. Change the mecanum drive to only rotate at half the speed
3. CHALLENGE - Make the right joystick snap the robot to the angle of where the joystick is pressed - sample solution NOT given. HINT: you'll need to get the theta of the joystick and figure out which way to make the robot rotate to get closer.

21. Some hardware to help with Odometry

Odometry is the use of data from motion sensors to estimate change in position over time.¹

There is some new hardware available for FTC students starting with FTC SDK 9.2. (Released in Summer of 2024)

21.1. OctoQuad

21.1.1. What is it?

An OctoQuad is a device that allows you to plug in up to 8 encoders and read the value of them over I2C. This allows us to read from encoders without giving up the precious encoder ports that go along with the motors. This allows us to use encoders for things where they may not have made sense before.

21.1.2. Using it simply

The easiest thing we can do is to get a single position from the OctoQuad.

Listing 21.1: SimpleOctoQuadOpMode.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.hardware.digitalchickenlabs.OctoQuad;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
6
7 @TeleOp
8 public class SimpleOctoQuadOpMode extends OpMode {
9     OctoQuad octoQuad;
10     final int ENCODER_POSITION = 0;
11
12     @Override
13     public void init() {
14         octoQuad = hardwareMap.get(OctoQuad.class, "octoquad");
15         octoQuad.setSingleEncoderDirection(
```

¹From Wikipedia: <https://en.wikipedia.org/wiki/Odometry>

21. Some hardware to help with Odometry

```
16         ENCODER_POSITION, OctoQuad.EncoderDirection.FORWARD);
17     octoQuad.saveParametersToFlash();
18     octoQuad.resetAllPositions();
19 }
20
21 @Override
22 public void loop() {
23     telemetry.addData("Octoquad position", octoQuad.readSinglePosition(←
        ↪ ENCODER_POSITION));
24 }
25 }
```

Let's go through this a bit at a time and explain all the pieces.

```
9     OctoQuad octoQuad;
```

This should look familiar. Just like we have a servo of class `Servo` and a motor of class `DcMotor`, we have a variable named `octoQuad` of class `OctoQuad`

```
10     final int ENCODER_POSITION = 0;
```

Good practice is to avoid magic numbers, so here we define which port our encoder is plugged into in the `OctoQuad`. (If it is plugged into a different one, change it here)

Like all opmodes, we have an `init` and a `loop`.

Let's look at the `init` first.

```
14     octoQuad = hardwareMap.get(OctoQuad.class, "octoquad");
```

This is just like every other piece of hardware we have gotten access to except for it is of type `OctoQuad`.

```
15     octoQuad.setSingleEncoderDirection(
16         ENCODER_POSITION, OctoQuad.EncoderDirection.FORWARD);
```

We should set the encoder direction. If the numbers are counting down when you expect them to count up, set this as `REVERSE` instead. We do this every time just in case another opmode has changed it to something else before this one runs.

```
17     octoQuad.saveParametersToFlash();
```

We save it to flash so that if there is a power blip the `OctoQuad` will come back with the right settings. (Don't worry about writing to flash unnecessarily. The `OctoQuad` is smart enough that if you give it the same parameters it had before, it won't actually rewrite the flash.)

```
18     octoQuad.resetAllPositions();
```

This resets all counters to zero, so we know it is only counting the ones from here forward.

```

22     public void loop() {
23         telemetry.addData("Octoquad position", octoQuad.readSinglePosition(↵
            ↵ ENCODER_POSITION));
24     }

```

Our loop doesn't do anything except display on telemetry the encoder position.

21.1.3. Using it to get multiple encoders

Probably the most common example for why someone would want to use an OctoQuad has to do with using tracking wheels. (Often called dead-wheel odometry in FTC). In this case, we would have three tracking wheels hooked up to encoders. I recommend the GoBilda Odometry Pod - <https://www.gobilda.com/odometry-pod-43mm-width-48mm-wheel/>

Here is an example that shows all three.

Listing 21.2: MultipleOctoQuadOpMode.java

```

1  package org.firstinspires.ftc.teamcode;
2
3  import com.qualcomm.hardware.digitalchickenlabs.OctoQuad;
4  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
6
7  @TeleOp
8  public class MultipleOctoQuadOpmode extends OpMode {
9      final int ENCODER_POSITION_LEFT = 0;
10     final int ENCODER_POSITION_RIGHT = 1;
11     final int ENCODER_POSITION_CROSS = 2;
12     OctoQuad octoQuad;
13
14     @Override
15     public void init() {
16         octoQuad = hardwareMap.get(OctoQuad.class, "octoquad");
17         octoQuad.setSingleEncoderDirection(
18             ENCODER_POSITION_LEFT, OctoQuad.EncoderDirection.REVERSE);
19         octoQuad.setSingleEncoderDirection(
20             ENCODER_POSITION_RIGHT, OctoQuad.EncoderDirection.FORWARD);
21         octoQuad.setSingleEncoderDirection(
22             ENCODER_POSITION_CROSS, OctoQuad.EncoderDirection.FORWARD);
23
24         octoQuad.saveParametersToFlash();
25         octoQuad.resetAllPositions();

```

21. Some hardware to help with Odometry

```
26     }
27
28     @Override
29     public void loop() {
30         int[] positions = octoQuad.readAllEncoderData().positions;
31         telemetry.addData("Left Position", positions[ENCODER_POSITION_LEFT]);
32         telemetry.addData("Right Position", positions[ENCODER_POSITION_RIGHT]);
33         telemetry.addData("Cross Position", positions[ENCODER_POSITION_CROSS]);
34     }
35 }
```

The only thing here that probably isn't obvious from before is that we read all the positions. While we could use `octoQuad.readPositionRange` for a slight optimization if they are all in order, it is so slight that it probably isn't worth the effort.

21.1.4. Using the cached attribute for clean programming

We might have multiple mechanisms that each use the `OctoQuad` for different things. For example, let's say that we have three Odometry pods. Here is a recommended way you might do that.

You might start with an `OdometryPod` class that describes an `OdometryPod`

Listing 21.3: `OdometryPod.java`

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.hardware.digitalchickenlabs.OctoQuad;
4
5 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
6
7 public class OdometryPod {
8     private final OctoQuad octoQuad;
9     private final int channel;
10    private final double TICKS_PER_REV = 2000;
11    private final double WHEEL_DIAMETER_CM = 4.8; // 48 MM wheel is 4.8 CM
12    private final double WHEEL_CIRCUMFERENCE_CM = Math.PI * WHEEL_DIAMETER_CM;
13    private final double CM_PER_ENCODER_TICK = WHEEL_CIRCUMFERENCE_CM / TICKS_PER_REV↵
    ↵ ;
14
15    OdometryPod(OctoQuad octoQuad, int channel, OctoQuad.EncoderDirection direction) ↵
    ↵ {
16        this.channel = channel;
17        this.octoQuad = octoQuad;
18        octoQuad.setSingleEncoderDirection(channel, direction);
19    }
20 }
```

```

21     private int readPosition() {
22         return octoQuad.readSinglePosition_Caching(channel);
23     }
24
25     public double getDistance(DistanceUnit distanceUnit) {
26         return distanceUnit.fromCm(readPosition() * CM_PER_ENCODER_TICK);
27     }
28 }

```

One of the neat things about classes is that we can have multiple instances. So we only need to describe an Odometry Pod once and then we can have three instances instead of having three different classes with a lot of repeated code.

```

10     private final double TICKS_PER_REV = 2000;
11     private final double WHEEL_DIAMETER_CM = 4.8; // 48 MM wheel is 4.8 CM
12     private final double WHEEL_CIRCUMFERENCE_CM = Math.PI * WHEEL_DIAMETER_CM;
13     private final double CM_PER_ENCODER_TICK = WHEEL_CIRCUMFERENCE_CM / TICKS_PER_REV ↵
        ↵ ;

```

This is the math that helps us give our users useful information (like how far the wheel has gone) without them needing to know information about the pod.

The ticks per revolution comes from the encoder datasheet.

You'll recall that $\text{circumference} = c = \pi d$ - a lot of times people learn it as $c = 2\pi r$ but since the diameter is twice the radius, we can just skip a step and measure the wheel's diameter.

Since we now know the wheel circumference and the ticks per revolution, we can think of that as the wheel circumference is the same as the distance the wheel goes in one revolution so to get the centimeters per tick, we just need to divide the wheel circumference by the number of ticks per revolution.

```

15     OdometryPod(OctoQuad octoQuad, int channel, OctoQuad.EncoderDirection direction) ↵
        ↵ {
16         this.channel = channel;
17         this.octoQuad = octoQuad;
18         octoQuad.setSingleEncoderDirection(channel, direction);
19     }

```

Our constructor deals with the things that we need to know and those that are different per class (so they can't be class constants). In this case, we need a reference to the octoQuad as well as what channel our encoder is on and we need to set the direction.

```

25     public double getDistance(DistanceUnit distanceUnit) {
26         return distanceUnit.fromCm(readPosition() * CM_PER_ENCODER_TICK);
27     }

```

21. Some hardware to help with Odometry

One of my favorite classes to use is `DistanceUnit` because then people can use whatever units they are familiar with and all of the conversion gets done for you automatically.

And then you might have a robot class that had a mecanum drive and three odometry pods

Listing 21.4: `Robot.java`

```
1 package org.firstinspires.ftc.teamcode.mechanisms;
2
3 import com.qualcomm.hardware.digitalchickenlabs.OctoQuad;
4 import com.qualcomm.robotcore.hardware.HardwareMap;
5
6
7 public class Robot {
8     final int CHANNEL_LEFT = 0, CHANNEL_RIGHT = 1, CHANNEL_CROSS = 2;
9     public OdometryPod leftPod, rightPod, crossPod;
10    public MecanumDrive mecanumDrive;
11    OctoQuad octoQuad;
12
13    public void init(HardwareMap hardwareMap) {
14        mecanumDrive.init(hardwareMap);
15        octoQuad = hardwareMap.get(OctoQuad.class, "octoquad");
16
17        leftPod = new OdometryPod(octoQuad, CHANNEL_LEFT, OctoQuad.EncoderDirection.↔
18            ↪ REVERSE);
19        rightPod = new OdometryPod(octoQuad, CHANNEL_RIGHT, OctoQuad.EncoderDirection↔
20            ↪ .FORWARD);
21        crossPod = new OdometryPod(octoQuad, CHANNEL_CROSS, OctoQuad.EncoderDirection↔
22            ↪ .FORWARD);
23
24        octoQuad.saveParametersToFlash();
25        octoQuad.resetAllPositions();
26        octoQuad.setCachingMode(OctoQuad.CachingMode.AUTO);
27    }
28
29    public void resetPods() {
30        octoQuad.resetAllPositions();
31    }
32 }
```

You can see from this that we create the odometry pods and then save their parameters. We reset all positions and then set the caching mode to AUTO. There are 3 caching modes:

Caching Mode	What it means
NONE	No caching, every read goes to the device
MANUAL	Everything is read from cache, to read from the device you have to call <code>refreshCache</code>
AUTO	When you read a particular position a second time since the cache has been refreshed, it refreshes the cache. But reading different positions independently doesn't cause a performance problem because they are reading from the cache.

And then you might have an opmode that let you drive and showed the distance each of the pods had gone

Listing 21.5: RobotOctoQuadOpmode.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
7 import org.firstinspires.ftc.teamcode.mechanisms.Robot;
8
9 @TeleOp()
10 public class RobotOctoQuadOpmode extends OpMode {
11     Robot robot;
12
13     @Override
14     public void init() {
15         robot.init(hardwareMap);
16     }
17
18     @Override
19     public void loop() {
20         if (gamepad1.x) {
21             robot.resetPods();
22         }
23         robot.mecanumDrive.drive(-gamepad1.left_stick_y, gamepad1.left_stick_x, ↵
                ↵ gamepad1.right_stick_x);
24         telemetry.addData("Left Pod (in)", robot.leftPod.getDistance(DistanceUnit.↵
                ↵ INCH));
25         telemetry.addData("Right Pod (in)", robot.leftPod.getDistance(DistanceUnit.↵
                ↵ INCH));
26         telemetry.addData("Cross Pod (in)", robot.leftPod.getDistance(DistanceUnit.↵
                ↵ INCH));
27     }
28 }

```

21. Some hardware to help with Odometry

Here we allow the pods to be reset using the x button on the controller, and it shows the distance each of the pods have gone on the telemetry.

21.1.5. Other features

21.1.5.1. Velocity

Some time you don't care about the current position, but you want to know the velocity (ie, how fast is it rotating). You can access that with either `readSingleVelocity` or `readSingleVelocity_Caching` or of course with `readAllVelocities`. For each channel, you can set the sampling time (1 - 255 ms) and the velocity is the change from the last sample to the one before that. To set the sampling time, use `setSingleVelocitySampleInterval`

For more details, see the official FTC SDK samples.

21.1.5.2. Absolute encoders

If you are using an encoder that has absolute encoders as well (giving the position, not just the count) you can use the `OctoQuad` for reading that. Often you'll want to setup the first four connectors for relative encoders and the second four as absolute encoders. For more details, see the official FTC SDK samples.

21.2. Sparkfun Optical Tracking Odometry Sensor

21.2.1. What is it?

This sensor takes pictures and compares each frame to the one before to estimate which direction you have moved in. It also has an IMU onboard and uses sensor fusion (a fancy way for when you combine an accurate sensor that only updates occasionally with a less accurate one that updates rapidly) to be able to give you back an estimated position.

You can buy this from <https://www.sparkfun.com/products/24904> - Don't forget to buy a QWIIC to STEMMA cable so you can plug it into your Control or Expansion Hub. (<https://www.sparkfun.com/products/25596>).

21.2.2. Using it

The easiest thing we can do with it is to get it to tell us where it thinks the robot is. (we can push the robot around with our hands and see it update)

Listing 21.6: UseSparkfunOTOS.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.hardware.sparkfun.SparkFunOTOS;
4 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
5 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
6
7 import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
8 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
9
10 @TeleOp
11 public class UseSparkfunOTOS extends OpMode {
12     SparkFunOTOS sparkfunOTOS;
13
14     @Override
15     public void init() {
16         sparkfunOTOS = hardwareMap.get(SparkFunOTOS.class, "otos");
17         configureOTOS();
18     }
19
20     private void configureOTOS() {
21         sparkfunOTOS.setLinearUnit(DistanceUnit.INCH);
22         sparkfunOTOS.setAngularUnit(AngleUnit.DEGREES);
23         sparkfunOTOS.setOffset(new SparkFunOTOS.Pose2D(0, 0, 0));
24         sparkfunOTOS.setLinearScalar(1.0);
25         sparkfunOTOS.setAngularScalar(1.0);
26         sparkfunOTOS.resetTracking();
27         sparkfunOTOS.setPosition(new SparkFunOTOS.Pose2D(0,0,0));
28         sparkfunOTOS.calibrateImu(255, false);
29     }
30
31     public void init_loop(){
32         telemetry.addData("Samples left to calibrate", sparkfunOTOS.↵
33             ↵ getImuCalibrationProgress());
34     }
35
36     @Override
37     public void loop() {
38         SparkFunOTOS.Pose2D pos = sparkfunOTOS.getPosition();
39         telemetry.addData("X (inch)", pos.x);
40         telemetry.addData("Y (inch)", pos.y);
41         telemetry.addData("Heading (degrees)", pos.h);
42     }

```

21. Some hardware to help with Odometry

42

```
}
```

Let's go through this a bit at a time and explain all the pieces.

12

```
SparkFunOTOS sparkfunOTOS;
```

This should look familiar. Just like we have a servo of class `Servo` and a motor of class `DcMotor`, we have a variable named `sparkfunOTOS` of class `SparkFunOTOS`

16

```
sparkfunOTOS = hardwareMap.get(SparkFunOTOS.class, "otos");
```

This should look familiar, it is how we get all of our items from the hardware map

17

```
configureOTOS();
```

While we could put all of our configuration in the `init()` method, it is better practice to split it out into its own method. This sensor doesn't keep its configuration across power resets, so we should configure it at the beginning of each `opMode`.

20

```
private void configureOTOS() {
```

We make it private because nobody outside of us should be asking for it to be configured.

21

```
sparkfunOTOS.setLinearUnit(DistanceUnit.INCH);
```

We can set which units for x and y we want the sensor to report in. We talked about the `DistanceUnit` class in [section 10.3](#).

22

```
sparkfunOTOS.setAngularUnit(AngleUnit.DEGREES);
```

We can set which unit for degrees we want the sensor to report for the heading. You can choose `DEGREES` or `RADIANS`.

23

```
sparkfunOTOS.setOffset(new SparkFunOTOS.Pose2D(0, 0, 0));
```

This describes the offset of the sensor from the center of your robot. X is negative to the left, positive to the right. Y is negative behind the center and forward of the center. The heading is negative for clockwise with relation to the robot or positive for counterclockwise.

24

```
sparkfunOTOS.setLinearScalar(1.0);
```

25

```
sparkfunOTOS.setAngularScalar(1.0);
```

These are to help calibrate the sensors if the reportings you are getting are off. They can be in the range from 0.872 to 1.127 in increments of 0.001. The

suggestion is to calibrate the angular scalar first by turning the robot around 10 times and then get the error and set the AngularScalar to the inverse of the error. So if you get the sensor saying -15 degrees after turning the robot perfectly 10 times, then you would set the scalar to $\frac{3,600}{3,585} = 1.004$. (Once around a circle is 360 degrees, so 10 times around is 3,600)

To calibrate the linear scalar, move the robot a known distance and measure the error. Do this multiple times at multiple speeds to get an average, then set the linear scalar to the inverse of the error. For example, if you move the robot 100 inches and the sensor reports 103 inches, set the linear scalar to $\frac{100}{103} = 0.971$.

You can tell that for both of these the formula is $scalar = \frac{expected}{reported}$. Since it is going to multiply the reported number by this scalar that will get you the expected result.

```
26 sparkfunOTOS.resetTracking();
```

This resets the tracking part of the system, getting rid of any past errors.

```
27 sparkfunOTOS.setPosition(new SparkFunOTOS.Pose2D(0,0,0));
```

This probably looks similar to setting the offset, but the offset is the relation of the sensor to the robot. This is the position of the robot on the field. That way you can set it based off of where you know the robot is. (either a starting position or because you have seen an AprilTag like we talked about in [section 16.1](#))

```
28 sparkfunOTOS.calibrateImu(255, false);
```

The first parameter here (255) is the number of samples to use in calibrating the IMU. It can be in the range of 1 to 255. (each one takes 2.4ms, so 255 is about 612ms) The second parameter here is whether to wait until done or not. It can be true or false. We set it to false so we can go about our business, but you do want to wait for it finish before moving the robot.

```
31 public void init_loop(){
32     telemetry.addData("Samples left to calibrate", sparkfunOTOS.↵
        ↵ getImuCalibrationProgress());
33 }
```

This will put on our telemetry how many samples are left to calibrate the IMU so we can make sure we don't move the robot until it tells us there are zero left.

```
36 public void loop() {
37     SparkFunOTOS.Pose2D pos = sparkfunOTOS.getPosition();
```

21. Some hardware to help with Odometry

```
38     telemetry.addData("X (inch)", pos.x);
39     telemetry.addData("Y (inch)", pos.y);
40     telemetry.addData("Heading (degrees)", pos.h);
41 }
```

This gets the position and puts it on telemetry for us.

21.3. GoBilda Pinpoint

GoBilda Pinpoint (<https://www.gobilda.com/pinpoint-v2-odometry-computer-imu-sensor-fusion-for-2-wheel-odometry/>) is beautiful in its simplicity. You simply attach 2 odometry pods to it and then you can read your robots position.

Listing 21.7: UseGoBildaPinpoint.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4
5 import com.qualcomm.hardware.gobilda.GoBildaPinpointDriver;
6 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
7
8 import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;
9 import org.firstinspires.ftc.robotcore.external.navigation.DistanceUnit;
10 import org.firstinspires.ftc.robotcore.external.navigation.Pose2D;
11
12 @TeleOp
13 public class UseGoBildaPinpoint extends OpMode {
14     GoBildaPinpointDriver pinpoint;
15
16     @Override
17     public void init() {
18         pinpoint = hardwareMap.get(GoBildaPinpointDriver.class, "pinpoint");
19         configurePinpoint();
20         pinpoint.setPosition(new Pose2D(DistanceUnit.INCH, 0, 0, AngleUnit.DEGREES, ↵
                ↵ 0));
21     }
22
23     @Override
24     public void loop() {
25         telemetry.addLine("Push your robot around to see it track");
26         telemetry.addLine("Press A to reset the position");
27         if(gamepad1.a){
28             pinpoint.setPosition(new Pose2D(DistanceUnit.INCH, 0, 0, AngleUnit.↵
                ↵ DEGREES, 0));
29         }
30     }
31 }
```

```

30     pinpoint.update();
31     Pose2D pose2D = pinpoint.getPosition();
32
33     telemetry.addData("X coordinate (IN)", pose2D.getX(DistanceUnit.INCH));
34     telemetry.addData("Y coordinate (IN)", pose2D.getY(DistanceUnit.INCH));
35     telemetry.addData("Heading angle (DEGREES)", pose2D.getHeading(AngleUnit.↵
        ↵ DEGREES));
36 }
37
38 void configurePinpoint(){
39     pinpoint.setOffsets(-84.0, -168.0, DistanceUnit.MM); //these are tuned for ↵
        ↵ 3110-0002-0001 Product Insight #1
40     pinpoint.setEncoderResolution(GoBildaPinpointDriver.GoBildaOdometryPods.↵
        ↵ goBILDA_4_BAR_POD);
41     pinpoint.setEncoderDirections(GoBildaPinpointDriver.EncoderDirection.FORWARD,
42                                 GoBildaPinpointDriver.EncoderDirection.FORWARD)↵
        ↵ ;
43     pinpoint.resetPosAndIMU();
44 }
45 }

```

Let's go through this a bit at a time and explain all the pieces.

```

14     GoBildaPinpointDriver pinpoint;

```

This should look familiar. Just like we have a servo of class `Servo` and a motor of class `DcMotor`, we have a variable named `pinpoint` of class `GoBildaPinpointDriver`

```

18     pinpoint = hardwareMap.get(GoBildaPinpointDriver.class, "pinpoint");

```

This should look familiar, it is how we get all of our items from the hardware map

```

19     configurePinpoint();

```

While we could put all of our configuration in the `init()` method, it is better practice to split it out into its own method. This sensor doesn't keep its configuration across power resets, so we should configure it at the beginning of each `opMode`.

```

38     void configurePinpoint(){
39         pinpoint.setOffsets(-84.0, -168.0, DistanceUnit.MM); //these are tuned for ↵
            ↵ 3110-0002-0001 Product Insight #1

```

Here, we first set the offset of where the pinpoint is on the robot.

```

40         pinpoint.setEncoderResolution(GoBildaPinpointDriver.GoBildaOdometryPods.↵
            ↵ goBILDA_4_BAR_POD);

```

21. Some hardware to help with Odometry

We tell it what the resolution is of the encoder. If we are using a GoBilda encoder, there are constants for those.

```
41 pinpoint.setEncoderDirections(GoBildaPinpointDriver.EncoderDirection.FORWARD,  
42                               GoBildaPinpointDriver.EncoderDirection.FORWARD)↵  
    ↵ ;
```

We set whether the encoders are moving forward or reversed (we might put them in reversed if it makes constructing the robot easier)

```
43 pinpoint.resetPosAndIMU();
```

This resets the position and the IMU. This recalibrates the IMU which takes approximately .25 seconds and the robot needs to be stationary during this time

```
20 pinpoint.setPosition(new Pose2D(DistanceUnit.INCH, 0, 0, AngleUnit.DEGREES, ↵  
    ↵ 0));
```

We now set the position of the robot. In this example we used 0,0, heading 0 degrees but you would want to set it based off of where you are placing your robot.

```
27 if(gamepad1.a){  
28     pinpoint.setPosition(new Pose2D(DistanceUnit.INCH, 0, 0, AngleUnit.↵  
        ↵ DEGREES, 0));  
29 }
```

If the a button is pressed on the gamepad, we will update the position. You could also update the position when your robot saw an april tag.

```
30 pinpoint.update();
```

This is CRITICAL. If you don't update the pinpoint, then when you ask for the position you will get the old position.

```
31 Pose2D pose2D = pinpoint.getPosition();  
32  
33 telemetry.addData("X coordinate (IN)", pose2D.getX(DistanceUnit.INCH));  
34 telemetry.addData("Y coordinate (IN)", pose2D.getY(DistanceUnit.INCH));  
35 telemetry.addData("Heading angle (DEGREES)", pose2D.getHeading(AngleUnit.↵  
    ↵ DEGREES));
```

This gets the position and then reports it on telemetry. Obviously you could use the position to figure out when to stop driving, or how to aim a turret, or any number of other decisions.

21.4. Exercises

1. Combine the Sparkfun Optical Tracking Odometry Sensor with the April Tags code to be even more accurate for where you are on the field.

22. LEDs - Adding some bling feedback...

I have had some people ask me about giving some examples of how to add LEDs to your robot. While some teams add these just for the cool factor (which should not be underestimated), other teams add them to give feedback to the drivers on the current state of the robot.

22.1. REV Digital LED Indicator

REV sells a 4 pack of “Digital LED Indicator” (<https://www.revrobotics.com/rev-31-2010/>) which is composed of a red and a green LED. (If you light up both of them, then you get yellow)

These are very easy to mount and give a nice way of showing state. Imagine a game where the possession limit was 2. (like CENTERSTAGE). Your robot could show green if it was ready to intake, yellow if it had one pixel, and red if it had two pixels.

These connect into the Digital IO portion of your hub and when configured the green needs to be the lower of the two numbers for the port and the red should be the higher one.

Listing 22.1: DigitalLEDIndicatorOpMode.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.LED;
6
7 @TeleOp
8 public class DigitalLEDIndicatorOpMode extends OpMode {
9     LED frontLED_red;
10    LED frontLED_green;
11    @Override
12    public void init() {
13        frontLED_green = hardwareMap.get(LED.class, "front_led_green");
14        frontLED_red = hardwareMap.get(LED.class, "front_led_red");
15    }
16
17    @Override
```

22. LEDs - Adding some *bling* feedback...

```
18  public void loop() {  
19      if (gamepad1.a) {  
20          frontLED_red.on();  
21      } else {  
22          frontLED_red.off();  
23      }  
24      if (gamepad1.b) {  
25          frontLED_green.on();  
26      } else {  
27          frontLED_green.off();  
28      }  
29  }  
30 }
```

This is pretty straightforward.

```
9      LED frontLED_red;  
10     LED frontLED_green;
```

We use the LED class for each LED.

```
13     frontLED_green = hardwareMap.get(LED.class, "front_led_green");  
14     frontLED_red = hardwareMap.get(LED.class, "front_led_red");
```

This is just like our `hardwareMap.get` we have used for all other hardware.

```
18  public void loop() {  
19      if (gamepad1.a) {  
20          frontLED_red.on();  
21      } else {  
22          frontLED_red.off();  
23      }  
24      if (gamepad1.b) {  
25          frontLED_green.on();  
26      } else {  
27          frontLED_green.off();  
28      }  
29  }  
30 }
```

This is pretty self explanatory. On your real robot you would likely use either the output of other sensors or variables to decide when to turn them on.

22.2. Sparkfun QWIIC LED Stick

Sparkfun sells a LED Stick with 10 LEDs on it that you can program individually. (<https://www.sparkfun.com/products/18354>)

If you get this, make sure you also pick up this cable (<https://www.sparkfun.com/products/25596>) to make it easy to connect to your Hub (either Control or Expansion Hub). This plugs into the I2C ports (right side).

Listing 22.2: LEDStickOpMode.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import android.graphics.Color;
4
5 import com.qualcomm.hardware.sparkfun.SparkFunLEDStick;
6 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
7 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
8
9 @TeleOp
10 public class LEDStickOpMode extends OpMode {
11     private SparkFunLEDStick ledStick;
12
13     @Override
14     public void init() {
15         ledStick = hardwareMap.get(SparkFunLEDStick.class, "back_leds");
16         int[] ledColors = {Color.RED, Color.YELLOW, Color.GREEN, Color.YELLOW, Color.↵
            ↵ RED,
17             Color.YELLOW, Color.GREEN, Color.YELLOW, Color.RED, Color.YELLOW};
18         ledStick.setColors(ledColors);
19         ledStick.setBrightness(5); // Between 0 and 31
20     }
21
22     @Override
23     public void loop() {
24         if (gamepad1.a) {
25             ledStick.setColor(Color.BLUE);
26         } else if (gamepad1.b) {
27             ledStick.setColor(Color.RED);
28         } else if (gamepad1.left_bumper) {
29             ledStick.turnAllOff();
30         } else {
31             ledStick.setColor(Color.GREEN);
32         }
33     }
34 }

```

This is pretty straightforward and is a great way to add bling to your robot. While you can set each LED to a different color, be aware that may make it hard for the driver to see what is going on. Setting the color of the whole stick to the same color can be a blinding way to send information. You can also put this on the underside of your robot so it will make it glow underneath.

```

15 ledStick = hardwareMap.get(SparkFunLEDStick.class, "back_leds");

```

We use the SparkFunLEDStick class for the stick.

```

16 int[] ledColors = {Color.RED, Color.YELLOW, Color.GREEN, Color.YELLOW, Color.↵
    ↵ RED,

```

22. LEDs - Adding some *bling* feedback...

```
17         Color.YELLOW, Color.GREEN, Color.YELLOW, Color.RED, Color.YELLOW};
18     ledStick.setColors(ledColors);
```

You can use the Android Colors to set the colors of each LED individually by creating an array and then sending it to the LED using the `setColors` method.

```
19     ledStick.setBrightness(5); // Between 0 and 31
```

Here we set the brightness to 5 so it isn't as blinding but you can set it from 0 (off) to 31 (full power)

```
25     ledStick.setColor(Color.BLUE);
```

In addition to setting the colors individually, you can set all of them at once. (You can also set the color of just an individual LED with `setColor(position, color)`)

22.3. REV Blinkin

REV sells a Blinkin LED Driver (<https://www.revrobotics.com/rev-11-1105/>). It allows you to connect a long LED strip and then control it like you are controlling a servo.

This is the most expensive of the solutions we are looking at in this chapter.

Listing 22.3: BlinkinOpMode.java

```
1 package org.firstinspires.ftc.teamcode;
2 import com.qualcomm.hardware.rev.RevBlinkinLedDriver;
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.LED;
6
7 @TeleOp
8 public class BlinkinOpMode extends OpMode{
9     RevBlinkinLedDriver blinkinLedDriver;
10    @Override
11    public void init() {
12        blinkinLedDriver = hardwareMap.get(RevBlinkinLedDriver.class, "blinkin");
13    }
14
15    @Override
16    public void loop() {
17        if(gamepad1.a){
18            blinkinLedDriver.setPattern(RevBlinkinLedDriver.BlinkinPattern.↵
↵ BEATS_PER_MINUTE_PARTY_PALETTE);
19        }
20        else{
```

```

21         blinkinLedDriver.setPattern(RevBlinkinLedDriver.BlinkinPattern.BLACK);
22     }
23 }
24 }

```

There are really only two things to note here:

```

12     blinkinLedDriver = hardwareMap.get(RevBlinkinLedDriver.class, "blinkin");

```

We use the RevBlinkinLedDriver for the class

```

18         blinkinLedDriver.setPattern(RevBlinkinLedDriver.BlinkinPattern.↵
        ↵ BEATS_PER_MINUTE_PARTY_PALETTE);

```

It only has one method that we use and that is setting the pattern. All of the possible patterns are below. The ones that are CP1 have the color set with a screwdriver on the until under color pattern1. CP2 is for color pattern 2, the ones that have both CP1 and CP2 use both of those.

```

RAINBOW_RAINBOW_PALETTE,
RAINBOW_PARTY_PALETTE,
RAINBOW_OCEAN_PALETTE,
RAINBOW_LAVA_PALETTE,
RAINBOW_FOREST_PALETTE,
RAINBOW_WITH_GLITTER,
CONFETTI,
SHOT_RED,
SHOT_BLUE,
SHOT_WHITE,
SINELON_RAINBOW_PALETTE,
SINELON_PARTY_PALETTE,
SINELON_OCEAN_PALETTE,
SINELON_LAVA_PALETTE,
SINELON_FOREST_PALETTE,
BEATS_PER_MINUTE_RAINBOW_PALETTE,
BEATS_PER_MINUTE_PARTY_PALETTE,
BEATS_PER_MINUTE_OCEAN_PALETTE,
BEATS_PER_MINUTE_LAVA_PALETTE,
BEATS_PER_MINUTE_FOREST_PALETTE,
FIRE_MEDIUM,
FIRE_LARGE,
TWINKLES_RAINBOW_PALETTE,
TWINKLES_PARTY_PALETTE,
TWINKLES_OCEAN_PALETTE,
TWINKLES_LAVA_PALETTE,
TWINKLES_FOREST_PALETTE,
COLOR_WAVES_RAINBOW_PALETTE,
COLOR_WAVES_PARTY_PALETTE,
COLOR_WAVES_OCEAN_PALETTE,

```

22. LEDs - Adding some *bling* feedback...

```
COLOR_WAVES_LAVA_PALETTE,
COLOR_WAVES_FOREST_PALETTE,
LARSON_SCANNER_RED,
LARSON_SCANNER_GRAY,
LIGHT_CHASE_RED,
LIGHT_CHASE_BLUE,
LIGHT_CHASE_GRAY,
HEARTBEAT_RED,
HEARTBEAT_BLUE,
HEARTBEAT_WHITE,
HEARTBEAT_GRAY,
BREATH_RED,
BREATH_BLUE,
BREATH_GRAY,
STROBE_RED,
STROBE_BLUE,
STROBE_GOLD,
STROBE_WHITE,
/*
 * CP1: Color 1 Pattern
 */
CP1_END_TO_END_BLEND_TO_BLACK,
CP1_LARSON_SCANNER,
CP1_LIGHT_CHASE,
CP1_HEARTBEAT_SLOW,
CP1_HEARTBEAT_MEDIUM,
CP1_HEARTBEAT_FAST,
CP1_BREATH_SLOW,
CP1_BREATH_FAST,
CP1_SHOT,
CP1_STROBE,
/*
 * CP2: Color 2 Pattern
 */
CP2_END_TO_END_BLEND_TO_BLACK,
CP2_LARSON_SCANNER,
CP2_LIGHT_CHASE,
CP2_HEARTBEAT_SLOW,
CP2_HEARTBEAT_MEDIUM,
CP2_HEARTBEAT_FAST,
CP2_BREATH_SLOW,
CP2_BREATH_FAST,
CP2_SHOT,
CP2_STROBE,
/*
 * CP1_2: Color 1 and 2 Pattern
 */
CP1_2_SPARKLE_1_ON_2,
CP1_2_SPARKLE_2_ON_1,
```

```

CP1_2_COLOR_GRADIENT,
CP1_2_BEATS_PER_MINUTE,
CP1_2_END_TO_END_BLEND_1_TO_2,
CP1_2_END_TO_END_BLEND,
CP1_2_NO_BLENDING,
CP1_2_TWINKLES,
CP1_2_COLOR_WAVES,
CP1_2_SINELON,
/*
 * Solid color
 */
HOT_PINK,
DARK_RED,
RED,
RED_ORANGE,
ORANGE,
GOLD,
YELLOW,
LAWN_GREEN,
LIME,
DARK_GREEN,
GREEN,
BLUE_GREEN,
AQUA,
SKY_BLUE,
DARK_BLUE,
BLUE,
BLUE_VIOLET,
VIOLET,
WHITE,
GRAY,
DARK_GRAY,
BLACK;

```

22.4. Exercises

These exercises do not have solutions in the appendix

1. If you have either the Sparkfun LED stick or the Blinkin, go add colored LEDs to your robot that go along with your team's brand.
2. For any of the three, make them light up when a color sensor sees a game element of your alliance color.

23. Limelight 3A

For the Into the Deep season, FTC made the Limelight 3A legal. <https://limelightvision.io/collections/products/products/limelight-3a> This is a device that has not only a camera but also a processor so you can make your own vision pipelines. This is much like we covered in Chapter [chapter 16](#), but this time instead of it being in Java code and running alongside with your other robot control code it is running on a different device.

This has both benefits and downsides. On the benefits, it has a much easier interface where you can quickly see the effects of your changes and it doesn't use up any of the processor and loop time that is shared with your robot controller. On the downside, it is another piece and type of configuration that you need to make sure you take control of.

23.1. Simple color example

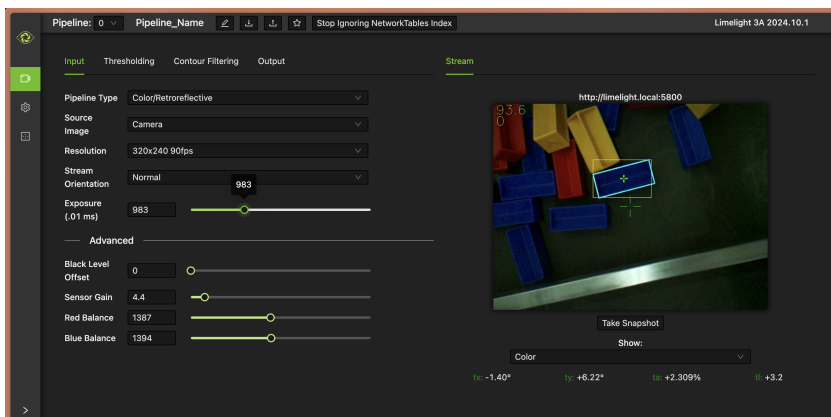
Let's start by looking for game elements that are blue.

23.1.1. On the Limelight

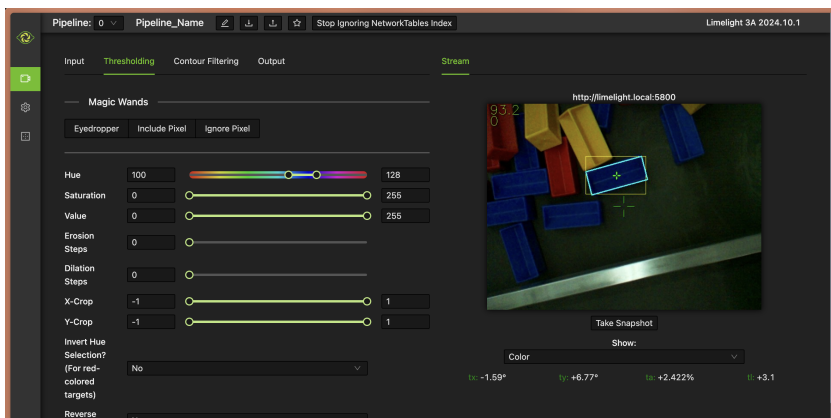
The first thing you need to do is connect your Limelight to a computer and after the lights are blinking on the front go to <http://limelight.local:5801> in your browser.

Select a pipeline - Here we are using Pipeline 0, but as long as it matches in your Java code and on the limelight, you can use any of the 10 (0-9) pipelines available. If it isn't letting you change the pipeline number, it is often because you need to press the "Start Ignoring NetworkTables index"

23. Limelight 3A



The first tab we want to work with is the “Input” tab. Here we set the pipeline type - For detecting Colors it should be “Color/Retroreflective”. We can change whether it is dealing with what the camera sees or a snapshot. This is useful for taking a snapshot when you are on a field and then working with it later. (To take a snapshot, you press the “Take Snapshot” button under the image on the right.) The resolution of the camera can be set here. In general you want the lowest resolution that will work because it takes less effort to process the images. The “Stream Orientation” allows you to mount the camera in different orientations. The Exposure should be set to the lowest number where you can clearly see the game elements. It does not need to be bright.



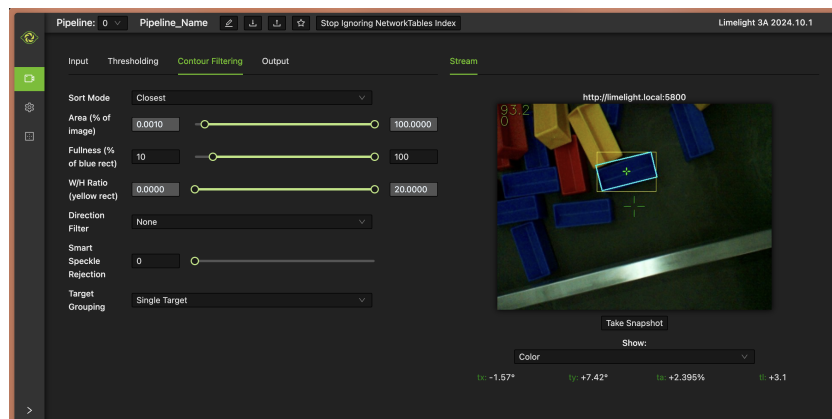
Now it is time to go to the second tab, which is “Thresholding”. Select the Hue and drag it (both the lower and the upper) until it is clearly finding the blue sample. While there are magic wands to use “eyedropper”, I find that it is too selective and is easier to leave everything else. The “Color” Pipelines

in Limelight use HSV. You can see an explanation of color spaces in Chapter [chapter 16](#).

Saturation is how “pure” the color is. More washed out is lower. You can set this if you are having problems with too many things being detected. You may want to raise this so you don’t get problems with the light reflecting off of the shiny metal.

Value is the darkness of a color. You should increase this so black won’t come through the filter.

Under the image, you should change Show from “Color” to “Threshold” so you will see what is coming through the filter.

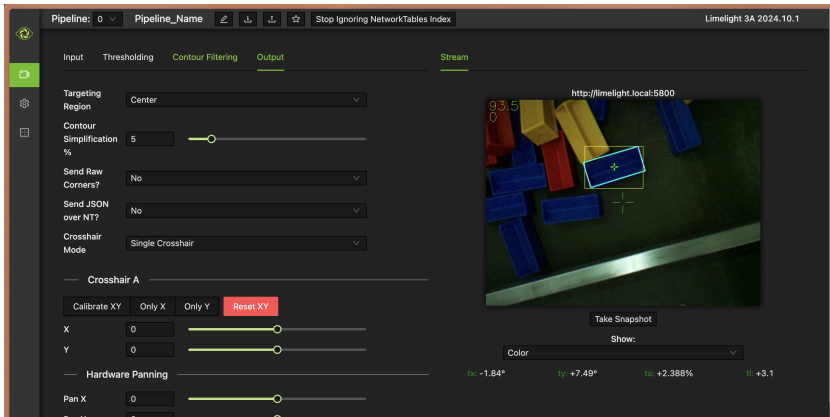


Now it is time to go to the third tab (Contour Filtering). This does filtering after the items applied in the second tab. Sort Mode allows you to decide which item has priority. I selected “Closest” which means the closest to your crosshairs (middle of the screen unless you move it on the output tab)

Area allows you to reject very small items. Fullness allows you to reject items that aren’t mostly that color. (This is useful in Into the Deep when the samples are rectangles, but may not be as useful in the future.)

W/H ratio is useful when the items are always a certain ratio but since in Into the Deep they are randomly distributed, it isn’t useful. Direction Filter allows you to only select items that are pointing a different way. (Say for example if you could only pick up items in a certain orientation)

23. Limelight 3A



The fourth (and last tab) is the Output tab. Here you can change where your cross hairs are. (say for example if your camera is offset from how you pick up the blocks)

I then recommend setting the pipeline name (press the pencil icon on the top row) and downloading it. The icon with the arrow pointing down. This will give you a file that you can save and then upload again to make sure you have your settings backed up.

23.1.2. Your Java Code

Now that you have your pipeline all ready, we have to talk to the Limelight with our Java code.

Listing 23.1: SimpleLimelightOpMode.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.hardware.limelightvision.LLResult;
4 import com.qualcomm.hardware.limelightvision.LLResultTypes;
5 import com.qualcomm.hardware.limelightvision.Limelight3A;
6 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
7 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
8
9 import java.util.List;
10
11 @TeleOp
12 public class SimpleLimelightOpMode extends OpMode {
13     Limelight3A limelight3A;
14     @Override
15     public void init() {
16         limelight3A = hardwareMap.get(Limelight3A.class, "limelight");
17     }
18 }
```

```

18
19     public void start(){
20         limelight3A.pipelineSwitch(0);
21         limelight3A.start();
22     }
23
24     @Override
25     public void loop() {
26         LLResult llResult = limelight3A.getLatestResult();
27         if(llResult != null && llResult.isValid()){
28             telemetry.addData("Tx", llResult.getTx());
29             telemetry.addData("Ty", llResult.getTy());
30             telemetry.addData("Ta", llResult.getTa());
31         }
32         else{
33             telemetry.addLine("None found");
34         }
35     }
36 }

```

There are three main parts to our program now. The `init()`, the `start()`, and the `loop()`.

```

13     Limelight3A limelight3A;
14     @Override
15     public void init() {
16         limelight3A = hardwareMap.get(Limelight3A.class, "limelight");
17     }

```

The Limelight is of type `Limelight3A` and this assumes you have named it `limelight` in your configuration file. Obviously if you name it differently there, you'll have to fix it here.

```

19     public void start(){
20         limelight3A.pipelineSwitch(0);
21         limelight3A.start();
22     }

```

This could be all in `init()` but since the Limelight consumes more power when it is running, it seems like a good idea to start it here. We also switch the pipeline. It is fast enough to switch that we could do it after we start it but it seems like a good practice to be using the right pipeline before starting.

```

26         LLResult llResult = limelight3A.getLatestResult();

```

We get the result from the Limelight

```

27         if(llResult != null && llResult.isValid()){
28             telemetry.addData("Tx", llResult.getTx());

```

23. Limelight 3A

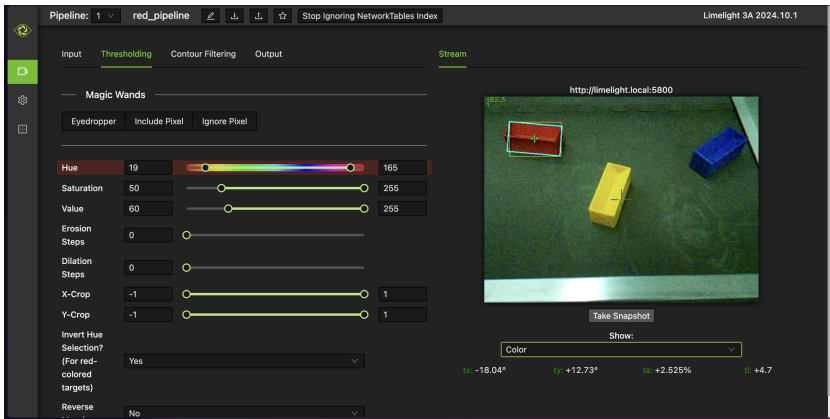
```
29 telemetry.addData("Ty", llResult.getTy());
30 telemetry.addData("Ta", llResult.getTa());
31 }
```

Here we only print the result if the last one we got was valid. We also have to make sure that `llResult` was not null so we won't get a crash when we call it and it is null. (If you call `getLatestResult` before a result has been returned, then it will be null. Since we start it in `start()`, and then immediately check it our first time it could be null.) `Tx` is the x of the target, `Ty` is the y of the target, and `Ta` is the angle of the target. (All are relative to the crosshairs.)

We could do something different if we didn't see any valid targets.

23.1.3. Changing Limelight Pipeline

Now, let's change our pipeline so instead of finding the blue game elements the Limelight will return the red game elements it sees. Make sure you do this one under Pipeline 1. If it won't let you change the pipeline number then you need to press the box at the top that says `Stop Ignoring NetworkTables Index`.



The thing that is tricky is that you need to set “Invert Hue selection” to yes since you are looking for a red target (red is at the bottom and top of Hue, so it is the only color you need to do this for.) Then change the hue until it is identifying the red sample.

23.1.4. Swapping between pipelines

Listing 23.2: `TwoPipelinesLimelightOpMode.java`

```
1 package org.firstinspires.ftc.teamcode;
```

```

2
3 import com.qualcomm.hardware.limelightvision.LLResult;
4 import com.qualcomm.hardware.limelightvision.LLResultTypes;
5 import com.qualcomm.hardware.limelightvision.Limelight3A;
6 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
7 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
8
9 import java.util.List;
10
11 @TeleOp
12 public class TwoPipelinesLimelightOpMode extends OpMode {
13     Limelight3A limelight3A;
14     @Override
15     public void init() {
16         limelight3A = hardwareMap.get(Limelight3A.class, "limelight");
17     }
18     public void start(){
19         limelight3A.pipelineSwitch(0);
20         limelight3A.start();
21     }
22
23     @Override
24     public void loop() {
25         if(gamepad1.a){
26             limelight3A.pipelineSwitch(0);
27         }else if(gamepad1.b){
28             limelight3A.pipelineSwitch(1);
29         }
30         LLResult llResult = limelight3A.getLatestResult();
31         if(llResult != null) {
32             telemetry.addData("Pipeline Num", llResult.getPipelineIndex());
33             if (llResult.isValid()) {
34                 telemetry.addData("Tx", llResult.getTx());
35                 telemetry.addData("Ty", llResult.getTy());
36                 telemetry.addData("Ta", llResult.getTa());
37             }
38         }
39     }
40 }

```

The only difference between this one and the earlier one is where we switch which pipeline is active and displaying which pipeline is active. First, the switching.

```

25         if(gamepad1.a){
26             limelight3A.pipelineSwitch(0);
27         }else if(gamepad1.b){
28             limelight3A.pipelineSwitch(1);
29         }

```

23. Limelight 3A

This uses the A button to select the pipeline zero and the B button to select the pipeline one. (Very tough, huh....)

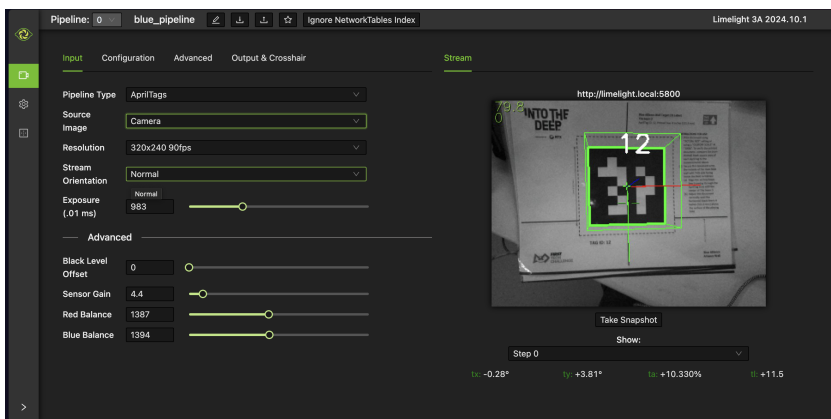
```
31 if(llResult != null) {
```

This sends which pipeline was used to get the result.

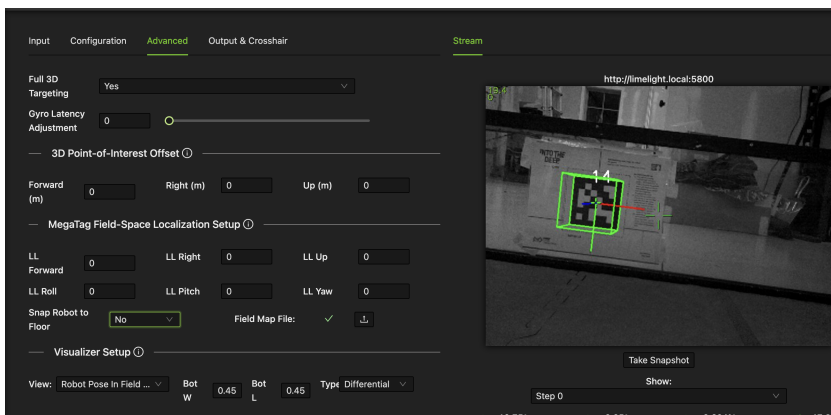
23.2. Localization with AprilTags

Limelight also has AprilTags support and you can load an image of the field into the Limelight saying where the april tags are and it can give you back your robot's position.

23.2.1. On the Limelight



Simply change the Pipeline Tag to be AprilTags on the Input screen



Make sure you have the current year's field loaded. To change the field map, See the upload button next to "Field Map Floor" on the "Advanced" tab. You can then set the view to Robot Pose in Field and you can see your robot placed on the field as you move it around.

23.2.2. Your Java Code

Listing 23.3: AprilTagsLimelightOpMode.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.hardware.limelightvision.LLResult;
4 import com.qualcomm.hardware.limelightvision.Limelight3A;
5 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
6 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
7
8 import org.firstinspires.ftc.robotcore.external.navigation.Pose3D;
9
10 @TeleOp
11 public class AprilTagsLimelightOpMode extends OpMode {
12     Limelight3A ll3a;
13     @Override
14     public void init() {
15         ll3a = hardwareMap.get(Limelight3A.class, "limelight");
16         ll3a.pipelineSwitch(2);
17     }
18     public void start(){
19         ll3a.start();
20     }
21
22     @Override
23     public void loop() {
24         LLResult llResult = ll3a.getLatestResult();
25
26         if(llResult != null && llResult.isValid()) {
27             Pose3D pose3d = llResult.getBotpose();
28             telemetry.addData("Bot Pose", pose3d);
29         }
30     }
31 }

```

You'll notice that this is very straightforward. If we see an AprilTag (there is a valid LLResult) then we'll put our BotPose in Telemetry. Obviously it doesn't require much imagination to use this in autonomous to know where your robot is.

23.3. Exercises

1. Make a pipeline that looks for yellow blocks (no solution given)
2. Make a teleop that if it doesn't see any targets rumbles the gamepad (no solution given)

24. Introduction to Control Theory

One of the things that is different about embedded software (software that is part of something physical) is that it takes time in the real world for motors to get to a certain place and they can overshoot (go too far)

Control theory is all about how to take system inputs and turn them into what command we send to the motor. (Control theory can apply to more than just motors, but for FTC that is the main place we use them.)

Here is some terminology that we'll use in this chapter.

- Where something is, is its position. (units here are meters (m)) (We'll talk about this all in terms of 1D or X, but the same thing applies in 3D or X,Y, and Z)
- How fast something's position changes, is its velocity. ($\frac{m}{sec}$)
- How fast something's velocity changes, is its acceleration ($\frac{m}{sec^2}$)
- How fast something's acceleration changes, is its jerk ($\frac{m}{sec^3}$)
- For those that are curious, the next 3 are snap ($\frac{m}{sec^4}$), crackle ($\frac{m}{sec^5}$), and pop ($\frac{m}{sec^6}$). I have never seen these used for real, but I find these terms amusing.

You can now impress your friends because this is calculus. (technically differential calculus). The calculus way of saying this is that the derivative of the position with respect to time is the velocity. The derivative of the velocity with respect to time is the acceleration. The derivative of the acceleration with regards to time is the jerk and so on....

Why does this matter? Well, your motor (and whatever is attached) has physical limits for each of these.

Two that we'll think about the most:

1. Acceleration is often limited by the amount of energy that a motor can supply. This has one of my favorite formulas: $F = ma$ which is Force equals mass times acceleration. If you think about this, it makes sense. You can speed up something light much faster than you can something heavy with the same amount of effort.

24. Introduction to Control Theory

2. Jerk can cause your robot to tear itself apart. That is one reason why in more advanced control theory we try to limit how quickly we change our acceleration.

24.1. Open Loop Control

Open Loop control means we don't get any additional information from the system. A good example of this would be an intake. Here is a simple example:

Listing 24.1: OpenLoopOpMode1.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.DcMotor;
6
7 @TeleOp
8 public class OpenLoopOpMode1 extends OpMode {
9     DcMotor intake;
10    @Override
11    public void init() {
12        intake = hardwareMap.get(DcMotor.class, "motor");
13    }
14
15    @Override
16    public void loop() {
17        if (gamepad1.a){
18            intake.setPower(1.0);
19        } else if (gamepad1.b){
20            intake.setPower(-1.0);
21        } else {
22            intake.setPower(0.0);
23        }
24    }
25 }
```

The only interesting bits here are in the `loop()` method. You'll notice that instead of having 3 `if` statements that we use `if-else` so that we cannot send multiple powers to the motor at the same time.

This is pretty straight forward. When we hold the a button, we make the intake go one direction. When we hold the b button (and not the a one), then it goes in the other direction. If neither are held, then the intake stops.

In Android Studio, you can right click on a constant, choose refactor and then introduce constant and name the constants so they are more easily readable.

Listing 24.2: OpenLoopOpMode2.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.DcMotor;
6
7 @TeleOp
8 public class OpenLoopOpMode2 extends OpMode {
9     public static final double INTAKE_POWER = 1.0;
10    public static final double EJECT_POWER = -1.0;
11    public static final double STOP_POWER = 0.0;
12    DcMotor intake;
13    @Override
14    public void init() {
15        intake = hardwareMap.get(DcMotor.class, "motor");
16    }
17
18    @Override
19    public void loop() {
20        if (gamepad1.a){
21            intake.setPower(INTAKE_POWER);
22        } else if (gamepad1.b){
23            intake.setPower(EJECT_POWER);
24        } else {
25            intake.setPower(STOP_POWER);
26        }
27    }
28 }

```

As you probably realize, a motor can't immediately go from full forward to full reverse. Here is a version that is a little smoother for our robot.

Listing 24.3: OpenLoopOpMode3.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.DcMotor;
6
7 @TeleOp
8 public class OpenLoopOpMode3 extends OpMode {
9     public static final double INTAKE_POWER = 1.0;
10    public static final double EJECT_POWER = -1.0;
11    public static final double STOP_POWER = 0.0;
12    public static final double CHANGE_AMOUNT_INTAKE = 0.01;
13    public static final double CHANGE_AMOUNT_EJECT = -0.01;
14    DcMotor intake;

```

24. Introduction to Control Theory

```
15     double power;
16     @Override
17     public void init() {
18         intake = hardwareMap.get(DcMotor.class, "motor");
19     }
20
21     @Override
22     public void loop() {
23         if (gamepad1.a){
24             if (power < STOP_POWER){
25                 power = STOP_POWER;
26             }
27             power += CHANGE_AMOUNT_INTAKE;
28         } else if (gamepad1.b){
29             if (power > 0){
30                 power = STOP_POWER;
31             }
32             power += CHANGE_AMOUNT_EJECT;
33         } else {
34             power = STOP_POWER;
35         }
36         if (power > INTAKE_POWER){ // assumes intake power is positive
37             power = INTAKE_POWER;
38         }
39         if (power < EJECT_POWER){ // assumes eject power is negative
40             power = EJECT_POWER;
41         }
42         intake.setPower(power);
43     }
44 }
```

Another option you can have for open loop control is to have it go for an amount of time. Let's imagine that you have a slide and you realized that you don't have an encoder cable. (because that would be the only legitimate reason to not have one attached, right??)

Listing 24.4: OpenLoopOpMode4.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.DcMotor;
6 import com.qualcomm.robotcore.util.ElapsedTime;
7
8 @TeleOp
9 public class OpenLoopOpMode4 extends OpMode {
10     public static final double EXTRACT_POWER = 0.5;
11     public static final double RETRACT_POWER = -0.5;
```

```

12     DcMotor slideMotor;
13     ElapsedTime timer;
14
15     @Override
16     public void init() {
17         slideMotor = hardwareMap.get(DcMotor.class, "motor");
18     }
19
20     @Override
21     public void loop() {
22         if (timer.seconds() > 1.0){
23             slideMotor.setPower(0);
24         }
25         //NOTE: This is for example only, don't really do this....
26         if (gamepad1.aWasPressed()){
27             slideMotor.setPower(EXTRACT_POWER);
28             timer.reset();
29         }
30         else if(gamepad1.bWasPressed()){
31             slideMotor.setPower(RETRACT_POWER);
32             timer.reset();
33         }
34     }
35 }
36
37 }

```

24.2. Closed Loop Control - Intro

Closed loop control means that we are modifying our input to our system based on its output. A simple example of this is a thermostat in the winter. It is reading the temperature in the room and if it is below the set temperature then it turns on the furnace. If it is above the set temperature than it turns off the furnace.

24.3. Closed loop control - Bang Bang

The simplest form of closed loop control is called “bang bang” because it only has 2 extreme states. The thermostat we looked at earlier is a great example of this. Let’s imagine that we have a slide.

Listing 24.5: BangBangOpMode1.java

```

1 package org.firstinspires.ftc.teamcode;

```

24. Introduction to Control Theory

```
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.DcMotor;
6 import com.qualcomm.robotcore.util.ElapsedTime;
7
8 @TeleOp
9 public class BangBangOpModel1 extends OpMode {
10     public static final double EXTRACT_POWER = 0.5;
11     public static final double RETRACT_POWER = -0.5;
12     DcMotor slideMotor;
13     static int FULLY_IN_POSITION = 0;
14     static int FULLY_OUT_POSITION = 1100; // TUNE for your slide
15     int desiredPosition = 0;
16
17     @Override
18     public void init() {
19         slideMotor = hardwareMap.get(DcMotor.class, "motor");
20         slideMotor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
21         slideMotor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);
22     }
23
24     @Override
25     public void loop() {
26         if (gamepad1.dpadRightWasPressed()){
27             desiredPosition += 100;
28             desiredPosition = Math.min(desiredPosition, FULLY_OUT_POSITION);
29         }
30         else if (gamepad1.dpadLeftWasPressed()){
31             desiredPosition -= 100;
32             desiredPosition = Math.max(desiredPosition, FULLY_IN_POSITION);
33         }
34         telemetry.addData("Desired Position", desiredPosition);
35         setSlideMotor();
36     }
37
38     void setSlideMotor(){
39         int actualPosition = slideMotor.getCurrentPosition();
40         if (desiredPosition < actualPosition){
41             slideMotor.setPower(EXTRACT_POWER);
42         } else {
43             slideMotor.setPower(RETRACT_POWER);
44         }
45     }
46
47 }
```

Let's talk about the different pieces here:

```

18     public void init() {
19         slideMotor = hardwareMap.get(DcMotor.class, "motor");
20         slideMotor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
21         slideMotor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);
22     }

```

After we get the motor out of the hardware map, we reset the encoder. After doing this the position we get from the motor will read 0. This means it is critical that we reset the slide before our init is called because whatever that point is will be considered the 0 encoder reading. RUN_WITHOUT_ENCODER is unfortunately named because what it means is that the encoder isn't used for us setting the power to the motor. But the encoder is still working and reporting the position whenever we read it.

```

25     public void loop() {
26         if (gamepad1.dpadRightWasPressed()){
27             desiredPosition += 100;
28             desiredPosition = Math.min(desiredPosition, FULLY_OUT_POSITION);
29         }
30         else if (gamepad1.dpadLeftWasPressed()){
31             desiredPosition -= 100;
32             desiredPosition = Math.max(desiredPosition, FULLY_IN_POSITION);
33         }
34         telemetry.addData("Desired Position", desiredPosition);

```

This allows us to set the desired position and reports it to the user using telemetry.

```

38     void setSlideMotor(){
39         int actualPosition = slideMotor.getCurrentPosition();
40         if (desiredPosition < actualPosition){
41             slideMotor.setPower(EXTRACT_POWER);
42         } else {
43             slideMotor.setPower(RETRACT_POWER);
44         }
45     }

```

This gets the actual position and if it is less than the desired position we put the motor into extract. If it is greater than we retract it.

24.3.1. Adding Hysteresis (Dead band)

```

38     void setSlideMotor(){
39         int actualPosition = slideMotor.getCurrentPosition();
40         int error = desiredPosition - actualPosition;
41
42         if (Math.abs(error) < DEADBAND_AMOUNT){

```

24. Introduction to Control Theory

```
43         slideMotor.setPower(0);
44     }
45     else if (desiredPosition < actualPosition){
46         slideMotor.setPower(EXTRACT_POWER);
47     } else {
48         slideMotor.setPower(RETRACT_POWER);
49     }
50 }
```

Here we add a simple step that says if we are within our dead band amount (either positive or negative) then we stop the motor. This is like the fancy thermometers that allow you to set a range of temperatures you are comfortable with and when it is below the bottom one it turns on the heat, and when it is above the top one it turns on the air conditioning.

While this may seem trivial, bang bang (and bang bang with Hysteresis) probably controls a vast majority of the embedded systems in the world.

24.3.2. Using a limit switch to improve!!

Listing 24.6: BangBangOpMode3.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5 import com.qualcomm.robotcore.hardware.DcMotor;
6 import com.qualcomm.robotcore.hardware.DigitalChannel;
7
8 @TeleOp
9 public class BangBangOpMode3 extends OpMode {
10     public static final double EXTRACT_POWER = 0.5;
11     public static final double RETRACT_POWER = -0.5;
12     DcMotor slideMotor;
13     DigitalChannel limitSwitch;
14     static int FULLY_IN_POSITION = 0;
15     static int FULLY_OUT_POSITION = 1100; // TUNE for your slide
16     static int DEADBAND_AMOUNT = 100;
17     int desiredPosition = 0;
18
19     @Override
20     public void init() {
21         slideMotor = hardwareMap.get(DcMotor.class, "motor");
22         limitSwitch = hardwareMap.get(DigitalChannel.class, "touch_sensor");
23         limitSwitch.setMode(DigitalChannel.Mode.INPUT);
24     }
25
26     @Override
```

```

27 public void init_loop(){
28     if (!limitSwitch.getState()){
29         slideMotor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
30         slideMotor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);
31     }
32     else{
33         slideMotor.setPower(RETRACT_POWER);
34     }
35 }
36
37 @Override
38 public void loop() {
39     if (gamepad1.dpadRightWasPressed()){
40         desiredPosition += 100;
41         desiredPosition = Math.min(desiredPosition, FULLY_OUT_POSITION);
42     }
43     else if (gamepad1.dpadLeftWasPressed()){
44         desiredPosition -= 100;
45         desiredPosition = Math.max(desiredPosition, FULLY_IN_POSITION);
46     }
47     telemetry.addData("Desired Position", desiredPosition);
48     setSlideMotor();
49 }
50
51 void setSlideMotor(){
52     int actualPosition = slideMotor.getCurrentPosition();
53     int error = desiredPosition - actualPosition;
54
55     if (Math.abs(error) < DEADBAND_AMOUNT){
56         slideMotor.setPower(0);
57     }
58     else if (desiredPosition < actualPosition){
59         slideMotor.setPower(EXTRACT_POWER);
60     } else {
61         slideMotor.setPower(RETRACT_POWER);
62     }
63 }
64
65 }

```

Here we add a limit switch (a switch that gets pressed when we are at our limit. In this case when we are fully retracted). Our `init_loop` will cause us to retract until the switch is pressed. This gets rid of the problem we had earlier where the encoder could be reset at the incorrect position. One rule for your team should be “whenever the robot or the driver makes a mistake, fix the robot so the mistake can’t happen again.”

24.4. Closed loop control - Proportional

The next step is that we can go faster the farther away we are from the desired position and slower the closer in we are. This is “proportional” - our speed is proportional to how far away we are. Instead of showing the whole thing, we are going to just show the `setSlideMotor` method since that is the only thing that is changing.

```
51 void setSlideMotor(){
52     double kP = .001;
53
54     int actualPosition = slideMotor.getCurrentPosition();
55     telemetry.addData("Actual Position", actualPosition);
56
57     int error = desiredPosition - actualPosition;
58     double power = error * kP;
59
60     slideMotor.setPower(power);
61     telemetry.addData("Power", power);
62 }
```

You’ll notice that we get the error by subtracting our actual position from our desired position. We then set the power by multiplying our error by `kP`. In papers this is often referred to as K_p but since Java doesn’t support subscripts, it is typically called `kP`. (I don’t know why the `k` is typically lower case.)

The higher the value of `kP`, the faster the system will get to the desired point but the more it will overshoot and oscillate around the desired point. I often start by thinking about what value of `kP` will give me full speed when I am as far away as I can get and will give me 0.1 when I am close enough.

24.5. Closed loop control - Full PID(f)

Don’t let PID scare you. While it stands for Proportional, Integral, Derivative which sounds like scary math, in reality it is pretty straightforward.

The P term is how far away from the desired state you currently are.

The I term is your sum of errors. That way, if you aren’t getting closer you can get an additional shove to get to your end point.

The D term is how fast your error is changing. This can help you keep from changing too much and overshooting.

So you end up with a formula that looks like: $power = (K_p * error) + (K_i * integralSum) + (K_d * derivative)$

```
53 static final double kP = 0.001;
```

```

54  static final double kI = 0.01;
55  static final double kD = 0.001;
56
57  double lastError = 0;
58  double sumErrors = 0;
59  void setSlideMotor(int desiredPosition){
60      int actualPosition = slideMotor.getCurrentPosition();
61      telemetry.addData("Actual Position", actualPosition);
62
63      int error = desiredPosition - actualPosition;
64      double derivative = (error - lastError) / timer.seconds();
65      sumErrors = sumErrors + (error * timer.seconds());
66
67      double power = (kP * error) + (kI * sumErrors) + (kD * derivative);
68
69      slideMotor.setPower(power);
70      telemetry.addData("Power", power);
71
72      timer.reset();
73      lastError = error;
74  }

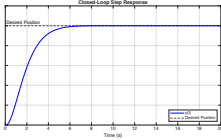
```

24.5.1. Tuning your PID

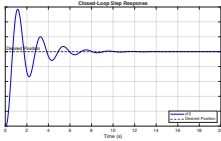
There are a few different ways to tune a PID, but this is a common one that works for many systems. Note: these are completely synthetic graphs so they don't contain any noise.

1. Start with kP, kI, and kD all at 0.
2. Increase kP until steady-state error is very low. (ie, it gets to where you want it to go in a reasonable time)

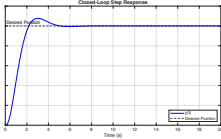
24. Introduction to Control Theory



When your k_P is too small (it takes a long time to get to desired value)

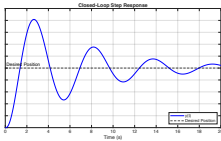


When your k_P is too big (it way overshoots and oscillates)

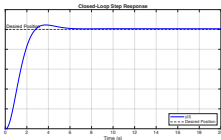


When your k_P looks good (but k_I and k_D are still 0)

3. Increase k_I until steady-state error is gone. (ie, you see that it goes to exactly the desiredPosition that you set)

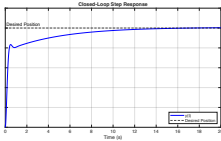


When your k_I is too high (notice this looks similar to your k_P being too high which is why it is important you do them one at a time)

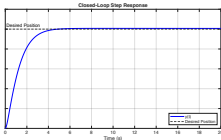


When your k_I looks good (but k_D is still 0). For many systems you can leave k_I at 0 and it will work just fine.

4. Increase k_D until oscillations are removed



When your k_D is too high, you can get weird graphs like this



When your system is all tuned

The trick here is to get “good enough” and not spend all of your time tuning your PID instead of driving your robot.

24.5.2. When to use F term

If your system requires some power to stay in the steady state, then you need an F term (this stands for Feedforward). This changes your formula to: $power = (K_p * error) + (K_i * integralSum) + (K_d * derivative) + K_f$

The most common reason for this is when you have slides that are vertical and you need some power to compensate for gravity. In order to find the value for kF , one way to do that would be to hold the slide up, set kP , kI , and kD to 0 and set kF at the value needed to hold it stationary.

```
78      double power = (kP * error) + (kI * sumErrors) + (kD * derivative) + kF;
```

24.5.3. Improving your PIDF

One of the huge problems that people have with PID systems is the integral part can become large. (This is called integral windup.) To fix this, you can limit how much the integral portion can affect things and also reset it when the desired position changes. Here is a version with all of those changes.

```
59      int lastDesiredPosition = 0;
60
61      void setSlideMotor(int desiredPosition) {
62          int actualPosition = slideMotor.getCurrentPosition();
63          telemetry.addData("Actual Position", actualPosition);
64
65          int error = desiredPosition - actualPosition;
66          double derivative = (error - lastError) / timer.seconds();
67          sumErrors = sumErrors + (error * timer.seconds());
68
69          if (desiredPosition != lastDesiredPosition) {
70              sumErrors = 0;
71              lastDesiredPosition = desiredPosition;
72          } else {
73              if (Math.abs(sumErrors) > MAX_INTEGRAL_SUM){
74                  sumErrors = Math.signum(sumErrors) * MAX_INTEGRAL_SUM;
75              }
76          }
77
78          double power = (kP * error) + (kI * sumErrors) + (kD * derivative) + kF;
79
80          slideMotor.setPower(power);
81          telemetry.addData("Power", power);
82
83          timer.reset();
84          lastError = error;
85      }
```

24. Introduction to Control Theory

Let's break out some of the pieces and talk about what they do.

```
69     if (desiredPosition != lastDesiredPosition) {
70         sumErrors = 0;
71         lastDesiredPosition = desiredPosition;
72     } else {
```

We reset the sumErrors when desiredPosition changes

```
72     } else {
73         if (Math.abs(sumErrors) > MAX_INTEGRAL_SUM) {
74             sumErrors = Math.signum(sumErrors) * MAX_INTEGRAL_SUM;
75         }
76     }
```

This makes sure that our sumErrors never gets larger than our MAX_INTEGRAL_SUM limit in either the positive or the negative direction.

24.5.4. Built-in vs Roll your own

24.5.4.1. Built-in PIDF

There is a way to use the SDK for a built-in PIDF. Here is an example file showing using it:

Listing 24.7: BuiltInPIDOpMode.java

```
1  package org.firstinspires.ftc.teamcode;
2
3  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5  import com.qualcomm.robotcore.hardware.DcMotor;
6  import com.qualcomm.robotcore.hardware.DcMotorEx;
7  import com.qualcomm.robotcore.hardware.DigitalChannel;
8  import com.qualcomm.robotcore.hardware.PIDFCoefficients;
9
10 @TeleOp
11 public class BuiltInPIDOpMode extends OpMode {
12     public static final double RETRACT_POWER = -0.5;
13     DcMotor slideMotor;
14     DigitalChannel limitSwitch;
15     static int FULLY_IN_POSITION = 0;
16     static int FULLY_OUT_POSITION = 1100; // TUNE for your slide
17     int desiredPosition = 0;
18
19     static final double kP = 0.001;
20     static final double kI = 0.01;
21     static final double kD = 0.001;
22     static final double kF = 0;
```

```

23  @Override
24  public void init() {
25      slideMotor = hardwareMap.get(DcMotor.class, "motor");
26      ((DcMotorEx)slideMotor).setPIDFCoefficients(DcMotor.RunMode.RUN_TO_POSITION, ↵
          ↵ new PIDFCoefficients(kP, kD, kI, kF));
27      limitSwitch = hardwareMap.get(DigitalChannel.class, "touch_sensor");
28      limitSwitch.setMode(DigitalChannel.Mode.INPUT);
29  }
30
31  @Override
32  public void init_loop() {
33      if (!limitSwitch.getState()) {
34          slideMotor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
35          slideMotor.setTargetPosition(0);
36          slideMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);
37      } else {
38          slideMotor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);
39          slideMotor.setPower(RETRACT_POWER);
40      }
41  }
42
43  @Override
44  public void loop() {
45      if (gamepad1.dpadRightWasPressed()) {
46          desiredPosition += 100;
47          desiredPosition = Math.min(desiredPosition, FULLY_OUT_POSITION);
48      } else if (gamepad1.dpadLeftWasPressed()) {
49          desiredPosition -= 100;
50          desiredPosition = Math.max(desiredPosition, FULLY_IN_POSITION);
51      }
52      telemetry.addData("Desired Position", desiredPosition);
53      slideMotor.setTargetPosition(desiredPosition);
54  }
55  }

```

There are only a few tricky things to know here.

```

26      ((DcMotorEx)slideMotor).setPIDFCoefficients(DcMotor.RunMode.RUN_TO_POSITION, ↵
          ↵ new PIDFCoefficients(kP, kD, kI, kF));

```

You have to cast your motor as a `DcMotorEx` to get access to the method `setPIDFCoefficients` and you have to set them for the run mode you want to use it with. (typically `RUN_TO_POSITION`).

```

35      slideMotor.setTargetPosition(0);
36      slideMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);

```

If you haven't set a target position and you change the mode to `RUN_TO_POSITION`, then you will get an exception and your program will

24. *Introduction to Control Theory*

die (unless you are handling it).

24.5.4.2. How to decide

So if there is a built-in PIDF controller, why would we write our own??

For reasons that baffle my mind, one written in your code is more responsive than one that uses the internal hardware on the Control Hub and Expansion Hub.

But for many teams, the built-in one is just fine.

24.6. For more information

This has been a VERY brief introduction to control theory. For more detail I recommend the website: <https://www.ctrlaltftc.com/>

24.7. Exercises

There are no exercises for this chapter other than implementing on your own robot.

25. Behavior Trees (another way to handle auto)

While Behavior Trees were originally created for robotics, they came to be used in video game design for NPC (Non playable characters) as an easier way to define their behaviors. One of the huge benefits of using behavior trees is that it lets you test out your logic separate from your programming. The other benefit is they allow you to write small pieces of code that can be tested independently and then combined together to do amazing things. (much like LEGO toy bricks.)

This is going to seem complicated at first, but I promise it will make sense when you see it all together.

Behavior Trees are made up of a combination of execution nodes (these actually do the work.) and control flow nodes (determines what is to be done next). When each node is called (“ticked”), it returns one of three statuses.

1. `SUCCESS` - The node completed its task successfully
2. `FAILURE` - The node was unable to complete its task
3. `RUNNING` - The node is still working on its task

25.1. Execution Nodes

So the simplest tree you could have would be a single execution node:

`SingleAction`

In this case, you would continue to call the `tick` method of this node until it returned `SUCCESS` or `FAILURE`.

25.1.1. Code for simplest Behavior Tree

Under Gradle Scripts, you’ll see a file called `build.dependencies.gradle`. Inside the section that is `dependencies {`, you’ll need to add the following line (either at the top or bottom, it doesn’t matter).

25. Behavior Trees (another way to handle auto)

```
implementation 'com.ftcteams:behaviortrees:0.0.2'
```

What this does is add a library to what is available for our code. We are going to start with this split into 2 files. That way we can keep the behavior tree code separate.

First, we'll look at the behavior tree:

Listing 25.1: BehaviorTrees/BTSimple.java

```
1 package org.firstinspires.ftc.teamcode.BehaviorTrees;
2
3 import com.ftcteams.behaviortrees.DebugTree;
4 import com.ftcteams.behaviortrees.Node;
5 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
6
7 public class BTSimple {
8     public static class SimpleAction extends Node{
9         @Override
10        public State tick(DebugTree debug, Object obj) {
11            ((OpMode)obj).telemetry.addData("SimpleAction", "Success");
12            return State.SUCCESS;
13        }
14    }
15
16    public static Node root(){
17        return new SimpleAction();
18    }
19 }
```

Now let's talk through the parts here.

```
8     public static class SimpleAction extends Node{
```

We are making our own node here so we need to extend from Node

```
10        public State tick(DebugTree debug, Object obj) {
```

We don't need to do anything with the debug parameter. It allows some house-keeping in keeping track of which nodes were visited. We'll talk about this in a later section. The obj parameter is what gets passed to all nodes. We can determine what this is.

```
11            ((OpMode)obj).telemetry.addData("SimpleAction", "Success");
```

This line looks a little strange, but what we are doing is telling the compiler that the obj passed in is of type OpMode and that allows us to call all its methods. (If we tell it a type that isn't what is passed in (or a parent class of what is passed in) we will get an exception at runtime.)

```
12     return State.SUCCESS;
```

This returns SUCCESS, so it will only be called once.

```
16     public static Node root(){
17         return new SimpleAction();
18     }
```

By convention we have a method that returns the root of our tree. In this case the root is only a single node.

Listing 25.2: AutoBTSimple.java

```
1  package org.firstinspires.ftc.teamcode;
2
3  import com.ftcteams.behaviortrees.DebugTree;
4  import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
5  import com.qualcomm.robotcore.eventloop.opmode.OpMode;
6  import com.ftcteams.behaviortrees.Node;
7
8  import org.firstinspires.ftc.teamcode.BehaviorTrees.BTSimple;
9
10 @Autonomous
11 public class AutoBTSimple extends OpMode {
12     Node root = BTSimple.root();
13     DebugTree debugTree = new DebugTree();
14     boolean done;
15
16     @Override
17     public void init() {
18     }
19
20     @Override
21     public void loop() {
22         if (!done){
23             Node.State state = root.tick(debugTree, this);
24             if (state != Node.State.RUNNING){
25                 done = true;
26             }
27         }
28     }
29 }
```

Now let's talk through a few of the pieces here.

```
12     Node root = BTSimple.root();
```

Traditionally we call the base of a tree the *root*, so that is what we call this variable.

25. Behavior Trees (another way to handle auto)

```
13 DebugTree debugTree = new DebugTree();
```

This creates an instance of `debugTree` that we can use. We'll talk more about this later.

```
14 boolean done;
```

We create a variable called `done`. In Java, all created `boolean` variables start out as `false` so we don't need to give it an initial value.

```
21 public void loop() {  
22     if (!done){  
23         Node.State state = root.tick(debugTree, this);  
24         if (state != Node.State.RUNNING){  
25             done = true;  
26         }  
27     }  
28 }
```

This simply keeps calling `tick` as long as the `root` tells us it is still running.

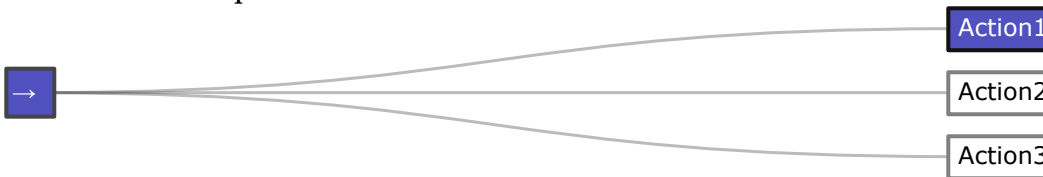
So far, this seems like a lot of work for something unremarkable. Control Flow nodes are where this starts to become powerful.

25.2. Control Flow Nodes

25.2.1. Sequence

A sequence starts with the first child and calls its `tick` until it returns `SUCCESS` or `FAILURE`. If it returns `SUCCESS` then it goes to the next child. If it returns `FAILURE`, then it returns `FAILURE`. If all children return `SUCCESS`, then it returns `SUCCESS`. This is much like steps in a recipe. You don't go on to step 2 until you finish step 1.

Here is an example:



So here is what our behavior tree looks like in code:

Listing 25.3: BehaviorTrees/BTSequenceInline.java

```
1 package org.firstinspires.ftc.teamcode.BehaviorTrees;  
2
```

```

3 import com.ftcteams.behaviortrees.DebugTree;
4 import com.ftcteams.behaviortrees.Node;
5 import com.ftcteams.behaviortrees.Sequence;
6 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
7
8 public class BTSequenceInline {
9     public static class Action1 extends Node{
10         @Override
11         public State tick(DebugTree debug, Object obj) {
12             ((OpMode)obj).telemetry.addData("Action 1", "Success");
13             return State.SUCCESS;
14         }
15     }
16     public static class Action2 extends Node{
17         @Override
18         public State tick(DebugTree debug, Object obj) {
19             ((OpMode)obj).telemetry.addData("Action 2", "Success");
20             return State.SUCCESS;
21         }
22     }
23     public static class Action3 extends Node{
24         @Override
25         public State tick(DebugTree debug, Object obj) {
26             ((OpMode)obj).telemetry.addData("Action 3", "Success");
27             return State.SUCCESS;
28         }
29     }
30
31     public static Node root(){
32         return new Sequence(
33             new Action1(),
34             new Action2(),
35             new Action3());
36     }
37 }

```

You'll notice that now we have three nodes.

```

31 public static Node root(){
32     return new Sequence(
33         new Action1(),
34         new Action2(),
35         new Action3());
36 }

```

Here our root method creates a sequence that has 3 execution nodes.

Now, this starts to get ugly with all of these nodes in the same class so we break them out each into their own files. In real code, you wouldn't name these Action1, Action2, Action3, etc.

25. Behavior Trees (another way to handle auto)

Listing 25.4: BehaviorTrees/Actions/Action1.java

```
1 package org.firstinspires.ftc.teamcode.BehaviorTrees.Actions;
2
3 import com.ftcteams.behaviortrees.DebugTree;
4 import com.ftcteams.behaviortrees.Node;
5 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
6
7 public class Action1 extends Node {
8     @Override
9     public State tick(DebugTree debug, Object obj) {
10         ((OpMode)obj).telemetry.addData("Action 1", "Success");
11         return State.SUCCESS;
12     }
13 }
```

Listing 25.5: BehaviorTrees/Actions/Action2.java

```
1 package org.firstinspires.ftc.teamcode.BehaviorTrees.Actions;
2
3 import com.ftcteams.behaviortrees.DebugTree;
4 import com.ftcteams.behaviortrees.Node;
5 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
6
7 public class Action2 extends Node {
8     @Override
9     public State tick(DebugTree debug, Object obj) {
10         ((OpMode)obj).telemetry.addData("Action 1", "Success");
11         return State.SUCCESS;
12     }
13 }
```

Listing 25.6: BehaviorTrees/Actions/Action3.java

```
1 package org.firstinspires.ftc.teamcode.BehaviorTrees.Actions;
2
3 import com.ftcteams.behaviortrees.DebugTree;
4 import com.ftcteams.behaviortrees.Node;
5 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
6
7 public class Action3 extends Node {
8     @Override
9     public State tick(DebugTree debug, Object obj) {
10         ((OpMode)obj).telemetry.addData("Action 1", "Success");
11         return State.SUCCESS;
12     }
13 }
```

This makes the “tree” file look like this:

Listing 25.7: BehaviorTrees/BTSequence.java

```

1 package org.firstinspires.ftc.teamcode.BehaviorTrees;
2
3 import com.ftcteams.behaviortrees.Node;
4 import com.ftcteams.behaviortrees.Sequence;
5
6 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action1;
7 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action2;
8 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action3;
9
10 public class BTSequence {
11     public static Node root(){
12         return new Sequence(
13             new Action1(),
14             new Action2(),
15             new Action3());
16     }
17 }

```

You'll notice there is almost no change to our Auto (other than which tree we set the root to)

Listing 25.8: AutoBTSequence.java

```

1 package org.firstinspires.ftc.teamcode;
2
3 import com.ftcteams.behaviortrees.DebugTree;
4 import com.ftcteams.behaviortrees.Node;
5 import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
6 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
7
8 import org.firstinspires.ftc.teamcode.BehaviorTrees.BTSequence;
9
10 @Autonomous
11 public class AutoBTSequence extends OpMode {
12     Node root = BTSequence.root();
13     DebugTree debugTree = new DebugTree();
14     boolean done;
15
16     @Override
17     public void init() {
18     }
19
20     @Override
21     public void loop() {
22         if (!done){
23             Node.State state = root.tick(debugTree, this);
24             if (state != Node.State.RUNNING){
25                 done = true;

```

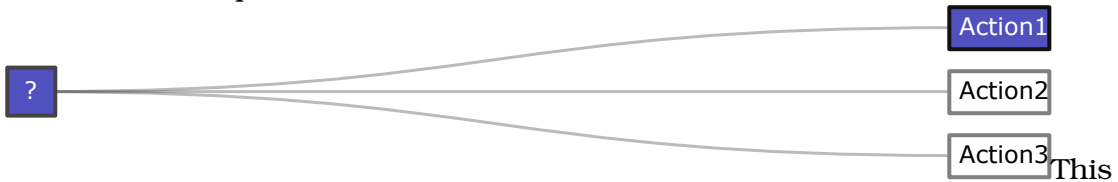
25. Behavior Trees (another way to handle auto)

```
26     }  
27 }  
28 }  
29 }
```

25.2.2. Failover

A failover starts with the first child and calls its tick until it returns SUCCESS or FAILURE. If it returns success, then it returns SUCCESS. If it returns FAILURE, then it goes to the next child. It returns as soon as the first child returns SUCCESS. It returns FAILURE only if all children return FAILURE. And as is probably obvious, if a child returns RUNNING, then it returns RUNNING.

Here is an example:



makes the most sense when you think about that you really want to do action1, but if you can't then do action2. If neither of those work, try action3.

Here is what our behavior tree looks like in code:

Listing 25.9: BehaviorTrees/BTFailover.java

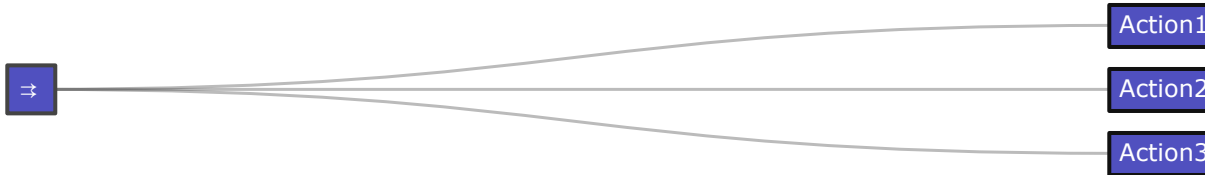
```
1 package org.firstinspires.ftc.teamcode.BehaviorTrees;  
2  
3 import com.ftcteams.behaviortrees.Failover;  
4 import com.ftcteams.behaviortrees.Node;  
5  
6 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action1;  
7 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action2;  
8 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action3;  
9  
10 public class BTFailover {  
11     public static Node root(){  
12         return new Failover(  
13             new Action1(),  
14             new Action2(),  
15             new Action3());  
16     }  
17 }
```

(This reuses the same Action classes) . The only change to our auto code is to use this tree instead of the other one.

25.2.3. Parallel

Parallel returns `SUCCESS` when the number of children return `SUCCESS` that is indicated. This can be all of them or a lesser number. One HUGE thing that is different from `Sequence` is that it ticks each of the children once before returning. (`Sequence` doesn't run the children after it gets a `SUCCESS`.) . If all children return `FAILURE`, then it returns `FAILURE`. (for example, if you call parallel with 2 and all but one children have failed, then it will fail.) If it is still possible, it will return `RUNNING`.

Here is an example:



is what it looks like in code:

Listing 25.10: BehaviorTrees/BTParallel.java

```

1 package org.firstinspires.ftc.teamcode.BehaviorTrees;
2
3 import com.ftcteams.behaviortrees.Parallel;
4 import com.ftcteams.behaviortrees.Node;
5
6 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action1;
7 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action2;
8 import org.firstinspires.ftc.teamcode.BehaviorTrees.Actions.Action3;
9
10 public class BTParallel {
11     public static Node root(){
12         return new Parallel(
13             2,
14             new Action1(),
15             new Action2(),
16             new Action3());
17     }
18 }

```

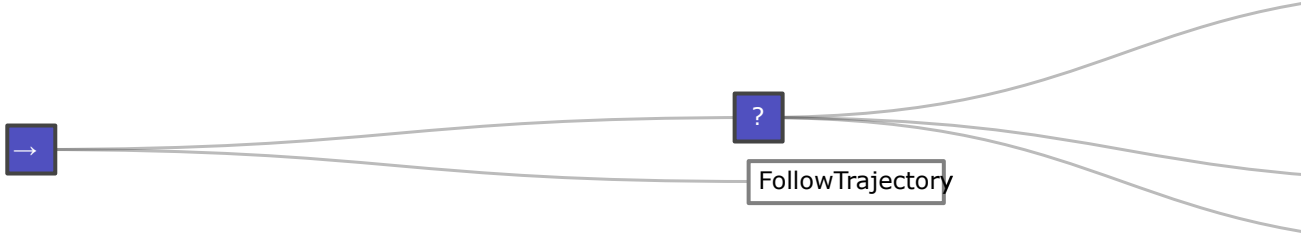
You'll notice that this one takes a number for how many required successes.

25.2.4. Combining these

The strength of this comes when you start combining these various types together. Here is an example from the `CenterStage` game. In it, depending on

25. Behavior Trees (another way to handle auto)

where a game element was placed you had to park in one of three locations. The spike could either be on the LeftSpike, the RightSpike, or the MiddleSpike.



introduces a different shape. For ease of drawing and understanding, we have a special type of execution node that is a conditional. These return only `SUCCESS` or `FAILURE`. (This is really a normal execution node, but it makes the website and packing of our code simpler to make it a special case)

25.2.5. NOT

It turns out we have a `NOT` (sometimes called a decorator instead of a control flow node). It returns `FAILURE` if its child returns `SUCCESS`, `SUCCESS` if its child returns `FAILURE`, and `RUNNING` if its child returns `RUNNING`. This can be useful so that you can reuse actions that you already have instead of needing to write a positive and a negative version.

25.3. Generating and testing your own behavior trees (the easy way)

We have a website that makes it easy to generate and test the logic of your own behavior trees. <http://behaviortrees.ftcteams.com> When you go to this site, you'll see something like this:

25.3. Generating and testing your own behavior trees (the easy way)

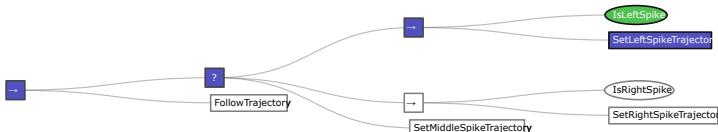
The screenshot shows the ETC Behavior Trees website. On the left, there's a sidebar with a tree diagram. The central part has a 'Num: 10072' field, a 'Tree: Sample' dropdown, and an 'UPDATE' button. The right part is a code editor showing a Java code snippet for a behavior tree sample.

For this website, you enter text on the left hand side, enter your team number and the name of the tree in the center and press Update. That will update both the code on the right hand side (that you can copy/paste directly into your project) as well as make the graph below. You can “Download SVG” to download an image of the tree (perhaps for your portfolio or other documentation), the ? tells you how to navigate, and load tree lets you load the tree from a file.

As you enter in text, use ? for Failover, -> for sequential, =2 for parallel, ! for not, and put conditionals in parenthesis and Actions in square brackets. After you have generated, any nodes that aren't visited will be white. If it is green, that means that it is returning SUCCESS, red means that it is returning FAILURE, and blue means that it is returning RUNNING. For actions that are active, clicking on them cycles through the states. For conditionals, clicking on them changes them from returning SUCCESS to returning FAILURE. As you do this, you'll notice that which nodes that are active changes.

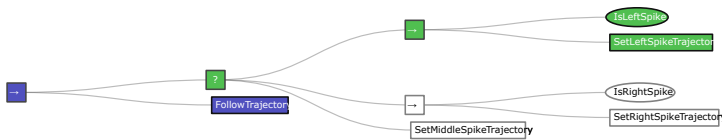
One thing to note is that if you continue to another line, you need a | symbol for each indentation you plan for it to have so that the website knows how to line things up.

For example, let's say that the condition IsLeftSpike returns SUCCESS, you get the tree looking like this:



You can see that by clicking on isLeftSpike, now it is waiting on SetLeft-SpikeTrajectory to return SUCCESS or FAILURE. If we select on it to return SUCCESS, we get a tree looking like this:

25. Behavior Trees (another way to handle auto)



The cool part about this is that we can have team members test out our logic before any of the code has been written (or even before the robot is being built.)

Now, you have to write each of the execution nodes (actions and conditionals) but hopefully you design these so they are simple to write and test.

25.4. Exercises

There are no exercises for this chapter other than implementing on your own robot.

A. Making your own Programming Board

The ProgrammingBoard has a number of electrical components:

- REV Expansion Hub (<http://www.revrobotics.com/rev-31-1153/>)
- REV Potentiometer (<http://www.revrobotics.com/rev-31-1155/>)
- REV Color Sensor (<http://www.revrobotics.com/rev-31-1557/>)
- REV Touch Sensor (<http://www.revrobotics.com/rev-31-1425/>)
- REV 40:1 HD Hex Motor (<http://www.revrobotics.com/rev-41-1301/>)
- REV SRS Servo (<http://www.revrobotics.com/rev-41-1097/>)

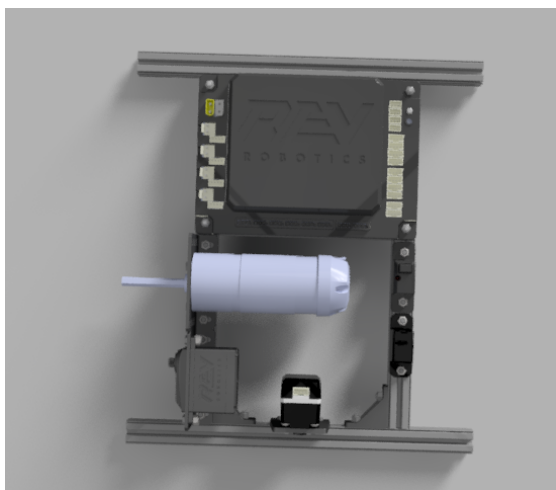
It should be connected in the following way:

- REV 40:1 HD Hex Motor - Power and encoder to Motor 0
- REV Potentiometer - connected to Analog/Digital 0:1
- REV Color Sensor - connected to I2C 1
- REV Touch Sensor - connected to Analog/Digital 2:3
- REV SRS Servo - connected to Servo 0

Here is an example CAD from one of my students¹ of a way to assemble it using all mechanical parts from the REV FTC Kit.

¹Thanks, Eric!!

A. *Making your own Programming Board*



B. LinearOpMode

B.1. What is it?

LinearOpMode is a class derived from OpMode that instead of having the five methods of an OpMode has only one. runOpMode(). Everything then occurs in that method. You are now responsible to update telemetry whenever you want it sent to the driver station, waiting for the Start button to be pressed, and checking to see if the opModeIsActive()

Here is our HelloWorld as a LinearOpMode

Listing B.1: HelloWorldLinear.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class HelloWorldLinear extends LinearOpMode {
8
9     @Override
10    public void runOpMode(){
11        telemetry.addData("Hello", "World");
12        telemetry.update();
13        waitForStart();
14        while (opModeIsActive()) {
15        }
16    }
17 }
```

So you can compare, here it is again from [chapter 1](#)

Listing B.2: HelloWorld.java

```
1 package org.firstinspires.ftc.teamcode;
2
3 import com.qualcomm.robotcore.eventloop.opmode.OpMode;
4 import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
5
6 @TeleOp()
7 public class HelloWorld extends OpMode {
```

B. LinearOpMode

```
8      @Override
9      public void init() {
10          telemetry.addData("Hello", "World");
11      }
12
13      @Override
14      public void loop() {
15
16      }
17 }
```

B.2. Should you use it?

I think that you are better off using OpMode instead of LinearOpMode but since a lot of the sample code and many (most?) teams do I think it is worth elaborating here why that is my opinion so you can make your own decision. There are teams I highly respect that use LinearOpMode so even if you disagree we can still be friends. :-)

B.2.1. Benefits of LinearOpMode

The reason LinearOpMode exists is that it allows code to be written that is more similar to how code is often taught. Instead of using state machines like we did in [chapter 12](#), it allows simple code like:

```
...
board.setMotorSpeed(0.5);
while(!board.touchSensorPressed()){
}
board.setMotorSpeed(0.0);
...
```

as opposed to code like:

```
...
switch(state){
    case State.BEGIN:
        board.setMotorSpeed(0.5);
        state = State.WAIT_FOR_TOUCH;
        break;
    case State.WAIT_FOR_TOUCH:
        if(board.touchSensorPressed){
            state = State.STOP;
        }
        break;
}
```

```
case State.STOP:
    board.setMotorSpeed(0.0);
    break;
...
```

The other large benefit is much of the sample code available online is written this way.

B.2.2. Drawbacks of LinearOpMode

1. LinearOpMode is derived from OpMode. If you look at the implementation of LinearOpMode, the start() method creates a thread and calls the user class runOpMode(). This means you have now introduced another thread into the system. Instead of variables like gamepad being updated between calls to your OpMode, they could be updated at anytime.
2. Your code is all in one main control method instead of being broken out into logical methods for the five methods in the OpMode. For both OpMode and LinearOpMode you should use class methods to break your code out into logical pieces to make it easier to read and maintain. Many professional programmers get nervous whenever a method is longer than fits on one screen.
3. You also are no longer protected from a loop taking too long so you don't respond in time to the driver station.
4. State machines are typically used in commercial embedded projects. Why not choose to learn how to do that now?

C. Sample Solutions

I have gotten feedback that the sample solutions here were rarely used and they added a lot to the cost for printing the physical book so in order to be able to add content and keep the price the same, I have removed them from the text and they are now in the github repository instead. It is located at: <https://github.com/alan412/LearnJavaForFTC>

D. Credits

Thanks to the following people that provided feedback on earlier versions of the book to make it better. It is better for them, but I bear the responsibility for any and all errors. If you have comments, please put them in at <https://github.com/alan412/LearnJavaForFTC/issues/new/choose>

- Karen (FTC #18175 - Team Techies)
- Joshua (FTC #16072 - Quantum Quacks)
- Eli (FTC #8569 - RoboKnights)
- Teja (alumnae of FTC #16072 - Quantum Quacks)
- Dan (FTC #10273 - The Cat in the Hat Comes Back)
- Abigail (alumnae of FRC #3459 - Team PyroTech)
- Ellie (FTC #8569 - RoboKnights)
- Ryan (alumna of FTC #16072 - Quantum Quacks)
- Burton (FTC #11214 - Ground Shakers)
- Michael (alumna of FTC #4634 - FROGbots)
- Jason (FTC #18291 - Mech Warriors)
- Roy Brabson (FTC #7083 - Tundrabots)
- Ovies Brabson (FTC #7083 - Tundrabots)
- Sebastien Erives (FTC #12887 - Devolt Phobos)
- Michael Hoogasian

Thanks to FTC Team 14169 - PartyTime Carets for giving some photos of their TSE to be used in the Computer Vision chapter.

Index

`%`, 19
`( and )`, 19
`*`, 19
`+`, 19
`-`, 19
`/`, 19
`=`, 19
`@Autonomous()`, 4
`@Disabled()`, 4
`@Override`, 5
`@Teleop()`, 4

A

`abstract`, 97
`addData`, 5
Analog Sensors, 63
`AnalogInput`, 64
`AngleUnit`, 75
annotation, 4
AprilTags, 109, 182
Arcade Drive, 141
`ArrayList`, 92
Arrays, 91
assignment operators, 20

B

Bang Bang, 189
Blinkin, 170
boolean, 14
byte, 14

C

Cartesian, 148

char, 14
class, 2, 5
Class Members, 29
Class Methods, 30
Classes, 29
Color sensor, 67
`ColorSensor`, 68
Comments, 8
conditional operators, 22
Configuration file, 41
Constructors, 32
Control Theory, 185

D

`DcMotor`, 51
`DcMotorSimple.Direction.FORWARD`,
56
`DcMotorSimple.Direction.REVERSE`,
56
`DcMotor.ZeroPowerBehavior.BRAKE`,
56
`DcMotor.ZeroPowerBehavior.FLOAT`,
56
Dead band, 191
`DigitalChannel`, 44
Distance Sensor, 67
`DistanceSensor`, 69
`DistanceUnit`, 70
double, 14
Driver Station, 2
Driving, 139

INDEX

E

EasyOpenCV, 109
else, 23
enum, 86
Enumerated types, 84
extends, 95

F

false, 14
Field oriented driving, 147
Field relative driving, 147
final, 134
float, 14
for, 26
for..each, 91
FTC SDK, 131

G

Gamepad, 17
getMaxVoltage(), 65
getRuntime(), 86
GoBilda Pinpoint, 162
Gotchas, 10
Gyro, 73
gyro, 147

H

HardwareMap, 44
hasa, 96
holonomic drive, 142
Hysteresis, 191

I

if, 21
implements, 136
IMU, 73, 75, 147
Inheritance, 95
init, 5
init(), 3
init_loop(), 3
INPUT, 44

int, 14

Isa, 96

J

Javadoc, 127

L

LED, 167
LED Stick, 168
Limelight 3A, 175
LinearOpMode, 215
long, 14
loop(), 3

M

Math, 18
math, 17
Math class, 133
mecanum, 142
Mechanisms, 43
method, 2
Motors, 49

O

OpenCV, 109
OpMode, 3
OUTPUT, 44

P

package, 2
Parameters, 31
PID(f), 194
Pinpoint, 162
Polar, 148
polymorphism, 97
potentiometer, 63
private, 34
Programming Board, 1, 213
protected, 34
public, 5, 34

R

Range, 65
 resetRunTime(), 86
 Return Types, 31
 REV Digital LED Indicator, 167
 Robot Controller, 1
 rotation sensor, 53
 Rumble, 105
 RUN_TO_POSITION, 52
 RUN_USING_ENCODER, 52
 RUN_WITHOUT_ENCODER, 52

S

scale(), 65
 Scope, 16, 34
 Servo.Direction.FORWARD, 60
 Servo.Direction.REVERSE, 60
 Servos, 59
 servo.scaleRange(), 60
 servo.setDirection(), 60
 servo.setPosition(), 60
 setDirection(), 56
 setPower(), 52
 setZeroBehavior(), 56
 short, 14

Sparkfun QWIIC LED Stick, 168

start(), 3
 State, 77
 static, 39
 stop(), 3
 STOP_AND_RESET_ENCODER, 52
 String, 15
 super(), 98
 switch, 81

T

Tank Drive, 139, 141
 telemetry, 5, 135
 telemetry.addLine, 104
 toString(), 33
 true, 14
 Tuning your PID, 195

V

Variables, 13
 Vision, 109
 VisionPortal, 109
 void, 5

W

while, 25